

Dragan Milićev

Objektno orijentisano programiranje na jeziku C++

Skripta sa praktikumom

Dragan Milićev: Primeri

Ljubica Lazarević: Zadaci sa rešenjima

Jelena Marušić: Uputstvo za Microsoft Visual C++



Povežite se s Mikro knjigom



Web prezentacije

- ❖ **Mikro knjiga** Upoznajite se sa sadržajem naših knjiga. Pogledajte šta Mikro knjiga sprema novo. Pretplatite se na naša izdanja. Pregledajte oblasti koje vas zanimaju. Posetite našu prezentaciju na adresi www.mikroknjiga.com
- ❖ **Časopis Mikro** Pregledajte sadržaj najnovijeg broja časopisa Mikro. Posetite našu prezentaciju na adresi www.mikro.co.yu
- ❖ **Računarski rečnik Mikro knjige** Potražite termine iz različitih oblasti računarske tehnike i informatike u našem rečniku na adresi www.mk.co.yu/recnik



Dopune i ispravke

- ❖ Na adresi www.mikroknjiga.co.yu/dopune.html nalaze se dopune i ispravke naših izdanja.



Elektronska pošta

- ❖ Pišite redakciji Mikro knjige ili redakciji časopisa Mikro na adresu pisma@mikroknjiga.co.yu, odnosno pisma@mk.co.yu.



Elektronski bilten

Bilteni su jednosmerne liste slanja preko kojih prijavljenim učesnicima periodično šaljemo obaveštenja elektronskom poštom.

- ❖ **Bilten Mikro knjige** Ako želite da primате vesti o novim izdanjima Mikro knjige i o pogodnostima nabavke naših izdanja, pošaljite pismo na adresu mkna1og@eunet.yu. U pismo (ne u polje Subject) upišite subscriptbe mikbi11ten
- ❖ **MikroVesti** Ako želite da primате kratke vesti iz oblasti računarstva i informacionih tehnologija, pošaljite prazno pismo na adresu mi-krovesti-subscrib@mk.co.yu

Predgovor

Objektno orijentisano programiranje na jeziku C++, skripta sa praktikumom

Urednik	Olga Milanko
Koordinator projekta	Aleksandra Stojanović
Lektor i korektor	Vesna Đukić
Tehnički urednik	Sanja Tasić
Prelom teksta	Sanja Tasić
Obrada slika	Jasmina Arsić
Korice	Sanja Tasić
	Miodrag Ivanišević
Izdavač	Mikro knjiga, Beograd
Direktor	Dragan Tanaskoši
Štampa	Foto futura, Beograd
Ako imate pitanja ili komentare, ili ako želite da dobijete besplatan katalog, pišite nam ili se javite:	
Mikro knjiga	Mikro knjiga
P. fah 20-87	Jevrejska bb
11030 Beograd	78000 Banja Luka
tel: 011/3340-344	tel: 051/220-960
pi.sma@mi.kroknj.ig.a.co.yu	pi.sma@mi.kroknj.ig.a.hr

Copyright©2001 Mikro knjiga. Sva prava zadržana. Nije dozvoljeno da nijedan deo ove knjige bude snimljen, emitovan ili reprodukovan na bilo koji način, uključujući, ali ne i ograničavajući se na fotokopiranje, fotografiju, magnetni upis ili bilo koji drugi vid zapisa, bez prethodne pismene dozvole izdavača.

Mikro knjiga je nastojala da u ovoj knjizi razdvoji zaštićene znakove i njihove vlasnike od opisnih termina, koristeći način ispisivanja upotrebljen od strane proizvođača.

Svi napori su učinjeni da se u ovoj knjizi ne pojave greške. Mikro knjiga ne može prihvatiti bilo kakvu odgovornost za eventualne greške u izloženoj materiji, kao ni za njihove posledice.

CIP - Katalogizacija u publikaciji
Narodna biblioteka Srbije, Beograd

519.682/683

MILICEV, Dragan
Objektno orijentisano programiranje na jeziku C++: skripta sa praktikumom / Dragan Milicev, Ljubica Lazarević, Jelena Marušić. - Beograd: Mikro knjiga, 2001 (Beograd: Foto futura). X, 206 str.: graf. prikazi, 24 cm

Sadržaj s masl. str.: Primeri / Dragan Milicev; Zadaci sa rešenjima / Ljubica Lazarević; Uputstvo za Microsoft Visual C++ / Jelena Marušić.

ISBN 86-7555-160-6
1. Lazarević, Ljubica 2. Marušić, Jelena
519.682.7.C++
a) Objektno orijentisano programiranje b) Programski jezik, C++
ID=89334540

CPPS/143/D4 5 4

Skripta *Objektno orijentisano programiranje na jeziku C++* nastala su kao prateći materijal za kurseve koje je autor držao na Elektrotehničkom fakultetu u Beogradu i na drugim mestima. Skripta predstavljaju izbor i sažetak materijala iz autorove istoimene knjige (izdavač Mikro knjiga, Beograd, 1995.) koji je namenjen za nastavu u okviru odgovarajućih kurseva. Skripta su namenjena studentima i svim ostalim polaznicima kurseva kao priručnik, a ne kao referentni kompletni izvor gradiva. One se zbog toga ne mogu smatrati potpuno autonomnim udžbenikom, jer se čvrsto oslanjaju na detaljna objašnjenja koja se daju na predavanjima ili se nalaze u pomenutoj knjizi. Međutim, kako pomenuta knjiga sadrži detaljna objašnjenja svih elemenata jezika C++, koja dobrim delom nisu neophodna za praksu, u ova skripta su uvršteni samo delovi koje je autor smatrao bitnim i potrebnim u praksi. Osim toga, neki elementi su dorade i korigovani.

U cilju bolje podrške studentima i ostalim polaznicima, skripta su proširena *Praktikumom* koji sadrži mnoštvo primera sa objašnjenjima, zadatka za vežbu sa rešenjima, kao i kratko uputstvo za korišćenje popularnog integrisanog razvojnog okruženja Microsoft Visual C++. Autori veruju da će ovi prilozi znatno pomoći čitaocima u savladavanju najvažnijih elemenata objektnog programiranja i jezika C++, kao i da će obezbediti početne uslove za rad u jednom modernom razvojnom okruženju za programiranje.

Autor još jednom želi da naglasi da se ovo izdanje ne može smatrati sasvim kompletnom i referentnom knjigom. Skripta su zamišljena kao koncizan i uredan materijal koji prati usmene izlaganja na kursovima ili pomenutu knjigu. Autorov cilj je bio da se brzo obezbedi lako dostupan materijal, koji bi olakšao praćenje predavanja, ali i izvor materijala za vežbanje koji bi dopunio već objavljenu knjigu. Zbog toga ovo izdanje ima nekoliko nedostataka koji ga i kvalifikuju samo kao skriptu. Prvo, tekst se sastoji od kratkih teza koje samo navode elementarne iskaze, uglavnom bez detaljnih objašnjenja. Drugo, ovo izdanje nema elemente koje bi trebalo da ima jedna potpuna knjiga: indeks, rečnik i spisak literature. Najzad, ovaj materijal nije recenziran.

Autor i koautor su uložili maksimalne napore da ovaj materijal ne sadrži greške. Ipak, sve sugestije, korekcije i primedbe čitalaca više su nego dobrodošle.

U Beogradu, januara 2001.
Dragan Milicev

Sadržaj

Predgovor

v

Skripta

1

Glava 1 Uvod	5
Zašto OOP?	5
Šta daju OOP i C++ kao odgovor?	6
Šta se menja uvođenjem OOP-a?	6
Glava 2 Pregled osnovnih koncepata OOP-a na jeziku C++	9
Klase	9
Konstruktori i destruktori	10
Nasledivanje	11
Polimorfizam	12
Glava 3 Pregled osnovnih koncepata nasleđenih iz jezika C	15
Ugrađeni tipovi i deklaracije	15
Pokazivači	15
Nizovi	16
Izrazi	17
Naredbe	18
Funkcije	19
Struktura programa	20
Glava 4 Elementi jezika C++ koji nisu objektno orijentisani	21
Oblast važenja imena	21
Objekti i lvrčnosti	22
Životni vek objekata	23
O konverziji tipova	24
Konstante	25
Dinamički objekti	27
Reference	28
Funkcije	30
Deklaracije funkcija i prenos argumenata	30
Neposredno ugrađivanje u kôd	30
Podrazumevane vrednosti argumenata	31
Preklapanje imena funkcija	31
Operatori i izrazi	32

Glava 5 Klase

Klase, objekti i članovi klase	35
Pojam i deklaracija klase	35
Pokazivač this	35
Primerci klase	36
Konstantne funkcije članice	37
Ugnežđivanje klase	38
Strukture	39
Zajednički članovi klase	39
Zajednički podaci članovi	39
Zajedničke funkcije članice	40
Prijatelj klase	41
Prijateljske funkcije	41
Prijateljske klase	42
Konstruktori i destruktori	43
Pojam konstruktora	43
Kada se poziva konstruktor?	43
Načini pozivanja konstruktora	43
Konstruktor kopije	45
Destruktor	45

Glava 6 Preklapanje operatora

Pojam preklapanja operatora	47
Operatorске funkcije	47
Osnovna pravila	48
Bočni efekti i veze između operatora	49
Operatorске funkcije kao članice i globalne funkcije	49
Unarni i binarni operatori	50
Neki posebni operatori	51
Operatori new i delete	51
Konstruktor kopije i operator dodele	52
Osnovni standardni ulazno/izlazni tokovi	53
Klase istream i ostream	53
Ulazno/izlazne operacije za korisničke tipove	54

Glava 7 Nasledivanje

Izvedene klase	55
Šta je nasledivanje i šta su izvedene klase?	55
Kako se definišu izvedene klase u jeziku C++?	56
Prava pristupa	56
Konstruktori i destruktori izvedenih klasa	57
Polimorfizam	59
Šta je polimorfizam?	59
Virtuelne funkcije	59
Dinamičko vezivanje	60
Virtuelni destruktori	61
Nizovi i izvedene klase	62
Apstraktne klase	63

Višestruko nasledivanje

Šta je višestruko nasledivanje?	63
Virtuelne osnovne klase	63
Privatno i zaštićeno izvođenje	64
Šta je privatno i zaštićeno izvođenje?	64
Semantička razlika između privatnog i javnog izvođenja	65

Glava 8 Uvod u objektno modelovanje

Apstraktni tipovi podataka	67
Projektovanje apstraktnih tipova podataka	67
Modelovanje strukture – klase i osnovne relacije između klasa	68
Klasa, atributi i operacije	71
Asocijacija	72
Zavisnost	72
Nasledivanje (generalizacija/specijalizacija)	74
Modelovanje ponašanja – interakcije	75

Praktikum**77****Primeri****79**

Glava 1 Pregled osnovnih koncepata OOP na jeziku C++	81
Glava 2 Pregled osnovnih koncepata nasleđenih iz jezika C	85
Glava 3 Elementi jezika C++ koji nisu objektno orijentisani	87
Glava 4 Klase i preklapanje operatora	93
Glava 5 Nasledivanje	109
Glava 6 Uvod u objektno modelovanje	119

Zadaci za vežbu**139**

Glava 7 Elementi jezika C++ koji nisu objektno orijentisani	141
Glava 8 Klase i preklapanje operatora	155
Glava 9 Nasledivanje	171

Integrirano razvojno okruženje Microsoft Visual C++ Uputstvo za korišćenje

Glava 10 Uvod	183
Osnovni pojmovi	185
Radno okruženje	185
Prozor radnog prostora	186
Glava 11 Opšte manipulacije projektima	187
Započinjanje novog projekta	187
Rad sa više projekata	188
Osnovna podešavanja	189
Glava 12 Uređivanje programa	191
Dodavanje fajlova u projekat	191
Osnovni alati	192
Class Wizard	192
Uređivanje koda	195
Sintaksno bojenje	196
Glava 13 Prevođenje programa	199
Prevođenje	199
Pravljenje projekta	200
Dobijanje pomoći u radu	200
Podešavanja	200
Glava 14 Debagovanje	203
Pripremanje aplikacije za debagovanje	203
Pokretanje aplikacije u režimu debagovanja	204
Prozori debagera i pregled objekata	204
Tačke prekida	205
Komande za izvršavanje korak po korak	205

Dragan Milićev

Objektno orijentisano programiranje na jeziku C++

Skripta

Skripta

Glava 1

Uvod

Glava 2

Pregled osnovnih koncepata OOP-a
na jeziku C++

Glava 3

Osnovni koncepti jezika C++ nasleđeni
iz jezika C

Glava 4

Elementi jezika C++ koji nisu
objektno orijentisani

Glava 5

Klase

Glava 6

Preklapanje operatora

Glava 7

Nasledivanje

Glava 8

Uvod u objektno modelovanje

Glava 1

Uvod

- Jezik C++ je objektno orijentisani programski jezik opšte namene. Veliki deo jezika C++ nasleđen je iz jezika C, pa C++ predstavlja (uz minimalne izuzetke) nadskup jezika C.
- Ovaj kurs uvodi u osnovne koncepte objektno orijentisanog programiranja korišćenjem jezika C++.

Zašto OOP?

- Objektno orijentisano programiranje (engl. *Object Oriented Programming, OOP*) je odgovor na tzv. krizu softvera. OOP pruža načine za rešavanje nekih problema softverske proizvodnje.
- Softverska kriza je posledica sledećih problema u proizvodnji softvera:
 1. Zahtevi korisnika su se *drastično* povećali. Za ovo su uglavnom „krivi“ sami programeri: oni su korisnicima pokazali šta sve računari mogu, i da mogu mnogo više nego što korisnik može da zamisli. Kao odgovor, korisnici su počeli da traže mnogo više nego što su programeri mogli da postignu.
 2. Zato je bilo neophodno povećati produktivnost programera da bi se odgovorilo na zahteve korisnika. To je moguće ostvariti najpre povećanjem broja ljudi u timu. Tradicionalno proceduralno programiranje dopuštalo je projektovanje softvera u modulima sa relativno jakom interakcijom, a jaka interakcija između delova softvera koga pravi mnogo ljudi može da stvori probleme u razvoju softvera.
 3. Produktivnost se može povećati i tako što se neki delovi softvera, koji su ranije već negde korišćeni, mogu ponovo iskoristiti, bez mnogo ili imalo dorade. Tradicionalni način programiranja nije omogućavao laku ponovnu upotrebu koda (engl. *software reuse*).
 4. Drastično su povećani i troškovi održavanja. Trebalo je naći način da projektovani softver bude čitljiviji i lakši za nadogradnju i modifikovanje. Primer: često se dešava da ispravljanje jedne greške u programu generiše mnogo novih problema; potrebno je lokalizovati realizaciju nekog dela tako da se promene u realizaciji ne šire po ostatku sistema.
- Proceduralno programiranje nije moglo da odgovori na ove probleme, pa je nastala kriza proizvodnje softvera. Zato je OOP došlo kao odgovor.

Šta daju OOP i C++ kao odgovor?

- C++ je trenutno najpopularniji objektno orijentisani programski jezik. Osnovna rešenja koja pruža OOP, a C++ podržava, jesu:
 1. Apstraktni tipovi podataka (engl. *abstract data types*). Kao što u C-u ili nekom drugom jeziku postoje ugrađeni tipovi podataka (`int`, `float`, `char`, ...), u jeziku C++ korisnik može proizvoljno definisati svoje tipove i potpuno ravnopravno ih koristiti (`Complex`, `Point`, `Disk`, `Printer`, `Jabuka`, `BankovniRacun`, `Klijent` itd.). Korisnik može kreirati proizvoljan broj primera tog tipa i izvršiti operacije nad njima (engl. *multiple instances*, višestruke instance ili pojave).
 2. Enkapsulacija (engl. *encapsulation*). Realizacija nekog tipa može (i treba) da se sakrije od ostatka sistema (od onih koji ga koriste). Korisnicima tipa treba precizno definisati samo šta se sa tipom može raditi, a način *kako* se to radi sakriva se od korisnika (definise se interno).
 3. Nasledjivanje (engl. *inheritance*). Pretpostavimo da je već formiran tip `Printer` koji ima operacije kao što su `printLine`, `lineFeed`, `formFeed`, `gotoxy` itd. i da je njegovim korišćenjem već realizovana velika količina softvera. Novost je da je firma nabavila i štampače koji imaju bogat skup stilova štampe i valja ih ubuduće iskoristiti. Nepotrebno je iz početka praviti novi tip štampača ili prepravljati stari kod. Dovoljno je kreirati novi tip `PrinterWithFonts` koji je „baš kao i običan“ štampač, samo još može da „menja stilove štampe. Novi tip će naslediti sve osobine starog, ali će i moći da uradi još ponešto.
 4. Polimorfizam (engl. *polymorphism*). Pošto je `PrinterWithFonts` već ionako `Printer`, i ostatak programa treba da ga posmatra kao i običan štampač, sve dok mu nisu potrebne nove mogućnosti štampača. Ranije napisani delovi programa koji koriste tip `Printer` ne moraju se uopšte prepravljati, oni će jednako dobro raditi i sa novim tipom. Pod određenim uslovima, stari delovi ne moraju se čak ni ponovo prevesti! Osobina novog tipa da se „odaziva“ na pravi način, iako ga je korisnik „prozvao“ kao da je stari tip, naziva se *polimorfizam*.
- Sve navedene osobine mogu se, na ovaj ili onaj način, pojedinačno realizovati i u proceduralnom jeziku (kakav je i C), ali je realizacija svih koncepata ili teška, ili sasvim nemoguća. U svakom slučaju, realizacija nekog od ovih principa u proceduralnom jeziku drastično povećava troškove i smanjuje čitljivost programa.
- Jezik C++ podržava sve navedene koncepte, oni su ugrađeni u sam jezik.

Šta se menja uvođenjem OOP-a?

- Jezik C++ nije „čisti“ objektno orijentisani programski jezik (engl. *Object-Oriented Programming Language*, *OOP*) koji bi korisnika „materao“ da ga koristi na objektno orijentisani (OO) način. C++ može da se koristi i kao „malo bolji C“, ali se time ništa ne dobija (čak se i gubi). C++ treba koristiti kao sredstvo za OOP i kao smenicu za razmišljanje. C++ ne sprečava pisanje loših programa, već samo omogućava pisanje mnogo boljih programa.
- OOP uvodi drugačiji način razmišljanja u programiranje!

- U OOP-u, mnogo više vremena troši se na *projekovanje*, a mnogo manje na samu implementaciju (kodovanje).
- U OOP-u, razmišlja se najpre o *problemu*, a tek naknadno o programskom rešenju.
- U OOP-u, razmišlja se o delovima sistema (objektima) koji nešto rade, a ne o tome kako se nešto radi (algoritmima). Drugim rečima, OOP prvenstveno koristi *objektnu dekompoziciju* umesto isključivo *algoritamske dekompozicije*.
- U OOP-u, pažnja se prebacuje sa realizacije na međusobne veze između delova. Teži se što većoj redukciji i strogoj kontroli tih veza. Cilj OOP-a je da smanji interakciju između softverskih delova.

Pregled osnovnih koncepata OOP-a na jeziku C++

- U ovoj glavi biće dat kratak i sasvim površan pregled osnovnih koncepata OOP koje podržava C++. Potpuna i precizna objašnjenja koncepata biće data kasnije, u posebnim glavama.
- Primeri koji se koriste u ovoj glavi samo su pokazni, nisu pravljeni da budu upotrebljivi. Iz realizacije primera izbačeno je sve što bi smanjivalo preglednost osnovnih ideja. Zato su primeri često i nekompletni.
- Čitalac ne treba da se trudi da posle čitanja ove glave zapamti sintaksu rešenja, niti da ot- krije sve pojedinosti pokazanih primera. Cilj je da čitalac samo stekne predstavu o os- novnim idejama OOP-a i jezika C++, da vidi šta je novo i šta se sve može uraditi, kao i da nauči da razmišlja na novi, objektni način.

Klase

- Klasa (engl. *class*) je osnovna organizaciona jedinica programa u OOP jezicima, pa i u jeziku C++. Klasa predstavlja strukturu u koju su grupisani podaci i funkcije:

```
/* Deklaracija klase: */
class Osoba {
public:
    void kosi();
private:
    char *ime;
    int god;
};

/* Svaka funkcija se mora i definisati: */
void Osoba::kosi() {
    cout<<"Ja sam "<<ime<<" i imam "<<god<<" godina.\n";
}
```

- Klasom se definiše novi, korisnički tip za koji se mogu formirati instance (primerci, promenljive). Klasa može da predstavlja realizaciju apstrakcije iz domena problema.
- Instance klase nazivaju se *objekti* (engl. *objects*). Svaki objekat ima sopstvene elemente koji su navedeni u deklaraciji klase. Ovi elementi klase nazivaju se *članovi* klase (engl. *class members*). Članovima se pristupa pomoću operatora „." (tačka):

```
/* Korišćenje klase Osoba: */
/* negde u programu se definišu objekti tipa Osoba, */
Osoba Pera, mojOtac, direktor;
```

```
/* a onda se oni koriste: */
```

```
Pera.kosi(); /* poziv funkcije kosi objekta Pera */
mojOtac.kosi(); /* poziv funkcije kosi objekta mojOtac */
direktor.kosi(); /* poziv funkcije kosi objekta direktor */
```

- Ako pretpostavimo da su ranije, na neki način, postavljene vrednosti članova svakog od navedenih objekata, ovaj segment programa daje:

```
Ja sam Petar Markovic i imam 25 godina.
Ja sam Slobodan Milicev i imam 58 godina.
Ja sam Aleksandar Simic i imam 40 godina.
```

- Specifikator `public`: govori prevodiocu da su samo članovi koji se nalaze iza njega dostupni spoja. Ovi članovi nazivaju se *javnim*. Članovi iza specifikatora `private`: su nedostupni korisnicima klase (ali ne i članovima klase) i nazivaju se *privatnim*.

```
/* Izvan članova klase nije moguće: */
Pera.ime="Petar Markovic"; /* nedozvoljeno */
mojOtac.god=55; /* takođe nedozvoljeno */
```

```
/* Šta bi tek bilo da je ovo dozvoljeno: */
direktor.ime="Du....., kr.....";
/* a onda ga neko pita (što je dozvoljeno): */
direktor.kosi();
/* ?! */
```

Konstruktori i destruktori

- Da bi se omogućila inicijalizacija objekta, u klasi se definiše posebna funkcija koja se implicitno (automatski) poziva kada objekat nastaje. Ova funkcija se naziva *konstruktor* (engl. *constructor*) i nosi isto ime kao i klasa.

```
class Osoba {
public:
    Osoba(char *ime, int godine); /* konstruktor */
    void kosi(); /* funkcija: predstavi se! */
private:
    char *ime; /* podatak: ime i prezime */
    int god; /* podatak: koliko ima godina */
};

/* Svaka funkcija se mora i definisati: */

void Osoba::kosi() {
    cout<<"Ja sam "<<ime<<" i imam "<<god<<" godina.\n";
}
```

```
Osoba::Osoba(char *i, int g) {
    if (proveriTime(i)) /* proveri ime da nije ružno */
        ime=i;
    else
        ime="necu da ti kazem ko";
    god=((g>0 && g<=100)?g:0); /* proveri godine */
}
```

```
/* Korišćenje klase Osoba sada je: */
Osoba Pera("Petar Markovic", 25), /* poziv konstruktora Osoba */
mojOtac("Slobodan Milicev", 58);
```

```
Pera.kosi();
mojOtac.kosi();
```

- Ovakav deo programa može dati ranije navedene rezultate.

- Moguće je definisati i funkciju koja se poziva uvek kada objekat prestaje da živi. Ova funkcija se naziva *destruktor*.

Nasledivanje

- Pretpostavimo da nam je potreban novi tip, Maloletnik. Maloletnik je „jedna vrsta“ osobe, koja „poseduje sve što i osoba, samo ima još nešto“, tj. ima staratelja. Ovakva relacija između klasa naziva se *nasledivanje*.

- Kada nova klasa predstavlja „jednu vrstu“ druge klase (engl. *a-kind-of*), kaže se da je ona izvedena iz osnovne klase.

```
class Maloletnik : public Osoba {
public:
    Maloletnik(char *ime, char *staratelj, int godine); /* konstruktor */
    void kojeOdgovoran();
private:
    char *staratelj;
};
```

```
void Maloletnik::kojeOdgovoran() {
    cout<<"Ja mene odgovara "<<staratelj<<"\n";
}
```

- Maloletnik: Maloletnik (char *i, char *s, int g) : Osoba(i,g), staratelj(s) {}
- Izvedena klasa Maloletnik ima sve članove kao i osnovna klasa Osoba, ali ima još i članove staratelj i kojeOdgovoran. Konstruktor klase Maloletnik definiše da se objekat ove klase inicijalizuje zadavanjem imena, staratelja i godina, i to tako da se konstruktor osnovne klase Osoba (koji inicijalizuje ime i godine) poziva sa odgovarajućim argumentima. Sam konstruktor klase Maloletnik samo inicijalizuje staratelja.

- Sada se mogu koristiti i nasleđene osobine objekata klase Maloletnik a na raspolaganju su i njihova posebna svojstva kojih nije bilo u klasi Osoba:

```
Osoba otac("Petar Petrovic",40);
Maloletnik dete("Milan Petrovic","Petar Petrovic",12);

otac.kosi();
dete.kosi();
dete.kojeOdgovoran();
otac.kojeOdgovoran();          /* ovo, naravno, ne može! */

/* Izlaz će biti:
Ja sam Petar Petrovic i imam 40 godina.
Ja sam Milan Petrovic i imam 12 godina.
Za mene odgovara Petar Petrovic.
*/
```

Polimorfizam

- Pretpostavimo da nam je potrebna nova klasa žena, koja je „jedna vrsta“ osobe, samo što još ima i devojčko prezime. Klasa žena biće izvedena iz klase Osoba.
- I objekti klase žena treba da se „odazivaju“ na funkciju kosi, ali je teško pretpostaviti da će jedna dama otvoreno priznati svoje godine. Zato objekat klase žena treba da ima funkciju kosi, samo što će ona izgledati malo drugačije, svojstveno izvedenoj klasi žena:

```
class Osoba {
public:
    Osoba(char* ime, int godine); /* konstruktor */
    virtual void kosi();          /* virtuelna funkcija */
protected:
    char* ime;                   /* dostupno nasleđenicima */
    int god;                      /* podatak: ime i prezime */
};

void Osoba::kosi() {
    cout<<<"Ja sam "<<ime<<" i imam "<<god<<" godina.\n";
}

Osoba::Osoba (char *i, int g) : ime(i), god(g) {}

class Zena : public Osoba {
public:
    Zena(char* ime, char* devojacko, int godine);
    virtual void kosi();          /* nova verzija funkcije kosi */
private:
    char *devojacko;
};

Zena::Zena (char *i, char *d, int g) : Osoba(i,g), devojacko(d) {}

void Zena::kosi() {
    cout<<<"Ja sam "<<ime<<" devojacko prezime "<<devojacko<<".\n";
}
```

- Funkcija članica koja će u izvedenim klasama imati nove verzije deklarise se u osnovnoj klasi kao *virtuelna funkcija* (virtual). Izvedena klasa može da dá svoju definiciju virtuelne funkcije, ali i ne mora. U izvedenoj klasi ne mora se navoditi reč virtual.
- Da bi članovi osnovne klase Osoba bili dostupni izvedenoj klasi žena, ali ne i korisnicima spolja, oni se deklaršu iza specifikatora protected: i nazivaju *zaštićenim* članovima.
- Drugi delovi programa, korisnici klase Osoba, ako su dobro projektovani, ne vide nikakvu promenu zbog uvođenja izvedene klase. Oni upšte ne moraju da se menjaju:

```
/* Funkcija "ispitaj" propituje osobe i ne mora da se menja: */

void ispitaj (Osoba *hejTi) {
    hejTi->kosi();
}

/* U drugom delu programa koristimo novu klasu žena: */

Osoba otac("Petar Petrovic",40);
Zena majka("Milka Petrovic","Mitrovic",35);
Maloletnik dete("Milan Petrovic","Petar Petrovic",12);

ispitaj(&otac);          /* pozvaće se Osoba::kosi() */
ispitaj(&majka);          /* pozvaće se Zena::kosi() */
ispitaj(&dete);           /* pozvaće se Osoba::kosi() */

/* Izlaz će biti:
Ja sam Petar Petrovic i imam 40 godina.
Ja sam Milka Petrovic, devojacko prezime Mitrovic.
Ja sam Milan Petrovic i imam 12 godina.
*/
```

- Funkcija ispitaj dobija pokazivač na tip Osoba. Kako je i žena osoba, C++ dozvoljava da se pokazivač na tip žena (&majka) *konvertuje* (pretvori) u pokazivač na tip Osoba (hejTi). Mehanizam virtuelnih funkcija obezbeđuje da funkcija ispitaj, preko pokazivača hejTi, pozove pravu verziju funkcije kosi. Zato će se za argument &majka pozivati funkcija Zena::kosi, a za argument &otac funkcija Osoba::kosi. Za argument &dete takode će se pozvati funkcija Osoba::kosi, jer klasa Maloletnik nije redefinisala virtuelnu funkciju kosi.
- Navedeno svojstvo da se odaziva prava verzija funkcije klase čiji su naslednici dali nove verzije naziva se *polimorfizam* (engl. *polymorphism*).

Pregled osnovnih koncepata nasleđenih iz jezika C

- Ovo poglavlje predstavlja pregled nekih osnovnih koncepata jezika C++ nasleđenih iz jezika C, tradicionalnog jezika za strukturano, proceduralno programiranje.
- Kao u prethodnom poglavlju, detalji su izostavljeni, a prikazani su samo najvažniji delovi jezika C.

Ugrađeni tipovi i deklaracije

- Ugrađeni tipovi nisu realizovani kao klase, već kao jednostavne strukture podataka.
- *Deklaracija* je iskaz koji uvodi neko ime u program. Ime se može koristiti samo ako je prethodno deklarirano. Deklaracija govori prevodiocu kojoj jezičkoj kategoriji pripada neko ime i šta se sa tim imenom može raditi.
- *Definicija* je ona deklaracija koja stvara objekat (alocira memorijski prostor za njega) ili daje telo funkcije.
- Neki osnovni ugrađeni tipovi su: ceo broj (`int`), znak (`char`) i racionalni broj (`float` i `double`). Objekat može biti inicijalizovan u deklaraciji; takva deklaracija je i definicija:

```
int i;
int j=0, k=3;
float f1=2.0, f2=0.0;
double PI=3.14;
char a='a', nul='0';
```

Pokazivači

- *Pokazivač* je objekat koji *ukazuje* na neki drugi objekat. Pokazivač se realizuje tako što sadrži adresu objekta na koji ukazuje.
- Ako pokazivač `p` ukazuje na objekat `x`, onda je rezultat operacije `*p` objekat `x` (operacija dereferenciranja pokazivača).
- Rezultat operacije `&x` je pokazivač koji ukazuje na objekat `x` (operacija uzimanja adrese).

- Tip „pokazivač na tip T“ označava se sa T*. Na primer:

```
int i=0, j=0; // objekti i i j tipa int;
int *pi; // objekat pi je tipa "pokazivač na int" (tip: int*);
pi=&i; // vrednost pokazivača pi je adresa objekta i,
// pa pi ukazuje na i;
*pi=2; // *pi označava objekat i; i postaje 2;
j=*pi; // j postaje jednak objektu na koji ukazuje pi,
// a to je i;
pi=&j; // pi sada sadrži adresu j, tj. ukazuje na j;
```

- Na isti način se mogu definisati pokazivači na proizvoljan tip. Ako je p pokazivač koji ukazuje na objekat klase sa članom m, onda je (*p).m isto što i p->m:

```
Osoba otac("Petar Simić", 40); // objekat otac klase Osoba;
Osoba *po; // po je pokazivač na tip Osoba;
po=&otac; // po ukazuje na objekat otac;
(*po).kosi(); // poziv funkcije kosi objekta otac;
po->kosi(); // isto što i (*po).kosi();
```

- Tip na koji pokazivač ukazuje može biti proizvoljan, pa i drugi pokazivač:

```
int i=0, j=0; // i i j tipa int;
int *pi=&i; // pi je pokazivač na int, ukazuje na i;
int **ppi; // ppi je tipa "pokazivač na - pokazivač na - int";
ppi=&pi; // ppi ukazuje na pi;
*pi=i; // pi ukazuje na i, pa i postaje 1;
**ppi=2; // ppi ukazuje na pi, pa je rezultat operacije *ppi objekat pi;
// rezultat još jedne operacije * je objekat na koji ukazuje
// pi, a to je i; i postaje 2;
*pi=j; // pi ukazuje na pi, pa pi sada ukazuje na j,
// a ppi još uvek na pi;
ppi=&i; // greška: ppi je pokazivač na pokazivač na int,
// a ne pokazivač na int!
```

- Pokazivač tipa void* može ukazivati na objekat bilo kog tipa. Ne postoje objekti tipa void, ali postoje pokazivači tipa void*.

- Pokazivač koji ima posebnu vrednost 0 ne ukazuje ni na jedan objekat. Ovakav pokazivač se razlikuje od bilo kog drugog pokazivača koji ukazuje na neki objekat. Može se ispitati da li pokazivač ukazuje na neki objekat ili ne (tj. da li je jednak 0 ili ne).

Nizovi

- Niz je objekat koji sadrži nekoliko objekata nekog tipa. Niz je, kao i pokazivač, izvedeni tip. Tip „niz objekata tipa T“ označava se sa T[].

- Niz se deklarise na sledeći način:

```
int a[100]; // a je objekat tipa "niz objekata tipa int" (tip: int[]);
// sadrži 100 elemenata tipa int;
```

- Ovaj niz ima 100 elemenata koji se indeksiraju od 0 do 99; i+1-vi element je a[i]:

```
a[2]=5; // treći element niza a postaje 5
a[0]=a[0]+a[99];
```

- Elementi mogu biti bilo kog tipa, pa čak i nizovi. Na ovaj način se definišu višedimenzionalni nizovi:

```
int m[5][7]; // m je niz od 5 elemenata;
// svaki element je niz od 7 elemenata tipa int;
m[3][5]=0; // pristupa se četvrtom elementu niza m;
// on je niz elemenata tipa int;
// pristupa se zatim njegovom šestom elementu i on postaje 0;
```

- Nizovi i pokazivači su blisko povezani u jezicima C i C++. Sledeća tri pravila povezuju nizove i pokazivače:

1. Svaki put kada se ime niza koristi u nekom izrazu, osim u operaciji uzimanja adrese (operator &), implicitno se konvertuje u pokazivač na svoj prvi element. Na primer, ako je a tipa int [], onda se on konvertuje u tip int*, sa vrednošću adrese prvog elementa niza (to je početak niza).
2. Definisan je operacija sabiranja pokazivača i celog broja, ako su zadovoljeni sledeći uslovi: pokazivač ukazuje na element nekog niza i rezultat sabiranja je opet pokazivač koji ukazuje na element istog niza ili za jedno mesto iza poslednjeg elementa niza. Rezultat sabiranja p+i, gde je p pokazivač a i ceo broj, jeste pokazivač koji ukazuje i elementa iza elementa na koji ukazuje pokazivač p. Ako navedeni uslovi nisu zadovoljeni, rezultat operacije je nedefinisan. Analognu pravila postoje za operacije oduzimanja celog broja od pokazivača, kao i inkrementiranja i dekrementiranja pokazivača.
3. Operacija a[i] je, po definiciji, ekvivalentna sa *(a+i).

- Na primer:

```
int a[10]; // a je niz objekata tipa int;
int *p=&a; // p ukazuje na a[0];
a[2]=1; // a[2] je isto što i *(a+2); a se konvertuje u pokazivač
// koji ukazuje na a[0]; rezultat sabiranja je pokazivač
// koji ukazuje na a[2]; dereferenciranje tog pokazivača (*)
// predstavlja zapravo a[2]; a[2] postaje 1;
p[3]=3; // p[3] je isto što i *(p+3), a to je a[3];
p-p+1; // p sada ukazuje na a[1];
*(p+2)=1; // a[3] postaje sada 1;
p[-1]=0; // p[-1] je isto što i *(p-1), a to je a[0];
```

Izrazi

- Izraz je iskaz u programu koji sadrži operande (objekte, funkcije ili literale nekog tipa), operacije nad tim operandima i proizvodi rezultat tačno definisanog tipa. Operacije se zadržaju pomoću operatora ugrađenih u jezik.

- Operator može da prihvata jedan, dva ili tri operanda strogo definisanih tipova, i proizvodi rezultat koji se može koristiti kao operand nekog drugog operatora. Na ovaj način se formiraju složeni izrazi.

- Prioritet operatora definiše redosled izračunavanja operacija unutar izraza. Podrazumevani redosled izračunavanja može se promeniti pomoću zagrada ().

- C i C++ su veoma bogati operatorima. Zapravo, najveći deo obrade u jednom programu definisan je izrazima.

- Mnogi ugrađeni operatori imaju sporedni efekat: pored toga što proizvode rezultat, oni menjaju vrednost nekog od svojih operandi.
- Postoje operatori za inkrementiranje (++) i dekrementiranje (--), u prefiksnoj i postfixnoj formi. Ako je i nekog od numeričkih tipova ili pokazivač, ++ znači „inkrementiraj i, a kao rezultat vrati njegovu staru vrednost“, ++i znači „inkrementiraj i a kao rezultat vrati njegovu novu vrednost“. Slično važi za dekrementiranje.
- Vrednosti se dodeljuju pomoću operatora dodele =: a=b znači „dodeli vrednost izraza b objektu a, a kao rezultat vrati tu dodeljenu vrednost“. Ovaj operator grupiše zdesna ulivo. Tako:


```
a=b=c; // dodeli c objektu b i vrati tu vrednost, a onda je dodeli objektu a;
      // prema tome, c je dodeljen i objektu b i objektu a;
```
- Postoji i operator složene dodele: a+=b znači isto što i a=a+b, ali se izraz a samo jednom izračunava:


```
a+=b; // isto što i a=a+b;
      a-=b; // isto što i a=a-b;
      a*=b; // isto što i a=a*b;
      a/=b; // isto što i a=a/b;
```

Naredbe

- *Naredba* podrazumeva neku obradu ali ne proizvodi rezultat kao izraz. Postoji samo nekoliko naredbi u jezicima C i C++.
- Deklaracija se sintakšno smatra naredbom. Izraz je takode jedna vrsta naredbe. *Složena naredba* (ili *blok*) je sekvenca naredbi uokvirena velikim zagradama {}. Na primer:


```
{
    int a, c=0, d=3; // početak složene naredbe (bloka);
    a=(c++)+d; // deklaracija kao naredba;
    int i=a; // izraz kao naredba;
    i++; // deklaracija kao naredba;
    // izraz kao naredba;
} // kraj složene naredbe (bloka);
```
- *Uslovna naredba* (if naredba): if (izraz) *naredba* else *naredba*. Prvo se izračunava izraz; njegov rezultat mora biti numeričkog tipa ili pokazivač. Ako je rezultat različit od nule (što se čini kao „tačno“), izvršava se prva *naredba*. Ako je rezultat jednak nuli (što se čini kao „netačno“), izvršava se druga *naredba* (else deo). Deo else je opcion:


```
if (a++) b=a; // inkrementiraj a; ako je a bilo različito od 0,
              // dodeli novu vrednost a objektu b;
if (c) a=c; // ako je c različito od 0, dodeli ga objektu a,
else a=c+1; // inače dodeli c+1 objektu a;
```
- *Petlja* (for naredba): for (inicijalna *naredba* izraz1; izraz2) *naredba*. Ovo je petlja sa izlaskom na vrhu (petlja tipa *while*). Prvo se, samo jednom pre ulaska u petlju, izvršava inicijalna *naredba*. Zatim se izvršava petlja. Pre svake iteracije izračunava se izraz1; ako je njegov rezultat jednak nuli, izlazi se iz petlje; inače, izvršava se iteracija petlje. Iteracija se sastoji od izvršavanja *naredbe* i zatim izračunavanja izraza2. Oba izraza i

inicijalna *naredba* su opcion. Ako se *izraz1* izostavi, uzima se da je njegova vrednost jednaka 1. Na primer:

```
for (int i=0; i<100; i++) {
    //... Ova petlja se izvršava tačno 100 puta
}
for (;;) {
    //... Beskonačna petlja
}
```

Funkcije

- Funkcije su jedina vrsta potprograma u jezicima C i C++. Funkcije mogu biti članice klase ili globalne funkcije (nisu članice nijedne klase).
- Ne postoji statičko (sintaktičko) ugnežđivanje tela funkcija. Dozvoljeno je dinamičko ugnežđivanje poziva funkcija, kao i rekurzija.
- Funkcija može, ali ne mora da ima argumente. Funkcija bez argumenta deklarise se sa praznim zagradama. Argumenti se prenose samo po vrednostima u jeziku C, a mogu se prenositi i po referenci u jeziku C++.
- Funkcija može da vraća rezultat, ali i ne mora. Funkcija koja nema povratnu vrednost deklarise se sa tipom void kao tipom rezultata.
- Deklaracija funkcije koja nije i definicija uključuje samo zaglavlje sa tipom argumenta i rezultat; imena argumenta su opciona i nemaju značaja za program:


```
int stringCompare (char*, char*); // deklaracija globalne funkcije;
                                // prima dva argumenta tipa char*,
                                // a vraća tip int;
                                // globalna funkcija bez argumenta
                                // koja nema povratnu vrednost;
```
- Definicija funkcije daje i telo funkcije. Telo funkcije je složena naredba (blok):


```
int Counter::inc () { // definicija funkcije članice; vraća int;
    return count++; // vraća se rezultat izraza;
}
```
- Funkcija može vratiti vrednost koja je rezultat izraza u naredbi return.
- Unutar tela funkcije (tačnije unutar svakog ugnežđenog bloka) mogu se deklarirati lokalna imena:


```
int Counter::inc () {
    int temp; // temp je lokalni objekat
    temp=count+1; // count je član klase Counter
    count=temp;
    return temp;
}
```
- Funkcija članica neke klase može pristupiti članovima sopstvenog objekta bez posebne specifikacije. Globalna funkcija mora navesti objekat čijem članu pristupa.

- Funkcija se poziva pomoću operatora (). Rezultat ove operacije je rezultat poziva funkcije:

```
int f(int); // deklaracija globalne funkcije
Counter c; // objekat c klase Counter
int a=0, b=1; // poziv funkcije c.inc koji vraća int
a=b+c.inc(); // poziv globalne funkcije f
a=f(b);
```

- Može se deklarirati i pokazivač na funkciju:

```
int f(int); // f je tipa "funkcija koja prima jedan argument tipa int"
// i vraća int";
int (*p)(int); // p je tipa
// "pokazivač na funkciju
// koja prima jedan argument tipa int
// i vraća int";
// p ukazuje na f;
p=&f;
int a;
a=(*p)(1); // poziva se funkcija na koju ukazuje p, a to je funkcija f;
```

Struktura programa

- Program se sastoji samo od deklaracija (klasa, objekata, ostalih tipova i funkcija). Sva obrada koncentrisana je unutar tela funkcija.
- Program se fizički deli na odvojene jedinice prevođenja – datoteke. Datoteke se prevode odvojeno i nezavisno, a zatim se povezuju u izvršni program. U svakoj datoteci se moraju deklarirati sva imena pre nego što se koriste.
- Zavisnosti između modula-datoteke definišu se pomoću *datoteka-zaglavlja*. Zaglavljia sadrže deklaracije svih entiteta koji se koriste u datom modulu, a definisani su u nekom drugom modulu. Zaglavljia (.h) se uključuju u tekst datoteke koja se prevodi (.cpp) pomoću direktive #include.
- Glavni program (izvor toka kontrole) definiše se kao obavezna funkcija main. Primer jednostavnog, ali kompletnog programa:

```
class Counter {
public:
    Counter();
    int inc(int by);
private:
    int count;
};

Counter::Counter () : count(0) {}

int Counter::inc (int by) {
    return count++by;
}

void main () {
    Counter a,b;
    int i=0, j=3;
    i=a.inc(2)+b.inc(++j);
}
```

Glava 4

Elementi jezika C++ koji nisu objektivno orijentisani

Oblast važenja imena

- Onaj deo teksta programa u kome se deklarirano ime može koristiti naziva se *oblast važenja imena* (engl. *scope*).
- Globalna imena su imena koja se deklariraju van svih funkcija i klasa. Njihova oblast važenja je deo teksta od mesta deklaracije do kraja datoteke.
- Lokalna imena su imena deklarirana unutar bloka, uključujući i blok tela funkcije. Njihova oblast važenja je od mesta deklariranja, do završetka bloka u kome su deklarirane.

```
int x; // globalni x

void f () {
    int x; // lokalni x, sakriva globalni x;
    x=1; // pristup lokalnom x
    {
        int x; // drugi lokalni x, sakriva prethodnog
        x=2; // pristup drugom lokalnom x
    }
    x=3; // pristup prvom lokalnom x
}
```

```
int *p=&x; // uzimanje adrese globalnog x
```

- Globalnom imenu se može pristupiti, iako je sakriveno, navođenjem operatora „::“ ispred imena:

```
int x; // globalni x

void f () {
    int x=0; // lokalni x
    ::x=1; // pristup globalnom x;
}
```

- Za formalne argumente funkcije smatra se da su lokalni, deklarirani u krajnje spoljašnjem bloku tela funkcije:

```
void f (int x) {
    int x; // pogrešno
}
```

- Prvi izraz u naredbi `for` može da bude definicija promenljive. Tako se dobija lokalna promenljiva za blok u kome se nalazi `for`:

```
{
    for (int i=0; i<10; i++) {
        //...
        if (a[i]==x) break;
        //...
    }
    if (i==10) // može se pristupiti imenu i
}
```

- Oblast važenja klase imaju svi članovi klase. To su imena deklarisanu unutar deklaracije klase. Imenu koje ima oblast važenja klase, izvan te oblasti može se pristupiti preko operatora „`::`“, „`->`“, gde je levi operand objekat, odnosno pokazivač na objekat date klase ili klase izvedene iz date klase; ili preko operatora „`::`“, gde je levi operand ime klase:

```
class X {
public:
    int x;
    void f();
};

void X::f () { /*...*/ }
X xx;
xx.x=0;
xx.f(); // može i ovako
```

- Oblast važenja funkcije imaju samo labelle (za gotovo naredbe). One se mogu navesti bilo gde (i samo) unutar tela funkcije, a vide se u celoj funkciji.

Objekti i lvrrednosti

- Objekat je neko područje u memoriji podataka tokom izvršavanja programa. To može biti eksplicitno definisana promenljiva (globalna ili lokalna), ili privremeni objekat koji nastaje pri izračunavanju izraza. Uopšte, objekat je primerak nekog tipa (ugrađenog ili klase), ali ne i funkcija.
- U jeziku C++ samo se nekonstantni objekat naziva promenljivom.
- *lvrrednost* (engl. *lvredness*) je izraz koji upućuje na objekat. *lvred* je kovanica od „nešto što može da stoji sa leve strane znaka dodele vrednosti“, iako ne mogu sve lvrrednosti da stoje sa leve strane znaka „`=`“, npr. konstanta.
- Za svaki operator se definiše da li zahteva kao operand lvrrednost, i da li vraća lvrrednost kao rezultat. „Početna“ lvrrednost je ime objekta ili funkcije. Na taj način se rekurzivno definišu lvrrednosti.
- Promenljiva lvrrednost (engl. *modifiable lvredness*) je ona lvrrednost, koja nije ime funkcije, ime niza, ili konstantni objekat. Samo ovakva lvrrednost može biti levi operand operatora dodele.

- Primeri lvrrednosti:

```
int i=0; // i je lvrrednost, jer je ime koje upućuje
        // na objekat - celobrojnu promenljivu u memoriji

int *p=&i; // i p je ime, odnosno lvrrednost

*p=7; // *p je lvrrednost, jer upućuje na objekat koga
      // predstavlja ime i; rezultat operacije * je
      // lvrrednost

int *q[100];
*q[a+13]=7; // *q[a+13] je lvrrednost
```

Životni vek objekata

- Životni vek objekta je vreme postojanja (u memoriji) tog objekta tokom izvršavanja programa. Za to vreme se objektu može pristupiti.
- Na početku životnog veka, objekat se inicijalizuje (poziva se njegov konstruktor ako je to primerak klase), a na kraju se objekat ukida (poziva se njegov destruktor ako je to primerak klase). Zbog toga se nastanak (ili kreiranje) objekta čvrsto vezuje za njegovu inicijalizaciju.
- U odnosu na životni vek, postoje automatski, statički, dinamički i privremeni objekti.
- Životni vek *automatskog* objekta (lokalni objekat koji nije deklarisan kao `static`) traje od nastanka na njegovu definiciju, do napuštanja oblasti važenja tog objekta. Automatski objekat nastaje iznova pri svakom pozivu bloka u kome je deklarisan. Definicija objekta klase je izvršna naredba jer uzrokuje poziv njegovog konstruktora u vreme izvršavanja.
- Životni vek *statičkih* objekata (globalni i lokalni `static` objekti) traje od izvršavanja njihove definicije do kraja izvršavanja programa. Globalni statički objekti se inicijalizuju samo jednom, na početku izvršavanja programa, pre korišćenja bilo koje funkcije ili objekta iz iste datoteke, ne obavezno pre poziva funkcije `main`, a prestaju da žive po završetku funkcije `main`. Lokalni statički objekti počinju da žive pri prvom nastanku toka programa na njihovu definiciju.

- Primer:

```
int glob=5; // globalni objekat; životni vek mu
            // traje do kraja programa;

void f () {
    int lok=2; // lokalni objekat; životni vek mu je do
               // izlaska iz spoljnog bloka funkcije;
    static int sl=3; // lokalni statički objekat; oblast
                    // važenja je funkcija, a životni vek je ceo
                    // program; inicijalizuje se samo jednom;
    for (int i=0; i<sl; i++) {
        int j=1; // j je lokalni za for blok
        //...
    }
}
```

• Primer:

```
int a=1;

void f () {
    int b=1;
    static int c=1; // inicijalizuje se pri svakom pozivu
    // inicijalizuje se samo jednom
    printf(" a = %d ",a++);
    printf(" b = %d ",b++);
    printf(" c = %d\n",c++);
}

void main () {
    while (a<4) f();
}

// izlaz će biti:
// a = 1 b = 1 c = 1
// a = 2 b = 1 c = 2
// a = 3 b = 1 c = 3
```

- Životni vek *dinamičkih* objekata neposredno kontroliše programer. Oni nastaju operacijom new, a nestaju operacijom delete.
- Životni vek *privremenih* objekata je kratak i nedefinisan. Ovi objekti nastaju pri izračunavanju izraza, za odlaganje međurezultata ili privremeno smeštanje vraćene vrednosti funkcije. Najčešće se uništavaju čim više nisu potrebni.
- Životni vek članova klase je isti kao i životni vek objekta kome pripadaju.
- Formalni argumenti funkcije, pri pozivu funkcije, nastaju kao lokalni automatski objekti i inicijalizuju se stvarnim argumentima. Dakle, na mestu ulaska u funkciju, u trenutku poziva funkcije, nastaju formalni argumenti kao lokalni automatski objekti i inicijalizuju se stvarnim argumentima. Semantika ove inicijalizacije formalnog argumenta ista je kao i inicijalizacija bilo kog objekta u definiciji.
- Rezultat poziva funkcije predstavlja objekat koji na mestu poziva funkcije, u trenutku povratka iz funkcije, nastaje kao bezimени privremeni objekat i koji se inicijalizuje izrazom iza naredbe return. Semantika te inicijalizacije potpuno je ista kao i inicijalizacija objekta u definiciji.

```
x f (X x1) { // U trenutku poziva iz main(), kao da se radi: X x1=x3;
    ...
    return x2;
}

void main () {
    ...f(x3)... // U trenutku povratka, kao da se radi: X temp = x2;
}
```

O konverziji tipova

- C++ je strogo tipizirani jezik, što je u duhu njegove objektno orijentacije.
- Tipizacija znači da svaki objekat ima tačno određen tip. Svaki put kada se na nekom mestu očekuje objekat jednog tipa, a koristi se objekat drugog tipa, potrebno je izvršiti *konverziju* tipova.

- Konverzija tipa znači pretvaranje objekta datog tipa u objekat potrebnog tipa.
- Slede slučajevi kada se može desiti da se očekuje jedan tip, a dostavlja se drugi, odnosno kada je potrebno vršiti konverziju:

1. operatori za ugrađene tipove zahtevaju operande odgovarajućeg tipa;
2. neke naredbe (if, for, do, while, switch) zahtevaju izraze odgovarajućeg tipa;
3. pri pozivu funkcije, kada su stvarni argumenti drugačijeg tipa od deklarisanih formalnih argumenata;
4. pri povratku iz funkcije, ako se u izrazu iza return koristi izraz drugačijeg tipa od deklarisanog tipa povratne vrednosti funkcije;
5. pri inicijalizaciji objekta jednog tipa pomoću objekta drugog tipa. Slučaj 3 se može svesti u ovu grupu, jer se formalni argumenti inicijalizuju stvarnim argumentima pri pozivu funkcije. I slučaj 4 se može svesti u ovu grupu, jer se privremeni objekat, koji prihvata vraćenu vrednost funkcije na mestu poziva, inicijalizuje izrazom iza naredbe return.

- Konverzija tipa može biti ugrađena u jezik (standardna konverzija) ili je definiše korisnik (programer) za svoje tipove (korisnička konverzija).
- Standardne konverzije su, na primer, konverzije iz tipa int u tip float, ili iz tipa char u tip int, konverzija pokazivača na izvedenu klasu u pokazivač na osnovnu klasu itd.
- Prevodilac može izvršiti konverziju koja mu je dozvoljena, na mestu gde je to potrebno; ovakva konverzija naziva se *implicitnom*. Programer može navesti koja konverzija treba da se izvrši; ova konverzija se naziva *eksplicitnom*.
- Jedan način zahtevanja eksplicitne konverzije je pomoću operatora cast: (tip) izraz.
- Primer:

```
char f(float i, float j) {
    //...
}

int k=f(5.5,5); // najpre se konvertuje float(5),
                // a posle i vraćena vrednost
                // iz char u int
```

Konstante

- Konstantni tip je izvedeni tip koji se iz nekog osnovnog tipa dobija stavljanjem specifikatora const u deklaraciju:
- ```
const float pi=3.14;
const char plus='+';
```
- Konstantni tip ima sve osobine osnovnog tipa, samo se objekti konstantnog tipa ne mogu menjati. Pristup konstantama kontroliše se u fazi prevodenja, a ne izvršavanja.
  - Konstanta mora da se inicijalizuje pri definisanju.
  - Konstante mogu da se koriste u konstantnim izrazima koje prevodilac treba da izračuna tokom prevodenja, na primer kao dimenzije nizova.

- Pokazivač na konstantu definiše se stavljanjem reči `const` ispred cele definicije. Konstantni pokazivač definiše se stavljanjem reči `const` ispred samog imena:

```
const char *pk="asdfgh"; // pokazivač na konstantu
pk[3]='a'; // pogrešno
pk="qwerty"; // ispravno

char *const kp="asdfgh"; // konstantni pokazivač
kp[3]='a'; // pogrešno
kp="qwerty"; // ispravno

const char *const kpk="asdfgh"; // konstantni pokazivač na konstantu
kpk[3]='a'; // pogrešno
kpk="qwerty"; // pogrešno
```

- Prevodilac kontrolise upotrebu pokazivača na konstantne objekte (inicijalizaciju i dodavanje), tako da se ne može povrediti „obecana“ konstantnost:

```
const char* p = "...";
char* q = p; // greška, jer bi inače bilo dozvoljeno:
*q = ...;

const char* r = q; // dozvoljeno,
// jer ovo znači da se preko r neće menjati objekat,
// koji se inače može menjati preko q;
*r = ...; // greška!
(char)r = ...; // može uz ekspllicitnu konverziju;
```

- Navođenjem reči `const` ispred deklaracije formalnog argumenta funkcije koji je pokazivač, obezbeđuje se da funkcija ne može da menja objekat na koji taj argument ukazuje:

```
char *strcpy(char *p, const char *q); // ne može da promeni *q
```

- Na ovaj način prevodilac kontrolise „obecanja“ programera da se u nekom delu programa (funkciji) neki objekti neće menjati:

```
void f1(const char*);
void f2(char*);
char* s1 = ...;
const char* s2 = ...;

void f() {
 f1(s1); // u redu;
 f2(s1); // u redu;
 f1(s2); // greška!
 f2(s2); // greška!
}
```

- Navođenjem reči `const` ispred tipa koji vraća funkcija, definiše se da će privremeni objekat koji nastaje od vraćene vrednosti funkcije biti konstantan, i njegovu upotrebu konstantni prevodilac. Za vraćenu vrednost koja je pokazivač na konstantu, ne može se preko vraćenog pokazivača menjati objekat:

```
const char* f(); // greška!
*f()='a';
```

- Preporuka je da se umesto tekstualnih konstanti koje se ostvaruju preprocesorom (kao u jeziku C) koriste konstante.
- Dosljedno korišćenje konstanti u programu obezbeđuje podršku prevodioca u sprečavanju grešaka – korektnost konstantnosti.

## Dinamički objekti

- Operator `new` pravi dinamički objekat, a operator `delete` uništava dinamički objekat nekog tipa T.
- Operator `new` za svoj operand ima identifikator tipa i eventualne argumente konstruktora. Operator `new` alokira potreban prostor u slobodnoj memoriji za objekat datog tipa, a zatim poziva konstruktor tipa sa zadatim vrednostima. Operator `new` vraća pokazivač na dati tip:

```
Complex *pc1 = new Complex(1.3, 5.6);
Complex *pc2 = new Complex(-1.0, 0);
*pc1 = *pc1 + *pc2;
```

- Objekat formiran pomoću operatora `new` naziva se dinamički objekat, jer mu je životni vek poznat tek u vreme izvršavanja programa. Ovakav objekat nastaje kada se izvrši operator `new`, a traje sve dok se ne ukine operatorom `delete` (može da traje i po završetku bloka u kome je nastao):

```
Complex *pc;

void f() {
 pc = new Complex(0.1, 0.2);
}

void main() {
 f();
 delete pc; // uklanjanje objekta *pc
}
```

- Operator `delete` ima jedan operand koji je pokazivač na neki tip. Ovaj pokazivač mora da ukazuje na objekat nastao pomoću operatora `new`. Operator `delete` poziva destruktora za objekat na koji ukazuje pokazivač, a zatim oslobađa zauzeti prostor. Ovaj operator vraća `void`.
- Operatorom `new` može se napraviti i niz objekata nekog tipa. Ovakav niz ukida se operatorom `delete` sa parom uglatih zagrada:

```
Complex *pc = new Complex[10];
//...
delete [] pc;
```

- Kada se alokira niz, nije moguće zadati inicijalizatore. Ako klasa nema definisan konstruktor, prevodilac obezbeđuje podrazumevanu inicijalizaciju. Ako klasa ima konstruktore, da bi se alokirao niz, potrebno je da postoji konstruktor koji se može pozvati bez argumenta.
- Kada se alokira niz, operator `new` vraća pokazivač na prvi element alociranog niza. Sve dimenzije niza, osim prve, treba da budu konstantni izrazi. Prva dimenzija može da bude i promenljivi izraz, ali takav da može da se izračuna u trenutku izvršavanja naredbe sa operatorom `new`.

## Reference

- U jeziku C argumenti su prenošeni funkciji isključivo po vrednosti (engl. *call by value*). Da bi neka funkcija mogla da promeni vrednost neke spoljne promenljive, trebalo je preneti pokazivač na tu promenljivu.
- U jeziku C++ moguć je i prenos po referenci (engl. *call by reference*):

```
void f(int i, int &j) {
 // i se prenosi po vrednosti, j po referenci
 // stvarni argument se neće promeniti
 i++;
 j++;
}
```

```
void main () {
 int si=0, sj=0;
 f(si, sj);
 cout<<"si="<<si<<"", sj="<<sj<<"\n";
}
```

```
// Izlaz će biti:
// si=0, sj=1
```

- C++ ide još dalje; u ovom jeziku postoji izvedeni tip *reference* na objekat (engl. *reference*). Reference se deklaršu upotrebom znaka & ispred imena.
- Reference je alternativno ime objekta. Kada nastaje, referenca mora da se inicijalizuje objektom na koji će upućivati. Od tada referenca postaje sinonim za objekat na koji upućuje. Svaka operacija nad referencom (uključujući i operaciju dodele) predstavlja, u stvari, operaciju nad referenciranim objektom:

```
int i=1;
int &j=i;
i=3;
j=5;
int *p=&j;
j+=1;
int k=j;
int m=*p;
```

- Referenca se realizuje kao (konstantni) pokazivač na objekat. Ovaj pokazivač pri inicijalizaciji dobija vrednost adrese objekta kojim se inicijalizuje. Svako dalje obraćanje referenci podrazumeva posredni pristup objektu preko ovog pokazivača. Nema načina da se, posle inicijalizacije, vrednost ovog pokazivača promeni.
- Referenca liči na pokazivač, ali se posredan pristup do objekta preko pokazivača vrši operatorom \*, a preko reference bez oznaka. Uzimanje adrese (operator &) reference znači uzimanje adrese objekta na koji ona upućuje.

- Primeri:

```
int &j = *new int(2); // j upućuje na dinamički objekat 2
int *p=&j; // p je pokazivač na isti objekat
(*p)++; // objekat postaje 3
j++; // objekat postaje 4
delete &j; // isto kao i delete p
```

- Ako je referenca tipa reference na konstantu, onda se referencirani objekat ne sme promeniti posredstvom te reference.
- Referenca može i da se vrati kao rezultat funkcije. U tom slučaju, funkcija treba da vrati referencu na objekat koji traje (živi) i posle izlaska iz funkcije, da bi se mogla koristiti ta referenca:

```
// Može ovako:
int& f(int &i) {
 int &r=new int(1);
 //...
 return r; // pod uslovom da nije bilo delete &r
}
```

```
// ili ovako:
int& f(int &i) {
 //...
 return i;
}
```

```
// ali ne može ovako:
int& f(int &i) {
 int r=1;
 //...
 return r;
}
```

```
// niti ovako:
int& f(int i) {
 //...
 return i;
}
```

```
// niti ovako:
int& f(int &i) {
 int r=new int(1);
 //...
 return r;
}
```

- Sličnosti između pokazivača i reference:

- pristup do objekta i preko pokazivača i preko reference je posredan;
- virtuelni mehanizam se aktivira pri posrednom pristupu do objekta i preko pokazivača i preko reference;
- istoga pravila, a naročito pravila konverzije, važe i za pokazivače i za reference.

- Razlike između pokazivača i reference:

- pokazivač se može preusmeriti tako da ukazuje na drugi objekat; referenca je od trenutka svog nastanka, tj. od inicijalizacije, trajno vezana za isti objekat;
- pokazivač može da ne ukazuje ni na šta (vrednost 0); referenca uvek, od početka do kraja svog životnog veka, upućuje na jedan (isti) objekat;
- pristup do objekta preko pokazivača vrši se preko operatora \*; pristup do objekta preko reference je neposredan – upotreba reference u izrazu odnosi se na referencirani objekat.
- Rezultat poziva funkcije je vrednost samo ako funkcija vraća referencu.
- Ne postoje nizovi referenci, pokazivači na reference, ni reference na reference.



## Funkcije

### Deklaracije funkcija i prenos argumenta

- Funkcije se deklariraju i definišu kao i u jeziku C, samo što je moguće kao tipove argumenta i rezultata navesti korisničke tipove (klase).
- U deklaraciji funkcije ne moraju da se navode imena formalnih argumenta.
- Pri pozivu funkcije, upoređuju se tipovi stvarnih argumenta sa tipovima formalnih argumenta navedenim u deklaraciji, i, po potrebi, vrši se konverzija. Semantička prenosa argumenta jednaka je semantici inicijalizacije.
- Pri pozivu funkcije, inicijalizuju se formalni argumenti, kao automatski lokalni objekti pozvane funkcije. Ovi objekti se konstruišu pozivom odgovarajućih konstruktora, ako ih ima. Pri vraćanju vrednosti iz funkcije, semantička je ista: konstruiše se privremeni objekat koji prihvata vraćenu vrednost na mestu poziva:

```
class Tip {
//...
public:
 Tip(int i); // konstruktor
};

Tip f (Tip k) {
//...
 return 2; // poziva se konstruktor Tip(2)
}

void main () {
 Tip k(0);
 k=f(1);
 //...
 // poziva se konstruktor Tip(1)
}
```

### Neposredno ugrađivanje u kôd

- Često se definišu vrlo jednostavne, kratke funkcije (na primer, samo prosleđuju argumente drugim funkcijama). Tada je vreme koje se troši na prenos argumenta i poziv duže od vremena izvršavanja tela same funkcije.
- Ovakve funkcije se mogu deklarirati tako da se neposredno ugrađuju u kôd (*inline* funkcije). Tada se telo funkcije direktno ugrađuje u pozivajućih kôd. Semantička poziva ostaje potpuno ista kao i za običnu funkciju.
- Ovakva funkcija deklarise se kao *inline*:  

```
inline int inc(int i) {return i+1;}
```
- Funkcija članica klase može biti *inline* ako se definiše unutar deklaracije klase, ili izvan deklaracije klase, kada se ispred njene deklaracije nalazi reč *inline*:

```
class C {
public:
 int val {return i;} // ovo je inline funkcija
private:
 int i;
};
```

```
// ili:
class D {
public:
 int val ();
private:
 int i;
};
```

```
inline int D::val() {return i;}
```

- Prevodilac ne mora da uvaži zahtev za neposredno ugrađivanje u kôd. Programeru ovo ne treba da predstavlja nikakvu prepreku, jer je semantička ista. *Inline* funkcije samo mogu da ubrzaju program, a ne i da izmene njegovo izvršavanje.
- Ako se *inline* funkcija koristi u više datoteka, u svakoj datoteci mora da postoji njena potpuna definicija (najbolje pomoću datoteke-zaglavlja).

### Podrazumevane vrednosti argumenta

- C++ obezbeđuje i mogućnost postavljanja podrazumevanih vrednosti za argumente. Ako se pri pozivu funkcije ne navede argument za koji je definisana podrazumevana vrednost (u deklaraciji funkcije), kao vrednost stvarnog argumenta uzima se ta podrazumevana vrednost:

```
Complex::Complex (double r=0, double i=0) // podrazumevana
{real=r; imag=i;} // vrednost za r i i je 0

void main () {
//...
 Complex c; // kao da je napisano "Complex c(0,0);"
 //...
}
```

- Podrazumevani argumenti mogu da budu samo nekoliko poslednjih iz liste:

```
Complex::Complex(double r=0, double i) // greška
{ real=r; imag=i; }
```

### Preklapanje imena funkcija

- Često se javlja potreba da se u programu naprave funkcije koje realizuju logički istu operaciju, samo sa različitim tipovima argumenta. Za svaki od tih tipova mora, naravno, da se realizuje posebna funkcija. U jeziku C to bi se ostvarilo tako što bi se tim funkcijama dodelila različita imena. To, međutim, smanjuje čitljivost programa.
- U jeziku C++ moguće je definisati više različitih funkcija sa istim identifikatorom. Ovakav koncept naziva se *preklapanje imena funkcija* (engl. *function overloading*). Uslov je da im se razlikuje broj i/ili tipovi argumenta. Tipovi rezultata ne moraju da se razlikuju:  

```
char* max (const char *p, const char *q)
{ return (strcmp(p,q)>=0)?p:q; }
double max (double i, double j) { return (i>j) ? i : j; }
```
- ```
double r=max(1.5,2.5); // poziva se max(double,double)
char *q=max("Pera","Mika"); // poziva se max(const char*,const char*)
```
- Koja će se funkcija stvarno pozvati, određuje se u fazi prevodjenja, prema slaganju tipova stvarnih i formalnih argumenta. Zato je potrebno da prevodilac može jednoznačno da odredi koja se funkcija poziva.

- Pravila za razrešavanje poziva veoma su složena (vidi knjigu), pa se u praksi svode samo na dovoljno jasno razlikovanje tipova formalnih argumenata preklapljenih funkcija. Kada razrešava poziv, prevodilac otprilike ovako pravi redosled slaganja tipova stvarnih i formalnih argumenata:

1. Najbolje je potpuno slaganje tipova; tipovi T* (pokazivač na T) i T [] (niz elemenata tipa T) se ne razlikuju.
2. Sledeće je slaganje tipova korišćenjem standardnih konverzija.
3. Zatim sledi slaganje tipova korišćenjem korisničkih konverzija;
4. Najmanje odgovara slaganje sa tri tačke (...).

Operatori i izrazi

- Pregled operatora dat je u sledećoj tabeli. Operatori su grupisani po prioritetima, tako da su operatori u istoj grupi istog prioriteta, višeg od operatora koji su u narednoj grupi. U tabelici su prikazane i ostale važne osobine: način grupisanja (asocijativnost, L – sleva udesno, D – zdesna ulevo), da li je rezultat lvrrednost (D – da, N – nije, D/N – zavisi od nekog operanda, pogledati specifikaciju operatora u knjizi), kao i način upotrebe. Prazna polja ukazuju da svojstvo grupisanja nije primereno datom operatoru.

Operator	Značenje	Grup.	lvrred.	Upotreba
::	razrešavanje oblasti važenja	L	D/N	ime_klase :: član
::	pristup globalnom imenu		D/N	:: ime
[]	indeksiranje	L	D	izraz1 izraz2
()	poziv funkcije	L	D/N	izraz (lista_izraza)
()	konstrukcija vrednosti	N	N	ime_tipa (lista_izraza)
.	pristup članu	L	D/N	izraz . ime
->	posredni pristup članu	L	D/N	izraz -> ime
++	postfiksni inkrement	L	N	lvrrednost++
--	postfiksni dekrement	L	N	lvrrednost--
++	prefiksni inkrement	D	D	++lvrrednost
--	prefiksni dekrement	D	D	--lvrrednost
sizeof	veličina objekta	D	N	sizeof izraz
sizeof	veličina tipa	D	N	sizeof (tip)
new	stvaranje dinamičkog objekta	N	N	new tip
delete	ukidanje dinamičkog objekta	N	N	delete izraz
~	komplement po bitovima	D	N	~izraz
!	logička negacija	D	N	!izraz
-	unarni minus	D	N	-izraz
+	unarni plus	D	N	+izraz

Operator	Značenje	Grup.	lvrred.	Upotreba
&	adresa	D	N	&lvrrednost
*	dereferenciranje pokazivača	D	D	* izraz
()	konverzija tipa (cast)	D	D/N	(tip) izraz
*	posredni pristup članu	L	D/N	izraz . * izraz
->	posredni pristup članu	L	D/N	izraz -> * izraz
*	množenje	L	N	izraz * izraz
/	deljenje	L	N	izraz / izraz
%	ostatak	L	N	izraz % izraz
+	sabiranje	L	N	izraz + izraz
-	oduzimanje	L	N	izraz - izraz
<<	pomeranje ulevo	L	N	izraz << izraz
>>	pomeranje udesno	L	N	izraz >> izraz
<	manje od	L	N	izraz < izraz
<=	manje ili jednako od	L	N	izraz <= izraz
>	veće od	L	N	izraz > izraz
>=	veće ili jednako od	L	N	izraz >= izraz
==	jednako	L	N	izraz == izraz
!=	nije jednako	L	N	izraz != izraz
&	I po bitovima	L	N	izraz & izraz
^	isključivo Ili po bitovima	L	N	izraz ^ izraz
	Ili po bitovima	L	N	izraz izraz
&&	logičko I	L	N	izraz && izraz
	logičko Ili	L	N	izraz izraz
? :	uslovni operator	L	D/N	izraz ? izraz : izraz
=	prosta dodela	D	D	lvrrednost = izraz
*	množenje i dodela	D	D	lvrrednost * = izraz
/	deljenje i dodela	D	D	lvrrednost / = izraz
%	ostatak i dodela	D	D	lvrrednost % = izraz
++	sabiranje i dodela	D	D	lvrrednost += izraz
--	oduzimanje i dodela	D	D	lvrrednost -= izraz
>>=	pomeranje udesno i dodela	D	D	lvrrednost >> = izraz
<<=	pomeranje ulevo i dodela	D	D	lvrrednost << = izraz
&=	I i dodela	D	D	lvrrednost & = izraz
=	Ili i dodela	D	D	lvrrednost = izraz
=	isključivo Ili i dodela	D	D	lvrrednost ^= izraz
,	sekvencija	L	D/N	izraz , izraz

Klase

Klase, objekti i članovi klase

Pojam i deklaracija klase

- *Klasa* je realizacija apstrakcije koja ima svoju internu predstavu (svoje attribute) i operacije koje se mogu izvršiti nad njenim instancama. Klasa definiše tip. Jedan primerak takvog tipa (instanca klase) naziva se *objektom* te klase (engl. *class object*).
- Podaci koji su deo klase nazivaju se *podaci članovi klase* (engl. *data members*). Funkcije koje su deo klase nazivaju se *funkcije članice klase* (engl. *member functions*).
- Članovi (podaci ili funkcije) klase iza ključne reči *private*: zaštićeni su od pristupa spolja (enkapsulirani su). Ovim članovima mogu pristupati samo funkcije članice klase. Ovi članovi nazivaju se *privatnim članovima klase* (engl. *private class members*).
- Članovi iza ključne reči *public*: dostupni su spolja i nazivaju se *javnim članovima klase* (engl. *public class members*).
- Članovi iza ključne reči *protected*: dostupni su funkcijama članicama date klase, kao i klasa izvedenih iz te klase, ali ne i korisnicima spolja, i nazivaju se *zaštićenim članovima klase* (engl. *protected class members*).
- Redosled sekcija *public*, *protected* i *private* proizvoljan je, ali se preporučuje baš navedeni redosled. Podrazumeva se da su članovi *privatni* (ako se ne navede specifikator ispred).
- Objekat klase ima unutrašnje stanje, predstavljeno vrednostima atributa, koje menja pomoću operacija. Funkcije članice nazivaju se još i *metodima* klase, a poziv ovih funkcija – *upućivanje poruke* objektu klase. Objekat klase menja svoje stanje kada se pozove njegov metod, odnosno kada mu se uputi poruka.
- Objekat unutar svoje funkcije članice može pozivati funkciju članicu neke druge ili iste klase, odnosno može uputiti poruku drugom objektu. Objekat koji šalje poruku (poziva funkciju) naziva se objektat-*klijent*, a onaj koji je prima (čija je funkcija članica pozvana) je objektat-*server*.
- Preporučuje se da se klase projektuju bez javnih podataka članova. Podaci članovi treba da budu *privatni*, osim ukoliko postoje jaki razlozi sa suprotnu odluku. Javne treba da budu samo funkcije članice koje predstavljaju operacije date apstrakcije koje su na raspolaganju korisnicima klase. Zaštićene su obično jednostavne operacije klase koje ne predstavljaaju operacije interfejsa date klase, nego su ili proste funkcije za pristup do podataka

članova, ili pomoćne funkcije koje služe za implementaciju javnih operacija jer se koriste na više mesta (nastale su algoritamskom dekompozicijom implementacije operacija i lokalizacijom zajedničkih delova). Ovakve jednostavnije operacije su pomoćne (engl. *helper functions*) i često su potrebne i izvedenim klasama, pa su zbog toga zaštićene.

- Unutar funkcije članice klase, članovima objekta čija je funkcija pozvana pristupa se direktno, samo navođenjem njihovog imena.
- Kontrola pristupa članovima nije stvar objekta, nego klase: jedan objekat neke klase iz svoje funkcije članice može da pristupi privatnim članovima drugog objekta iste klase.
- Kontrola pristupa članovima potpuno je odvojena od provere oblasti važenja: najpre se, na osnovu oblasti važenja, određuje entitet na koji se odnosi dato ime na mestu obraćanja u programu, a zatim se određuje da li se tom entitetu može pristupiti.

- Moguće je preklopiti (engl. *overlap*) funkcije članice, uključujući i konstruktore.

- Deklaracijom klase smatra se deo kojim se navodi ono što korisnici klase treba da vide. To su uvek javni članovi. Međutim, da bi prevodilac korektno zauzimao prostor za objekte klase, mora da zna njegovu veličinu, pa u deklaraciju klase ulaze i deklaracije privatnih podataka članova.

```
// deklaracija klase complex
class complex {
public:
    void cAdd(complex);
    void cSub(complex);
    float cre();
    float cim();
    //...
private:
    float real, imag;
};
```

- Gorenavedena deklaracija je, zapravo, definicija klase, ali se iz istorijskih razloga naziva deklaracijom.
- Pravu deklaraciju klase predstavlja samo deklaracija `class S;`. Pre potpune deklaracije (zapravo definicije) mogu samo da se definišu pokazivači i reference na tu klasu, ali ne i objekti te klase, jer se njihova veličina ne zna.

Pokazivač this

- Unutar svake funkcije članice postoji implicitni (podrazumevani, ugrađeni) lokalni objekat `this`. Tip ovog objekta je „konstantni pokazivač na klasu čija je funkcija članica“ (ako je klasa `X`, `this` je tipa `X*const`). Ovaj pokazivač ukazuje na objekat čija je funkcija članica pozvana:

```
// definicija funkcije cAdd članice klase complex
complex complex::cAdd (complex c) {
    complex temp=*this;
    // u temp se prepisuje objekat koji je prozvan
    temp.real+=c.real;
    temp.imag+=c.imag;
    return temp;
}
```

- Pristup članovima objekta čija je funkcija članica pozvana obavlja se neposredno; implicitno je to pristup preko pokazivača `this` i operatora `->`. Može se i eksplicitno pristupati članovima preko ovog pokazivača unutar funkcije članice:

```
// nova definicija funkcije cAdd članice klase complex
complex complex::cAdd (complex c) {
    complex temp;
    temp.real=this->real+c.real;
    temp.imag=this->imag+c.imag;
    return temp;
}
```

- Pokazivač `this` je, u stvari, skriveni argument funkcije članice. Poziv objekat.`f()` prevodilac prevodi u kôd koji ima semantiku kao `f(&objekat)`.
- Pokazivač `this` može da se iskoristi prilikom povezivanja (uspostavljanja relacije između) dva objekta. Na primer, neka klasa `X` sadrži objekat klase `Y`, pri čemu objekat klase `Y` treba da „zna“ ko ga sadrži (ko mu je „nadređeni“). Veza se inicijalno može uspostaviti pomoću konstruktora:

```
class X {
public:
    X() : y(this) {...}
private:
    Y y;
};

class Y {
public:
    Y (X* theContainer) : myContainer(theContainer) {...}
private:
    X* myContainer;
};
```

Primeri klase

- Za svaki objekat klase formira se poseban komplet svih podataka članova te klase.
- Za svaku funkciju članicu, postoji jedinstven skup lokalnih statičkih objekata. Ovi objekti žive od prvog nailaska programa na njihovu definiciju, do kraja programa, bez obzira na broj objekata te klase. Lokalni statički objekti funkcija članica imaju sva svojstva lokalnih statičkih objekata funkcija nečlanica, pa nemaju nikakve veze sa klasom i njenim objektima.
- Podrazumevano se sa objektima klase može raditi sledeće:
 1. definisati primeri (objekti) te klase i nizovi objekata klase;
 2. definisati pokazivači na objekte i reference na objekte;
 3. dodeljivati vrednosti (operator `=`) jednog objekta drugom;
 4. uzimati adrese objekata (operator `&`) i posredno pristupati objektima preko pokazivača (operator `*`);
 5. pristupati članovima i pozivati funkcije članice neposredno (operator `.`) ili posredno (operator `->`);

6. prenositi objekti kao argumenti funkcija i to po vrednosti ili referenci, ili prenositi pokazivači na objekte;
 7. vraćati objekti iz funkcija po vrednosti ili referenci, ili vraćati pokazivači na objekte.
- Neki od ovih operacija korisnik može redefinisati preklapanjem operatora. Ostale ovde navedene operacije korisnik mora definisati posebno ako su potrebne (ne podrazumevaju se).

Konstantne funkcije članice

- Dobra programerska praksa je da se korisnicima klase najavi da li neka funkcija članica menja unutrašnje stanje objekta ili ga samo „čita“ i vraća informaciju korisniku klase.
- Funkcije članice koje ne menjaju unutrašnje stanje objekta nazivaju se *inspektori* ili *selektori* (engl. *inspector, selector*). Reč `const` iza zaglavlja funkcije ukazuje korisniku klase da je funkcija članica inspektor. Ovakve funkcije članice nazivaju se u jeziku C++ *konstantnim* funkcijama članicama (engl. *constant member functions*).
- Funkcija članica koja menja stanje objekta naziva se *mutator* ili *modifikator* (engl. *mutator, modifier*) i ne označava se posebno:

```
class X {
public:
    int read () const { return i; }
    int write (int j=0) { int temp=i; i=j; return temp; }
private:
    int i;
};
```

- Deklarisanje funkcije članice kao inspektora samo je notaciona pogodnost i „stvar lepog ponašanja prema korisniku“. To je „obecanje“ projektanta klase da funkcija ne menja stanje objekta koje je projektant klase definisao. Prevodilac nema načina da u potpunosti proveri da li inspektor posredno menja neke podatke članove klase.
- Inspektor može da menja podatke članove pomoću eksplicitne konverzije, koja „probija“ kontrolu konstantnosti. To je ponekad slučaj kada inspektor treba da izračuna podatak koji vraća, pa ga onda sačuva u nekom članu da bi sledeći put brže vratio odgovor.
- U konstantnoj funkciji članici tip pokazivača `this` je `const X* const`, tako da pokazivač na konstantni objekat, pa nije moguće menjati objekat preko ovog pokazivača (svaki nepoštedni pristup članu je implicitno pristup preko ovog pokazivača). Takođe, za kretanje objekta klase nije dozvoljeno pozivati nekonstantnu funkciju članicu (korektnost konstantnosti). Za prethodni primer:

```
X x;
const X cx;

X::read(); // u redu: konstantna funkcija nekonstantnog objekta;
X::write(); // u redu: nekonstantna funkcija konstantnog objekta;
cx.read(); // u redu: konstantna funkcija konstantnog objekta;
cx.write(); // greška: nekonstantna funkcija konstantnog objekta;
```

Ugnežđivanje klase

- Klase mogu da se deklarishu i unutar deklaracije druge klase (ugnežđivanje deklaracija klasa). Ugnežđena klasa se nalazi u oblasti važenja okružujuće klase, pa se njenom imenu može pristupiti samo preko operatora razrešavanja oblasti važenja `::`.
- Okružujuća klasa nema nikakva posebna prava pristupa članovima ugnežđene klase, niti ugnežđena klasa ima posebna prava pristupa članovima okružujuće klase. Ugnežđivanje je samo stvar oblasti važenja, a ne i kontrole pristupa članovima.

```
int x,y;

class Spoljna {
public:
    int x;
    class Unutrasnja {
    void f(int i, Spoljna *ps) {
        x=i; // greška: pristup Spoljna::x nije korektan!
        ::x=i; // u redu: pristup globalnom x;
        y=i; // u redu: pristup globalnom y;
        ps->x=i; // u redu: pristup Spoljna::x objekta *ps;
    }
};
```

Unutrasnja u; // greška: Unutrasnja nije u oblasti važenja!
 Spoljna::Unutrasnja u; // u redu;

- Unutar deklaracije klase mogu se navesti i deklaracije nabrajanja (enum), i `typedef` deklaracije. Ugnežđivanje se koristi kada neki tip (nabrajanje ili klasa npr.) semantički pripada samo datoj klasi, a nije globalno važan i za druge klase. Ovakvo korišćenje povećava čitljivost programa i smanjuje potrebu za globalnim tipovima.

Struktura

- Struktura je klasa kod koje su svi članovi podrazumevano javni. To se može promeniti eksplicitnim umećanjem `public:` i `private:`:

```
struct a { // isto što i:
    //...
private: //...
};

class a {
public: //...
private: //...
};
```

- Struktura se obično koristi za definisanje slogova podataka koji ne predstavljaju apstrakciju, odnosno nemaju ponašanje (nemaju značajnije operacije), nego najčešće služe samo za implementaciju složene strukture neke apstrakcije. Strukture tipično poseduju samo konstruktore i možda destruktore kao funkcije članice.

Zajednički članovi klase

Zajednički podaci članovi

- Pri nastanku objekata klase, za svaki objekat se pravi poseban komplet podataka članova. Ipak, moguće je definisati podatke članove za koje postoji samo jedan primerak za celu klasu, tj. za sve objekte klase.

- Ovakvi članovi se nazivaju *statičkim članovima*, i deklariraju se pomoću reči `static`:

```
class X {
public:
    //...
private:
    static int i;      // postoji samo jedan i za celu klasu
    int j;             // svaki objekat ima svoj j
    //...
};
```

- Svaki pristup statičkom članu iz bilo kog objekta klase znači pristup istom zajedničkom članu-objektu.
- Statički član klase ima životni vek kao i globalni statički objekat: nastaje na početku programa i traje do kraja programa. Statički član klase ima sva svojstva globalnog statičkog objekta, osim oblasti važenja klase i kontrole pristupa.
- Statički član mora da se inicijalizuje posebnom definicijom van deklaracije klase. Obrađanje ovakvom članu van klase vrši se preko operatora `::`. Za prethodni primer:

```
int X::i=5;
```

- Statičkom članu može da se pristupi iz funkcije članice, ali i van funkcija članica, čak i pre formiranja jednog objekta klase (jer statički član nastaje kao i globalni objekat), naravno uz poštovanje prava pristupa. Tada mu se pristupa preko operatora `::` (`X::i`).
- Zajednički članovi se uglavnom koriste kada svi primerici jedne klase treba da dele neku zajedničku informaciju, npr. kada predstavljaju neku kolekciju, odnosno kada je potrebno imati ih „sve na okupu i pod kontrolom“. Na primer, svi objekti neke klase se uvezuju u listu, a glava liste je zajednički član klase.
- Zajednički članovi smanjuju potrebu za globalnim objektima i tako povećavaju čitljivost programa, jer je, za razliku od globalnih objekata, moguće ograničiti pristup do njih. Zajednički članovi logički pripadaju klasi i „upakovani“ su u nju.

Zajedničke funkcije članice

- I funkcije članice mogu da se deklariraju kao zajedničke za celu klasu, dodavanjem reči `static` ispred deklaracije funkcije članice.

Statičke funkcije članice imaju sva svojstva globalnih funkcija, osim oblasti važenja i kontrole pristupa. One ne poseduju pokazivač `this` i ne mogu neposredno (bez pominjanja konkretnog objekta klase) koristiti nestatičke članove klase. Statičke funkcije mogu neposredno koristiti samo statičke članove te klase.

- Statičke funkcije članice mogu se pozivati za konkretan objekat (što nema posebno značenje), ali i pre formiranja jednog objekta klase, preko operatora `::`.
- Primer:

```
class X {
    static int x;      // statički podatak član;
    int y;
public:
    static int f(X x); // statička funkcija članica;
    int g();
};

int X::x=5;           // definicija statičkog podatka člana;
```

```
int X::f(X x1, X x2) { // definicija statičke funkcije članice;
    int i=x;           // pristup statičkom članu X::x;
    int j=y;           // greška: X::y nije statički,
                        // pa mu se ne može pristupiti neposredno!
    int k=x1.y;        // ovo može;
    return x2.x;       // i ovo može,
                        // ali se izraz "x2" ne izračunava;

    int X::g () {      // nestatička funkcija članica može da
        int i=x;       // koristi i pojedinačne i zajedničke
        int j=y;       // članove; y je ovde this->y;
        return j;
    }

    void main () {
        X xx;
        int p=X::f(xx,xx); // X::f može neposredno, bez objekta;
        int q=X::g();      // greška: za X::g mora konkretan objekat!
        xx.g();            // ovako može;
        p=xx.f(xx,xx);     // i ovako može,
                            // ali se izraz "xx" ne izračunava;
    }
```

- Statičke funkcije predstavljaju operacije klase, a ne svakog posebnog objekta. Pomoću njih se definišu neke opšte usluge klase (engl. *class utilities*), npr. tipično stvaranje novih, dinamičkih objekata te klase (operator `new` je implicitno definisan kao statička funkcija klase). Na primer, na sledeći način obezbeđuje se da se za datu klasu mogu formirati samo dinamički objekti:

```
class X {
public:
    static X* create () { return new X; }
private:
    X(); // konstruktor je privatn
};
```

Prijatelji klase

- Često je dobro da se klasa projektuje tako da ima i povlašćene korisnike, odnosno funkcije ili druge klase koje imaju pravo pristupa njenim privatnim članovima. Takve funkcije i klase nazivaju se prijateljima (engl. *friends*).

Prijateljske funkcije

- Prijateljske funkcije (engl. *friend functions*) neke klase nisu članice te klase, ali imaju pristup do privatnih članova te klase. Te funkcije mogu da budu globalne funkcije ili članice drugih klasa.
- Da bi se neka funkcija proglasila prijateljem klase, potrebno je bilo gde u deklaraciji te klase navesti deklaraciju te funkcije sa ključnom reči `friend` ispred. Prijateljska funkcija se definiše na uobičajen način:

```
class X {
public:
    void f(int ip) {i=ip;}
private:
    friend void g (int,x); // prijateljska globalna funkcija
    friend void y::h ();  // prijateljska članica druge klase
    int i;
```

```

void g (int k, x ex) {
    x.i=k;
    // prijateljska funkcija može da pristupa
    // privatnim članovima klase
}

void main () {
    x x;
    x.f(5);
    // postavljanje preko prijatelja
    g(6,x);
}

```

- Globalne funkcije koje predstavljaju usluge neke klase ili operacije nad tom klasom (najčešće su prijatelji te klase) nazivaju se *klasnim uslugama* (engl. *class utilities*).
- Nema formalnih razloga da se koristi globalna (najčešće prijateljska) funkcija umesto funkcije članice. Globalne funkcije su ponekad pogodnije:
 1. globalnoj funkciji može da se dostavi kao argument i objekat drugog tipa, koji će se koristovati u potreban tip, dok funkcija članica mora da se pozove za objekat date klase;
 2. kada funkcija treba da pristupa članovima više klase;
 3. kada je notaciono pogodnije da se koriste globalne funkcije (poziv je $f(x)$ umesto članica (poziv je $x.f()$); na primer, $\max(a, b)$ je čitljivije od $a.\max(b)$);
- „Prijateljstvo“ se ne nasleđuje: ako je funkcija f prijatelj klasi X , a klasa Y izvedena (naslednik) iz klase X , funkcija f nije prijatelj klasi Y .

Prijateljske klase

- Ako je potrebno da sve funkcije članice klase Y budu prijateljske funkcije klasi X , onda se klasa Y deklarise kao prijateljska klasa (engl. *friend class*) klasi X . Tada sve funkcije članice klase Y mogu da pristupaju privatnim članovima klase X , ali obratno ne važi („prijateljstvo“ nije simetrična relacija):

```

class X {
    friend class Y;
    //...
};

```

- „Prijateljstvo“ nije ni tranzitivna relacija: ako je klasa Y prijatelj klasi X , a klasa Z prijatelj klasi Y , klasa Z nije automatski prijatelj klasi X , već to mora eksplicitno da se naglasi (ako je potrebno).
- Prijateljske klase se obično koriste kada dve klase imaju teško međusobne veze. Pri tome je nepotrebno (i loše) „otkrivati“ delove neke klase da bi oni bili dostupni drugoj klasi, jer će onda biti dostupni i ostalima (nuži se enkapsulacija). U tom slučaju se ove dve klase proglašavaju prijateljskim. Na primer, na sledeći način može se obezbediti da samo klasa *Creator* može da stvara objekte klase X :

```

class X {
public:
    ...
private:
    friend class Creator;
    X(); // konstruktor je dostupan samo klasi Creator
    ...
};

```

Konstruktori i destruktori

Pojam konstruktora

- Funkcija članica koja nosi isto ime kao i klasa naziva se konstruktor (engl. *constructor*). Ova funkcija se uvek poziva prilikom nastanka objekta te klase.
- Konstruktor nema tip koji vraća. Konstruktor može da ima argumente proizvoljnog tipa. Unutar konstruktora, članovima objekta pristupa se kao i u bilo kojoj drugoj funkciji članici.
- Konstruktor se uvek implicitno poziva pri nastanku objekta klase, odnosno na početku životnog veka svakog objekta date klase.
- Konstruktor, kao i svaka funkcija članica, može biti preklopljen (engl. *overloaded*). Konstruktor koji se može pozvati bez stvarnih argumentata (nema formalne argumente ili ima sve argumente sa podrazumevanim vrednostima) naziva se podrazumevanim konstruktorom.
- Ukoliko u klasi nije eksplicitno deklarisan nijedan konstruktor, prevodilac implicitno generiše podrazumevani konstruktor koji je javni i koji vrši podrazumevanu inicijalizaciju podobjekta osnovne klase i objekata članova pozivima njihovih podrazumevanih konstruktora, ako ih ima (bilo kao eksplicitno deklarisan ili implicitno generisan). Ako takvih konstruktora u odgovarajućim klasama nema, javlja se greška.

Kada se poziva konstruktor?

- Konstruktor je funkcija koja pretvara „presne“ memorijske lokacije koje je sistem odvojio za novi objekat (i sve njegove podatke članove) u „pravi“ objekat koji ima svoje članove i koji može da prima poruke, odnosno ima sva svojstva svoje klase i konzistentno početno stanje. Pre nego što se pozove konstruktor, objekat je u trenutku definisanja samo „gomila praznih bitova“ u memoriji računara. Konstruktor ima zadatak da od ovih bitova napravi objekat tako što će inicijalizovati podobjekat osnovne klase i članove.
- Konstruktor se poziva uvek kada objekat klase nastaje, tj. kada se:
 1. izvršava definicija statičkog objekta;
 2. izvršava definicija automatskog (lokalnog nestatičkog) objekta unutar bloka; formalni argumenti, pri pozivu funkcije, nastaju kao lokalni automatski objekti;
 3. inicijalizuje objekat, pozivaju se konstruktori njegovih podataka članova;
 4. stvara dinamički objekat operatorom *new*;
 5. stvara privremeni objekat, pri povratku iz funkcije, koji se inicijalizuje vraćenom vrednošću funkcije.

Način pozivanja konstruktora

- Konstruktor se poziva kada nastaje objekat klase. Na tom mestu je moguće navesti inicijalizatore, tj. stvarne argumente poziva konstruktora. Poziva se onaj konstruktor koji se najbolje slaže po broju i tipovima argumentata (pravila su ista kao i pri preklapanju funkcija):

```

class X {
public:
    X();
    X(double);
    X(char*);
    //...
};

```

```
void main () {
    double d=3.4;
    char *p="Niz znakova";
    // poziva se x(double)
    x a(d),
    // poziva se x(char*)
    b(p),
    // poziva se x()
    c;
    //...
}
```

- Pri definisanju objekta c sa zahtevom da se poziva podrazumevani konstruktor klase X, ne treba navesti X c(); (jer je to deklaracija funkcije), već samo X c;
- Pre izvršavanja samog tela konstruktora klase, pozivaju se konstruktori članova. Argumen-
menti ovih poziva mogu da se navedu iza zaglavlja definicije (ne deklaracije) konstrukto-
ra klase, iza znaka : (dvotačka):

```
class YY {
public:
    YY (int j) {...}
    //...
};

class XX {
public:
    XX (int);
private:
    YY y;
    int i;
};

XX::XX (int k) : y(k+1), i(k-1) {
    // y je inicijalizovan sa k+1, a i sa k-1
    // ... ostatak konstruktora
}
```

- Prvo se pozivaju konstruktori članova, po redosledu deklarisanja u deklaraciji klase, pa se onda izvršava telo konstruktora klase.
- Ovaj način ne samo da je moguć, već je i jedino ispravan: navođenje inicijalizatora u zaglavlju konstruktora predstavlja specifikaciju *inicijalizacije* članova (koji su ugrađeni tipa ili su objekti klase), što je različito od *operacije dodele* koja se može vršiti unutar tela konstruktora. Osim toga, kada član korisničkog tipa nema podrazumevani konstruktor, ili kada je član konstanta ili referenca, ovaj način je i jedini način inicijalizacije člana.
- Konstruktor se može pozvati i eksplicitno u nekom izrazu. Tada nastaje privremeni objekat klase pozivom odgovarajućeg konstruktora sa navedenim argumentima. Isto se dešava ako se u inicijalizatoru eksplicitno navede poziv konstruktora:

```
void main () {
    complex c1(1.2,4), c2;
    c2=c1+complex(3.4,-1.5); // privremeni objekat
    complex c3=complex(0.1,5); // opet privremeni objekat
    // koji se kopira u c3
}
```

- Kada se pravi niz objekata neke klase, poziva se podrazumevani konstruktor za svaku komponentu niza ponosob, po rastućem redosledu indeksa.

Konstruktor kopije

- Kada se objekat x1 klase XX inicijalizuje drugim objektom x2 iste klase, C++ će podrazumevano (ugrađeno) izvršiti prostu inicijalizaciju članova objekta x1 članovima objekta x2. To ponekad nije dobro (ako objekti sadrže članove koji su pokazivači ili reference na pridružene dinamičke objekte koji ekskluzivno pripadaju datom objektu), pa programer treba da ima potpunu kontrolu nad inicijalizacijom objekta drugim objektom iste klase (potrebno je tzv. *duboko kopiranje*, engl. *deep copy*).
- Za ovu svrhu služi tzv. konstruktor kopije (engl. *copy constructor*). To je konstruktor klase XX koji se može pozvati sa samo jednim stvarnim argumentom tipa XX. Taj konstruktor se poziva kada se objekat inicijalizuje objektom iste klase.

- Konstruktor kopije nikad ne sme imati formalni argument tipa XX, a može imati argument tipa XX& ili, najčešće, const XX&.

- Ako klasa nema eksplicitno deklarisan konstruktor kopije, prevodilac implicitno generiše konstruktor kopije koji je javni i koji vrši podrazumevanu inicijalizaciju podobjekta osnovne klase i članova, pozivom njihovih konstruktora kopije (eksplicitno deklarisanih ili implicitno generisanih). Za članove koji su instance ugrađenih tipova, podrazumevana inicijalizacija u ovom slučaju znači prosto kopiranje njihove vrednosti.

```
class XX {
public:
    XX (int);
    XX (const XX&); // konstruktor kopije
    //...
};

XX f(XX x1) {
    XX x2=x1; // poziva se konstruktor kopije XX(XX&) za x2
    //...
    return x2; // poziva se konstruktor kopije za
               // privremeni objekat u koji se smešta rezultat
}

void g() {
    XX xa=3, xb=1;
    //...
    xa=f(xb); // poziva se konstruktor kopije samo za
              // formalni argument x1,
              // a za xa poziva XX::operator=
}
```

Destruktor

- Funkcija članica koja ima isto ime kao klasa, uz znak ~ ispred imena, naziva se *destruktor* (engl. *destructor*). Ova funkcija poziva se automatski, pri prestanku života objekta klase, za sve navedene slučajeve (statičkih, automatskih, klasnih članova, dinamičkih i privremenih objekata):

```
class X {
public:
    ~X () { cout<<"Poziv destruktora klase X!\n"; }
}

void main () {
    X x;
    //...
} // ovde se poziva destruktor objekta x
```


- Destruktor nema tip koji vraća i ne može imati argumente. Unutar destruktora, članovima se pristupa kao i u bilo kojoj drugoj funkciji članici. Svaka klasa može da ima najviše jedan destruktor.
- Destruktor se implicitno poziva i pri uništavanju dinamičkog objekta pomoću operatora `delete`. Za niz, destruktor se poziva za svaki element ponaosob. Redosled poziva destruktora je, u svakom slučaju, obratan od redosleda poziva konstruktora
- Ako klasa nema eksplicitno deklarisan destruktor, prevodilac implicitno generiše podrzumevani destruktor koji je javni i koji vrši destrukciju podataka članova i podobjekta osnovne klase pozivom njihovih destruktora.
- Destruktori se uglavnom koriste kada objekat treba da dealocira memoriju ili neke sistemske resurse koje je konstruktor alocirao; to je najčešće potrebno kada klasa sadrži članove koji su pokazivači na pridružene dinamičke objekte koji ekskluzivno pripadaju datom objektu, pa ih je potrebno uništiti prilikom uništavanja tog objekta.

Članka 6

Preklapanje operatora

Pojam preklapanja operatora

- Prepostavimo da su nam u programu potrebni kompleksni brojevi i operacije nad njima. Treba nam struktura podataka koja će, pomoću osnovnih (u jezik ugrađenih) tipova, predstaviti strukturu kompleksnog broja, a takođe i funkcije koje će realizovati operacije nad kompleksnim brojevima.
- Kada je potrebna struktura podataka za koju nisu bini detalji implementacije, već operacije koje se nad njom vrše, sve ukazuje na klasu. Klasa upravo predstavlja apstraktni tip podataka za koji su definisane operacije.
- U jeziku C++, operatori za korisničke tipove su specijalne funkcije koje nose ime `operator`, gde je `@` neki operator ugrađen u jezik:


```
class complex {
public:
    complex(double re, double im);           /* konstruktor */
    friend complex operator+ (complex, complex); /* operator + */
    friend complex operator- (complex, complex); /* operator - */
private:
    double real, imag;
};

complex::complex (double r, double i) : real(r), imag(i) {}

complex operator+ (complex c1, complex c2) {
    complex temp(0,0); /* priprema promenljiva tipa complex */
    temp.real=c1.real+c2.real;
    temp.imag=c1.imag+c2.imag;
    return temp;
}

complex operator- (complex c1, complex c2) {
    /* može i ovako: vratiti privremenu promenljivu
    koja se kreira konstruktorom sa odgovarajućim argumentima */
    return complex(c1.real-c2.real, c1.imag-c2.imag);
}
```
- Operatorske funkcije se mogu koristiti u izrazima kao i operatori nad ugrađenim tipovima. Izraz `t1@t2` se tumači kao `t1.operator@(t2)` ili `operator@(t1,t2)`:


```
complex c1(3,5.4), c2(0,-5.4), c3(0,0);
c3=c1+c2; /* poziva se operator+(c1,c2) */
c1=c2-c3; /* poziva se operator-(c2,c3) */
```

Operatorske funkcije

Osnovna pravila

- U jeziku C++, pored „običnih“ funkcija koje se eksplicitno pozivaju navođenjem identifikatora sa zagradama, postoje i operatorske funkcije.
- Operatorske funkcije su posebna vrsta funkcija koje imaju posebna imena i način pozivanja. Kao i obične funkcije, i one se mogu preklapati za operande koji pripadaju korisničkim tipovima. Ovaj princip se naziva preklapanje operatora (engl. *operator overloading*).
- Ovaj princip omogućava da se definišu značenja operatora za korisničke tipove i formiraju izrazi sa objektima ovih tipova, na primer operacije nad kompleksnim brojevima ($(ca*cb+cc-cd)$, matricama ($(ma*mb+mc-md)$ itd).
- Ipak, postoje neka ograničenja u preklapanju operatora. Ne mogu se:
 1. preklapati operatori `., *, ::, ? : i sizeof`, dok svi ostali mogu;
 2. redefinisati značenja operatora za ugrađene tipove podataka;
 3. uvoditi novi simboli za operatore;
 4. menjati osobine operatora koje su ugrađene u jezik: broj operanada, prioriteti i asocijativnost (smer grupisanja).

- Operatorske funkcije imaju imena `operator@`, gde je `@` znak operatora. Operatorske funkcije mogu biti članice ili globalne funkcije (uglavnom prijatelji klase) kod kojih je bar jedan argument tipa korisničke klase:

```
complex operator+ (complex c, double d) {
    return complex(c.real+d, c.imag);
} // ovo je globalna funkcija prijatelj

complex operator** (complex c, double d) { // ovo ne može
    // hteli smo stepenovanje
}
```

- Za korisničke tipove su implicitno definisana dva operatora: `=` (dodela vrednosti) i `&` (uzimanje adrese). Sve dok ih korisnik ne redefiniše, oni imaju podrazumevano značenje.
- Ukoliko klasa nema eksplicitno deklarisan operator dodele, prevodilac implicitno generiše podrazumevani operator dodele koji je javni i koji vrši operaciju dodele podobijekta osnovne klase i objekata-članova, pozivom njihovih operatora dodele (eksplicitno deklariranih ili implicitno generisanih). Za ugrađene tipove, dodela znači prosto kopiranje vrednosti. Ako objekat sadrži člana koji je pokazivač, kopiraće se, naravno, samo taj pokazivač, a ne i pokazivana vrednost. Ovo nekad nije odgovarajuće i korisnik treba da redefiniše operator `=`.
- Vrednosti operatorskih funkcija mogu da budu bilo kog tipa, pa i `void`.

Bočni efekti i veze između operatora

- Bočni efekti koji postoje kod operatora za ugrađene tipove nikad se ne podrazumevaju za redefinisane operatore: `++` ne mora da menja stanje objekta, niti da znači sabiranje sa 1. Isto važi i za `--` i sve operatore dodele (`=, +=, -=, *=` itd.).
- Operator `=` (i ostali operatori dodele) ne mora da menja stanje objekta. Ipak, ovakve upotrebe treba strogo izbegavati: redefinisani operator treba da ima isto ponašanje kao i za ugrađene tipove.
- Veze koje postoje između operatora za ugrađene tipove ne podrazumevaju se za redefinisane operatore. Na primer, `a+=b` ne mora automatski da znači `a=a+b`, ako je definisan operator `+`, već operator `+=` mora posebno da se definiše.
- Preporučuje se da operatori koje definiše korisnik imaju očekivano značenje, radi čitljivosti programa. Na primer, ako su definisani i operator `++` i operator `+`, dobro je da `a+=b` ima isti efekat kao i `a=a+b`. Treba izbegavati neočekivana značenja, na primer da operator `-` realizuje sabiranje matrica.
- Kada se definišu operatori za klasu, treba težiti da njihov skup bude kompletan. Na primer, ako su definisani operatori `=` i `+`, treba definisati i operator `+=`; ili, uvek treba definisati oba operatora `==` i `!=`, a ne samo jedan.

Operatorske funkcije kao članice i globalne funkcije

- Operatorske funkcije mogu da budu članice klase ili (najčešće prijateljske) globalne funkcije. Ako je `@` neki binarni operator (na primer `+`), on može da se realizuje kao funkcija članica klase `X` na sledeći način (argumenti se mogu prenositi i po referenci):

```
tip operator@ (X)
```

ili kao prijateljska globalna funkcija na sledeći način:

```
tip operator@ (X, X)
```

Nije dozvoljeno da se u programu nalaze obe ove funkcije.

- Poziv `a@b` se sada tumači kao:

`a.operator@ (b)`, za funkciju članicu, ili

`operator@ (a, b)`, za globalnu funkciju.

- Primer:

```
class complex {
public:
    complex (double re=0, double im=0) : real(re), imag(i) {}
    complex operator+(complex c)
    { return complex(real+c.real, imag+c.imag); }
private:
    double real, imag;
};

// ili, alternativno:

class complex {
public:
    complex (double re=0, double im=0) : real(re), imag(i) {}
    friend complex operator+(complex, complex);
};
```

```
private:
    double real, imag;
};

complex operator+ (complex c1, complex c2) {
    return complex(c1.real+c2.real, c1.imag+c2.imag);
}

void main () {
    complex c1(2,3), c2(3,4);
    complex c3=c1+c2; // poziva se c1.operator+(c2) ili
    // operator+(c1, c2)
    //...
```

- Razlozi za izbor jednog ili drugog načina (članica ili globalna prijateljska funkcija) isti su kao i za druge funkcije. Ako se želi da se u prethodnom primeru mogu sabrati realni i kompleksni broji, treba definisati globalnu funkciju. Ako se želi da se može izvršiti $d+c$, gde je d tipa `double`, ne može se definisati nova operatorska „članica klase `double`“, jer ugrađeni tipovi nisu klase (C++ nije čist OO jezik). Operatorska funkcija članica „ne dozvoljava promociju levog operanda“, što znači da se neće konvertovati operand `d` u tip `complex`. Treba izabrati drugi navedeni postupak (sa prijateljskom operatorskom funkcijom).

Unarni i binarni operatori

- Mnogi operatori jezika C++ (kao i jezika C) mogu da budu i unarni i binarni (unarni i binarni - unarni &-adresa i binarni &-logičko I po bitovima itd.). Kako razlikovati unarne i binarne operatore prilikom preklapanja?
- Unarni operator ima samo jedan operand, pa se može realizovati kao operatorska funkcija članica bez argumentata (prvi operand je objekat čija je funkcija članica pozvana):


```
tip operator@ ()
// kao globalna funkcija sa jednim argumentom:
tip operator@ (X x)
```
- Binarni operator ima dva operanda, pa se može realizovati kao funkcija članica sa jednim argumentom (prvi operand je objekat čija je funkcija članica pozvana):


```
tip operator@ (X xdesni),
// kao globalna funkcija sa dva argumenta:
tip operator@ (X xlevi, X xdesni)
```

• Primer:

```
class complex {
public:
    complex (double re=0, double im=0) : real(re), imag(im) {}
    friend complex operator+(complex, complex);
    complex operator+ () // unarni operator, konjugovani broj
    { return complex(real, -imag); }
private:
    double real, imag;
};
```

Neki posebni operatori

Operatori new i delete

- Ponekad programer želi da preuzme kontrolu nad alokacijom dinamičkih objekata neke klase, a ne da je prepusti ugrađenom alokatoru. To je zgodno npr. kada su objekti klase mali i može se precizno kontrolisati njihova alokacija, tako da se smanje troškovi alokacije.
- Za ovakve potrebe mogu se preklopiti operatori `new` i `delete` za neku klasu. Operatorske funkcije `new` i `delete` moraju biti statičke (`static`) funkcije članice, jer se one pozivaju pre nego što je objekat stvarno inicijalizovan, odnosno pošto je uništen.
- Ako je korisnik definisao ove operatorske funkcije za neku klasu, one će se pozivati kad god nastaje dinamički objekat te klase operatorom `new`, odnosno kada se takav objekat dealocira operatorom `delete`.
- Unutar tela ovih operatorskih funkcija ne treba eksplicitno pozivati konstruktor, odnosno destruktor. Konstruktor se implicitno poziva posle operatorske funkcije `new`, a destruktor se implicitno poziva pre operatorske funkcije `delete`. Ove operatorske funkcije služe samo da obezbede prostor za smeštanje objekta i da ga posle oslobode, a ne da od „presnih“ bitova naprave objekat (što rade konstruktori), odnosno pretvore ga u „presne bitove“ (što radi destruktor). Operator `new` treba da vrati pokazivač na alocirani prostor.
- Ove operatorske funkcije deklaršu se na sledeći način:


```
void* operator new (size_t velicina)
void operator delete (void* pokazivac)
```
- Tip `size_t` je celobrojni tip definisan u `<stdlib.h>` i služi za izražavanje veličina objekata. Argument velicina daje veličinu potrebnog prostora koji treba alocirati za objekat. Argument pokazivac je pokazivač na prostor koji treba osloboditi.
- Podrazumevani (ugrađeni) operatori `new` i `delete` mogu da se pozivaju unutar tela redefinisanih operatorskih funkcija ili eksplicitno, preko operatora `::`, ili implicitno, kada se prave dinamički objekti tipova za koje nisu redefinisani ovi operatori.

• Primer:

```
#include <stdlib.h>

class XX {
public:
    void* operator new (size_t sz)
    { return new char[sz]; } // koristi se ugrađeni new
    void operator delete (void* p)
    { delete [] p; } // koristi se ugrađeni delete
};
```

Konstruktor kopije i operator dodele

- Inicijalizacija objekta pri njegovom nastanku i dodela vrednosti predstavljaju dve suštinski različite operacije.
- Inicijalizacija se vrši u svim slučajevima kada objekat nastaje (statički, automatski, klasni član, privremeni i dinamički). Tada se poziva konstruktor, tako se inicijalizacija obavlja preko znaka =. Ako je izraz sa desne strane znaka = istog tipa kao i objekat koji se inicijalizuje, poziva se konstruktor kopije.
- Dodelom se izvršava operatorska funkcija operator=. To se dešava kada se eksplicitno u nekom izrazu poziva ovaj operator. Ovaj operator najčešće prvo uništava prethodno formirane delove objekta, pa onda formira nove, uz kopiranje delova objekta sa desne strane znaka dodele. Ova operatorska funkcija mora biti nestatička funkcija člana.
- Inicijalizacija podrazumeva da objekat još ne postoji. Dodela podrazumeva da objekat sa leve strane operatora postoji.
- Ako neka klasa sadrži pokazivač na dinamički objekat koji je pridružen svakom objektu te klase i koji je u njegovoj isključivoj nadležnosti, onda treba razmotriti potrebu da ona ima definisan i destruktor, i konstruktor kopije i operator dodele.
- Primer – klasa koja realizuje niz znakova:

```
class String {
public:
    String(const char*);
    String(const String&);
    ~String();
    String& operator= (const String&);
    //...
private:
    char *niz;
};

String::String (const String &s) {
    if (niz=new char [strlen(s.niz)+1]) strcpy(niz,s.niz);
}

String::~String () { delete [] niz; }

String& String::operator= (const String &s) {
    if (&s!=this) {
        delete [] niz;
        if (niz=new char [strlen(s.niz)+1]) strcpy(niz,s.niz);
    }
    return *this;
}

void main () {
    String a("Hello world!"), b=a; // String(const String&);
    a=b;
    //...
```

Osnovni standardni ulazno/izlazni tokovi

Klase istream i ostream

- Kao i jezik C, ni C++ ne sadrži (u jezik ugrađene) ulazno/izlazne (U/I) operacije, već se one realizuju standardnim bibliotekama. Ipak, C++ sadrži standardne U/I biblioteke realizovane u duhu OOP-a.
- Na raspolaganju su i stare C biblioteke sa funkcijama scanf i printf, ali njihovo korišćenje nije u duhu jezika C++.
- Biblioteka čije se deklaracije nalaze u zaglavlju <istream.h> sadrži dve osnovne klase, istream i ostream (ulazni i izlazni tok). Svakom primerku (objektu) klase ifstream i ofstream, koje su redom izvedene iz navedenih klasa, može da se pridruži jedna datoteka za ulaz/izlaz, tako da se datotekama pristupa isključivo preko ovakvih objekata, odnosno funkcija člana ili prijatelja ovih klasa. Time je podržan princip enkapsulacije.
- U ovoj biblioteci definisana su i dva korisniku dostupna (globalna) statička objekta:
 1. Objekat cin klase istream koji je pridružen standardnom ulaznom uređaju (obično tastaturnu);
 2. Objekat cout klase ostream koji je pridružen standardnom izlaznom uređaju (obično ekran).
- Klasa istream je preklopila operator >> za sve ugrađene tipove, koji služi za ulaz podataka:


```
istream& operator>> (istream &is, tip &t);
```

 gde je tip neki ugrađeni tip objekta koji se učitava.
- Klasa ostream je preklopila operator << za sve ugrađene tipove, koji služi za izlaz podataka:


```
ostream& operator<< (ostream &os, tip x);
```

 gde je tip neki ugrađeni tip objekta koji se ispisuje.
- Ove funkcije vraćaju reference, tako da se može vršiti višestruki U/I u istoj naredbi. Osim toga, ovi operatori su asocijativni sleva udesno, tako da se podaci ispisuju prirodnim redosledom.
- Ove operatore treba koristiti za uobičajene, jednostavne U/I operacije:

```
#include <istream.h> // obavezno ako se želi U/I
void main () {
    int i;
    cin>>i;
    cout<<"i="<<i<<"\n";
    // učitava se i
    // ispisuje se npr.: i=5 i prelazi u
    // novi red
}
```

Ulazno/izlazne operacije za korisničke tipove

- Korisnik može da definiše značenje operatora `>>` i `<<` za svoje tipove. To se radi definisanjem prijateljskih funkcija korisnikove klase, jer je prvi operand tipa `istream&` odnosno `ostream&`.

- Primer za klasu `complex`:

```
#include <iostream.h>

class complex {
public:
    //...
    friend ostream& operator<< (ostream&,const complex&);
    //...
    //...
    ostream& operator<< (ostream& kos, const complex& c) {
        return os<<<(" <<<c.real<<<"," <<<c.imag<<<");
    }

    void main () {
        complex c(0.5,0.1);
        cout<<<"c="<<<c<<<"\n"; // ispisuje se: c=(0.5,0.1)
    }
}
```

Glava 7

Nasledivanje

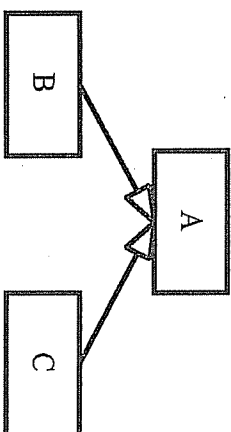
Izvedene klase

Šta je nasledivanje i šta su izvedene klase?

- U praksi se često sreće slučaj da je jedna klasa objekata (klasa B) podvrsta neke druge klase (klasa A). To znači da su objekti klase B „(specijalna) vrsta“ (engl. „a-kind-of“) objekata klase A, ili da objekti klase B „imaju sve osobine klase A, i još neke, sebi svojstvene“. Ovakva relacija između klase naziva se *nasledivanje* (engl. *inheritance*): klasa B nasleduje klasu A.

- Primeri:

1. Sisari su klasa kojoj je svojstven način razmnožavanja. Mesožderci su sisari koji se hrane mesom. Biljojedi su sisari koji se hrane biljkama. Uopšte, u životu svetu odnosi podvrsta mogu da predstavljaju relaciju nasledivanja klase.
 2. Geometrijske figure u ravni su klasa objekata koji imaju zajedničko svojstvo – koordinate težišta. Krug je figura koja ima i dužinu poluprečnika. Kvadrat je figura koja ima i dužinu ivice.
 3. Izlazni uređaji računara su klasa koja ima operacije pisanja jednog znaka. Ekran je izlazni uređaj koji ima mogućnost i crtanja, brisanja, pomeranja kursora itd.
- Relacija nasledivanja se u programskom modelu definiše u odnosu na to šta želimo da klase rade, odnosno koja svojstva i servise da imaju. Primer: da li je krug vrsta elipse, ili je elipsa vrsta kruga, ili su i krug i elipsa podvrste ovalnih figura?
 - Ako je klasa B nasledila klasu A, kaže se još da je klasa A *osnovna klasa* (engl. *base class*), a klasa B *izvedena klasa* (engl. *derived class*). Može se reći i da je klasa A nadklasa (engl. *superclass*), a klasa B podklasa (engl. *subclass*), ili da je klasa A roditelj (engl. *parent*), a klasa B dete (engl. *child*). Relacija nasledivanja se najčešće prikazuje usmerenim acikličnim grafom:



- Jezici koji podržavaju nasljeđivanje nazivaju se *objektno orijentisanim* (engl. *Object-Oriented Programming Languages, OOP*).

Kako se definišu izvedene klase u jeziku C++?

- Da bi se klasa izvela iz postojeće klase, nije potrebno menjati postojeću klasu, niti je ponovo prevoditi. Izvedena klasa se deklarise navođenjem reči `public` i naziva osnovne klase, iza znaka : (dvotačka):

```
class Base {
    int i;
public:
    void f();
};

class Derived : public Base {
    int j;
public:
    void g();
};
```

- Objekti izvedene klase imaju sve članove osnovne klase, i svoje posebne članove koji su navedeni u deklaraciji izvedene klase.
- Objekti izvedene klase definišu se i koriste na uobičajen način:

```
void main () {
    Base b;
    Derived d;
    b.f();
    b.g(); // ovo, naravno, ne može
    d.f(); // d ima i funkciju f,
    d.g(); // i funkciju g
}
```

- Izvedena klasa ne nasljeđuje funkciju članicu operator=.

Prava pristupa

- Ključna reč `public` u zaglavlju deklaracije izvedene klase znači da su svi javni članovi osnovne klase ujedno i javni članovi izvedene klase.
- Privatni članovi osnovne klase uvek to i ostaju. Funkcije članice izvedene klase ne mogu da pristupaju privatnim članovima osnovne klase. Nema načina da se „povredi privatnost“ osnovne klase (ukoliko neko nije prijatelj te klase, što je zapisano u njenoj deklaraciji), jer bi to značilo da postoji mogućnost da se probije enkapsulacija koju je zamislio projektant osnovne klase.
- Javnim članovima osnovne klase se iz funkcija članica izvedene klase pristupa neposredno, kao i sopstvenim članovima:

```
class Base {
    int pb;
public:
    int jb;
    void put(int x) {pb=x;}
};
```

```
class Derived : public Base {
    int pd;
public:
    void write(int a, int b, int c) {
        pd=a;
        jb=b;
        pb=c; // ovo ne može,
        put(c); // već mora ovako
    }
};
```

- Deklaracija člana izvedene klase sakriva istoimeni član osnovne klase. Sakrivenom članu osnovne klase može da se pristupi pomoću operatora :. Na primer, `Base : jb`.
- Često postoji potreba da nekim članovima osnovne klase pristupaju funkcije članice izvedene klase, ali ne i korisnici klase. To su najčešće funkcije članice koje direktno pristupaju privatnim podacima članovima. Članovi koji su dostupni samo izvedenim klasama, ali ne i korisnicima spolja, navode se iza ključne reči `protected`: i nazivaju se *zaštićeni članovi* (engl. *protected members*).
- Zaštićeni članovi ostaju zaštićeni i za sledeće izvedene klase pri sukcesivnom nasljeđivanju. Uopšte, ne može se povećati pravo pristupa nekom članu koji je privatn, zaštićen ili javni.

```
class Base {
    int pb;
protected:
    int zb;
public:
    int jb;
    //...
};
```

```
class Derived : public Base {
    //...
public:
    void write(int x) {
        jb=zb=x; // može da pristupi javnom i zaštićenom članu,
        pb=x; // ali ne i privatnom: greška!
    }
};
```

```
void f() {
    Base b;
    b.zb=5; // odatle ne može da se pristupa zaštićenom članu
}
```

Konstruktori i destruktori izvedenih klasa

- Prilikom nastanka objekta izvedene klase, poziva se konstruktor te klase, ali se pre izvršavanja njegovog tela poziva konstruktor osnovne klase. U zaglavlju definicije konstruktora izvedene klase, u listi inicijalizatora, moguće je navesti i inicijalizator osnovne klase (argumente poziva konstruktora osnovne klase). To se radi navođenjem imena osnovne klase i argumenta poziva konstruktora osnovne klase:

```
class Base {
    int bi;
    //...
public:
    Base(int); // konstruktor osnovne klase
    //...
};
```

```
Base::Base (int i) : bi(i) { /*...*/ }
```

```
class Derived : public Base {
    int di;
    //...
public:
    Derived(int);
    //...
};
```

```
Derived::Derived (int i) : Base(i), di(i+1) { /*...*/ }
```

- Pri inicijalizaciji objekta izvedene klase redosled poziva konstruktora je sledeći:
 1. Inicijalizuje se podobjekat osnovne klase, pozivom konstruktora osnovne klase.
 2. Inicijalizuju se podaci članovi, eventualno pozivom njihovih konstruktora, po redosledu njihovog deklarisanja.
 3. Izvršava se telo konstruktora izvedene klase.
- Pri uništavanju objekta, redosled poziva destruktora uvek je obratan.

```
class XX {
    //...
public:
    XX() {cout<<"Konstruktor klase XX.\n";}
    ~XX() {cout<<"Destruktor klase XX.\n";}
};
```

```
class Base {
    //...
public:
    Base() {cout<<"Konstruktor osnovne klase.\n";}
    ~Base() {cout<<"Destruktor osnovne klase.\n";}
    //...
};
```

```
class Derived : public Base {
    XX xx;
    //...
public:
    Derived() {cout<<"Konstruktor izvedene klase.\n";}
    ~Derived() {cout<<"Destruktor izvedene klase.\n";}
    //...
};
```

```
void main () {
    Derived d;
}

/* Izlaz će biti:
Konstruktor osnovne klase.
Konstruktor klase XX.
Konstruktor izvedene klase.
Destruktor izvedene klase.
Destruktor klase XX.
Destruktor osnovne klase.
*/
```

Polimorfizam

Šta je polimorfizam?

- Pretpostavimo da smo projektovali klasu geometrijskih figura sa namerom da sve figure imaju funkciju `crtaaj()` kao članicu. Iz ove klase izveli smo klase kruga, kvadrata, trougla itd. Naravno, svaka izvedena klasa treba da realizuje funkciju `crtaaj` na sebi svojstven način (krug se sasvim drugačije crta od trougla). Sada nam je potrebno da u nekom delu programa iscrtamo sve figure koje se nalaze na našem crtežu. Ovim figurama pristupamo preko niza pokazivača tipa `Figure*`. C++ omogućava da figure iscrtamo prostim navođenjem:

```
void crtanje () {
    for (int i=0; i<br0jFigure; i++)
        nizFigure[i] -> crtaaj();
}
```

- Iako se u ovom nizu mogu naći različite figure (krugovi, trouglovi itd.), mi im pristupamo kao figurama, jer sve vrste figura imaju zajedničku osobinu „da mogu da se nacrtaju“. Ipak, svaka od figura svoj zadatak ispunije onako kako joj to i priличи, odnosno svaki objekat će „prepoznati“ kojoj izvedenoj klasi pripada, bez obzira na to što mu se obračunamo „upšteno“, kao objektu osnovne klase. To je posledica naše pretpostavke da su i krug i kvadrat i trougao vrste figura.

- Svojsstvo da svaki objekat izvedene klase, čak i kada mu se pristupa kao objektu osnovne klase, izvršava metod tačno onako kako je to definisano u njegovoj izvedenoj klasi, naziva se *polimorfizam* (engl. *polymorphism*).

Virtuelne funkcije

- Funkcije članice osnovne klase koje se u izvedenim klasama mogu realizovati specifično za svaku izvedenu klasu, nazivaju se *virtuelne funkcije* (engl. *virtual functions*).
- Virtuelna funkcija se u osnovnoj klasi deklarise pomoću ključne reči `virtual` na početku deklaracije. Prilikom definisanja virtuelnih funkcija u izvedenim klasama ne mora se stavljati reč `virtual`, ali se to preporučuje radi povećanja čitljivosti i razumljivosti programa.

- Prilikom poziva odaziva se ona funkcija koja pripada klasi kojoj i objekat koji prima poziv.

```
class ClanBiblioteke {
public:
    virtual void placiclanarinu () // virtuelna funkcija
    { r=-clanarina; }
    //...
private:
    Racun r;
    //...
};

class PocasniciCian : public ClanBiblioteke {
public:
    virtual void placiclanarinu () { }
    //...
};
```

```

void main () {
    ClanBiblioteke *clanovi[100];
    //...
    for (int i=0; i<brojClanova; i++)
        clanovi[i] -> platiClanarinu();
    //...
}

```

- Virtualna funkcija osnovne klase ne mora da se redefiniše u svakoj izvedenoj klasi. U izvedenoj klasi u kojoj virtualna funkcija nije definisana, važi značenje te virtualne funkcije iz osnovne klase.
- Deklaracija neke virtualne funkcije u svakoj izvedenoj klasi mora da se u potpunosti slaže sa deklaracijom te funkcije u osnovnoj klasi (broj i tipovi argumenata, kao i tip rezultata).
- Ako se u izvedenoj klasi deklarira neka funkcija koja ima isto ime kao i virtualna funkcija iz osnovne klase, ali različit broj i/ili tipove argumenata, onda ona sakriva (a ne redefiniše) sve ostale istoimene funkcije iz osnovne klase.

Dinamičko vezivanje

- Pokazivač na objekat izvedene klase može se implicitno konvertovati u pokazivač na objekat osnovne klase (pokazivaču na objekat osnovne klase može se dodeliti pokazivač na objekat izvedene klase direktno, bez eksplicitne konverzije). Isto važi i za reference. Ovo je interpretacija činjenice da se objekat izvedene klase može smatrati i objektom osnovne klase.
- Pokazivaču na objekat izvedene klase može se dodeliti pokazivač na objekat osnovne klase samo uz eksplicitnu konverziju. Ovo je interpretacija činjenice da objekat osnovne klase nema sve osobine izvedene klase.
- Objekat osnovne klase može se inicijalizovati objektom izvedene klase, i objektu osnovne klase može se dodeliti objekat izvedene klase bez eksplicitne konverzije. To se obavlja „odsecanjem“ članova izvedene klase koji nisu i članovi osnovne klase.
- Virtualni mehanizam se aktivira ako se objektu pristupa preko reference ili pokazivača:

```

class Base {
public:
    virtual void f();
    //...
};

class Derived : public Base {
public:
    virtual void f();
};

void g1(Base b) {
    b.f();
}

void g2(Base *pb) {
    pb->f();
}

void g3(Base &rb) {
    rb.f();
}

```

```

void main () {
    Derived d;
    g1(d);
    g2(&d);
    g3(d);
    Base *pb=new Derived;
    pb->f();
    Derived &rd=d;
    rd.f();
    Base b=d;
    b.f();
    delete pb;
    pb=&b;
    pb->f();
    // poziva se Base::f
}

```

- Postupak koji obezbeđuje da se funkcija koja se poziva određuje u vreme izvršavanja, a ne prevođenja, naziva se *dinamičko vezivanje* (engl. *dynamic binding*).

Virtualni destruktor

- Destruktor je specifična funkcija članica klase koja pretvara „živi“ objekat u „običnu gomilu bitova u memoriji“. Zbog toga destruktor može da bude virtualna funkcija.
- Virtualni mehanizam obezbeđuje da se pozove odgovarajući destruktor (osnovne ili izvedene klase) kada se objektu pristupa posredno:

```

class Base {
public:
    virtual ~Base(); // destruktor je virtualan
    //...
};

class Derived : public Base {
public:
    ~Derived(); // destruktor je virtualan
    //...
};

void release(Base *pb) { delete pb; }

void main () {
    Base *pb=new Base;
    Derived *pd=new Derived;
    release(pb); // poziva se ~Base
    release(pd); // poziva se ~Derived
}

```

- Kada klasa ima neku virtualnu funkciju, verovatno i njen destruktor (ako ga ima) treba da bude virtualan.
- Unutar virtualnog destruktora izvedene klase ne treba eksplicitno pozivati destruktor osnovne klase, jer se on uvek implicitno poziva. Definisanjem destruktora kao virtualne funkcije obezbeđuje se da se dinamičkim vezivanjem tačno određuje koji će destruktor (osnovne ili izvedene klase) biti *prvo* pozvan; destruktor osnovne klase se uvek izvršava (ili kao jedini ili posle destruktora izvedene klase).
- Konstruktork je funkcija koja od „obične gomile bitova u memoriji“ pravi „živi“ objekat. Pošto se konstruktork poziva pre nego što je objekat nastao, nema smisla da bude virtualan, pa C++ to ne dozvoljava. Kada se definiše objekat, uvek se navodi i tip (klasa) kome pripada, pa je određen i konstruktork koji se poziva.

Nizovi i izvedene klase

- Objekat izvedene klase je vrsta objekta osnovne klase. Međutim, niz objekata izvedene klase nije vrsta niza objekata osnovne klase. Uopšte, neka kolekcija objekata izvedene klase nije vrsta kolekcije objekata osnovne klase.
- Na primer, iako je automobil vrsta vozila, parking za automobile nije i parking za sve vrste vozila, jer na parking za automobile ne mogu da stanu i kamioni (koji su takođe vozila). Ili, ako korisnik neke funkcije prosledi toj funkciji korpu banana (banana je vrsta voća), ne bi valjalo da mu ta funkcija vrati korpu u kojoj je jedna šljiva (koja je takođe vrsta voća), smatrajući da je korpa banana isto što i korpa bilo kakvog voća.
- Ako se računa sa nasleđivanjem, u programu ne treba koristiti nizove objekata, već nizove pokazivača na objekte. Ako se formira niz objekata izvedene klase i on prenese kao niz objekata osnovne klase (što, po prethodno rečenom, semantički nije ispravno, ali je moguće), može doći do greške:

```
class Base {
public: int bi;
};

class Derived : public Base {
public: int di;
};

void f(Base *b) { cout<<b[2].bi; }

void main () {
    Derived d[5];
    d[2].bi=77;
    f(d);           // neće se ispisati 77
}
```

- U prethodnom primeru, funkcija f smatra da je dobila niz objekata osnovne klase koji su kraći (nemaju sve članove) od objekata izvedene klase. Kada joj se prosledi niz objekata izvedene klase (koji su duži), funkcija nema načina da odredi da se niz sastoji samo od objekata izvedene klase. Rezultat je, u opštem slučaju, neodređen.
- Pored navedene greške, fizički nije moguće direktno smeštati objekte izvedene klase u niz objekata osnovne klase. Objekti izvedene klase su duži, a za svaki element niza je odvojen samo prostor koji je dovoljan za smeštanje objekta osnovne klase.
- Zbog svega što je rečeno, kolekcije (nizove) objekata treba realizovati kao nizove pokazivača na objekte:

```
void f(Base **b, int i) { cout<<b[i]->bi; }

void main () {
    Base b1,b2;
    Derived d1,d2,d3;
    Base *b[5];

    // b se može konvertovati u tip Base**
    b[0]=&d1; b[1]=&b1; b[2]=&d2; // konverzije Derived* u Base*
    b[3]=&d3; b[4]=&b2;
    d2.bi=77;
    f(b,2);           // ispisace se 77
}
```

- Kako je objekat izvedene klase vrsta objekta osnovne klase, C++ dozvoljava implicitnu konverziju pokazivača Derived* u Base* (prethodni primer). Zbog logičkog pravila da niz objekata izvedene klase nije vrsta niza objekata osnovne klase, a kako se nazovi

ispravno realizuju pomoću nizova pokazivača, C++ ne dozvoljava implicitnu konverziju pokazivača Derived** (u koji se može konvertovati tip niza pokazivača na objekte izvedene klase) u Base** (u koji se može konvertovati tip niza pokazivača na objekte osnovne klase). Za prethodni primer nije dozvoljeno:

```
void main () {
    Derived *d[5]; // d je tipa Derived**
    //...
    f(d,2);        // nije dozvoljena konverzija Derived** u Base**
}
```

Apstraktne klase

- Čest je slučaj da neka osnovna klasa nema nijedan konkretan primerak (objekat), već samo predstavlja generalizaciju izvedenih klasa.
 - Na primer, svi izlazni, znakovno orijentisani uređaji računara imaju funkciju za ispis jednog znaka, ali se u osnovnoj klasi izlaznog uređaja ne može definisati način ispisa tog znaka, već je to specifično za svaki uređaj posebno. Ili, ako iz osnovne klase osoba izvedemo dve klase muškaraca i žena, onda klasa osoba ne može imati primerke, jer ne postoji osoba koja nije ni muškog ni ženskog pola.
 - Klasa koja nema instance (objekte), već su iz nje samo izvedene druge klase, naziva se *apstraktna klasa* (engl. *abstract class*).
 - U jeziku C++, apstraktna klasa sadrži bar jednu virtuelnu funkciju članicu koja je u njoj samo deklarirana, ali ne i definisana. Definicije te funkcije daje izvedene klase. Ovakva virtuelna funkcija naziva se čistom virtuelnom funkcijom. Njena deklaracija u osnovnoj klasi završava se sa =0:
- ```
class OcharDevice {
public:
 virtual int put (char) =0; // čista virtuelna funkcija
};
```
- U jeziku C++ apstraktna klasa je klasa koja sadrži bar jednu čistu virtuelnu funkciju. Ovakva klasa ne može imati instance, već se iz nje izvode druge klase. Ako se u izvedenoj klasi ne navede definicija neke čiste virtuelne funkcije iz osnovne klase, i ova izvedena klasa je takođe apstraktna.
  - Pokazivači i reference na apstraktnu klasu mogu da se definišu, ali oni ukazuju na objekte izvedenih konkretnih (neapstraktnih) klasa.

## Višestruko nasleđivanje

### Šta je višestruko nasleđivanje?

- Nekad postoji potreba da izvedena klasa ima osobine više osnovnih klasa istovremeno. Tada se radi o *višestrukome nasleđivanju* (engl. *multiple inheritance*).
- Na primer, motocikl sa prikolicom je vrsta motocikla, ali i vrsta vozila sa tri točka. Pri tom, motocikl nije vrsta vozila sa tri točka, niti je vozilo sa tri točka vrsta motocikla, već su ovo dve različite klase. Klasa motocikala sa prikolicom nasleđuje obe ove klase.

- Klasa se deklarise kao naslednik više klasa tako što se u zaglavlju deklaracije, iza znaka `;`, navode osnovne klase razdvojene zarezima. Ispred svake osnovne klase treba da stoji reč `public`. Na primer:

```
class Derived : public Base1, public Base2, public Base3 {
//...
};
```

- Sva navedena pravila o nasleđenim članovima važe i ovde. Konstruktori svih osnovnih klasa pozivaju se pre poziva konstruktora članova izvedene klase i pre izvršavanja tela konstruktora izvedene klase. Konstruktori osnovnih klasa pozivaju se po redosledu deklarisanja tih klasa u zaglavlju izvedene klase. Destruktori osnovnih klasa izvršavaju se na kraju, posle izvršavanja tela destruktora izvedene klase i destruktora članova.

### *Virtualne osnovne klase*

- Posmatrajmo sledeći primer:

```
class B { /*...*/ };
class X : public B { /*...*/ };
class Y : public B { /*...*/ };
class Z : public X, public Y { /*...*/ };
```

- U ovom primeru klase X i Y nasleđuju klasu B, a klasa Z klase X i Y. Klasa Z ima sve što imaju X i Y. Kako svaka od klasa X i Y ima po jedan primerak članova klase B, to će klasa Z imati dva skupa članova klase B. Njih je moguće razlikovati pomoću operatora `::` (npr. `z.X::i` ili `z.Y::i`).

- Ako ovo nije potrebno, klasu B treba deklarirati kao virtualnu osnovnu klasu:

```
class B { /*...*/ };
class X : virtual public B { /*...*/ };
class Y : virtual public B { /*...*/ };
class Z : public X, public Y { /*...*/ };
```

- Sada klasa Z ima samo jedan skup članova klase B.
- Ako neka izvedena klasa ima virtualne i nevirtualne osnovne klase, onda se konstruktori virtualnih osnovnih klasa pozivaju pre konstruktora nevirtualnih osnovnih klasa, po redosledu deklarisanja. Svi konstruktori osnovnih klasa se, naravno, pozivaju pre konstruktora članova i konstruktora izvedene klase.

## Privatno i zaštićeno izvođenje

### *Šta je privatno i zaštićeno izvođenje?*

- Ključna reč `public` u zaglavlju deklaracije izvedene klase znači da je osnovna klasa javna, odnosno da su svi javni članovi osnovne klase ujedno i javni članovi izvedene klase. Privatni članovi osnovne klase nisu dostupni izvedenoj klasi, a zaštićeni članovi osnovne klase ostaju zaštićeni i u izvedenoj klasi. Ovakvo izvođenje se u jeziku C++ naziva još i javno izvođenje.

- U zaglavlje deklaracije može se, ispred imena osnovne klase, umesto reči `public` upisati reč `private`, što se i podrazumeva ako se ne navede ništa drugo. U ovom slučaju javni i zaštićeni članovi osnovne klase postaju privatni članovi izvedene klase. Ovakvo izvođenje se u jeziku C++ naziva privatno izvođenje.

- U zaglavlje deklaracije može se, ispred imena osnovne klase, upisati reč `protected`. Tada javni i zaštićeni članovi osnovne klase postaju zaštićeni članovi izvedene klase. Ovakvo izvođenje se u jeziku C++ naziva zaštićeno izvođenje.

- U svakom slučaju, privatni članovi osnovne klase nisu dostupni izvedenoj klasi. Izvedena klasa može samo nadalje „sakriti“ zaštićene i javne članove osnovne klase izborom načina izvođenja.

- U slučaju privatnog i zaštićenog izvođenja, kada izvedena klasa smanjuje nivo prava pristupa do javnih i zaštićenih članova osnovne klase, može se ovaj nivo vratiti na početni ek-splcitnim navođenjem deklaracije javnog ili zaštićenog člana osnovne klase u javnom ili zaštićenom delu izvedene klase. U svakom slučaju, izvedena klasa ne može povećati nivo prava pristupa do člana osnovne klase.

```
class Base {
private:
 int bpriv;
protected:
 int bprot;
public:
 int bpub;
};

class PrivDerived : Base { // privatno izvođenje
protected:
 Base::bprot; // vraćanje na nivo protected
public:
 PrivDerived () {
 bprot=2; bpub=3; // može se pristupiti
 }
};

class ProtDerived : protected Base { // zaštićeno izvođenje
//...
};

void main () {
 PrivDerived pd;
 pd.bpub=0; // greška: bpub nije javni član
}
```

- Pokazivač na izvedenu klasu može se implicitno konvertovati u pokazivač na javnu osnovnu klasu. Pokazivač na izvedenu klasu može se implicitno konvertovati u pokazivač na privatnu osnovnu klasu samo unutar izvedene klase, jer se samo unutar nje zna da je ona izvedena. Isto važi i za reference.

### *Semantička razlika između privatnog i javnog izvođenja*

- Javno izvođenje realizuje koncept nasleđivanja, koji je iskazan relacijom „B je vrsta A“ (engl. *a-kind-of*). Ova relacija podrazumeva da izvedena klasa ima sve što i osnovna, što znači da je sve što je dostupno korisniku osnovne klase, dostupno i korisniku izvedene klase. U jeziku C++ to znači da javni članovi osnovne klase treba da budu javni i u izvedenoj klasi.

- Privatno izvođenje ne odslikava ovu relaciju, jer korisnik izvedene klase ne može da pristupi onome čemu je mogao pristupiti u osnovnoj klasi. Javnim članovima osnovne klase mogu pristupiti samo funkcije članice izvedene klase, što znači da objekat izvedene klase u sebi sakriva podbjekat osnovne klase. Zato privatno izvođenje realizuje sasvim drugu relaciju, relaciju „A je deo od B“ (engl. *a-part-of*). Ovo je suštinski različito od relacije nasleđivanja.
- Pošto privatno izvođenje realizuje relaciju „A je deo od B“, ono je semantički ekvivalentno sa implementacijom kada klasa B sadrži člana koji je tipa A.
- Prilikom projektovanja, treba strogo voditi računa o tome u kojoj su od ove dve relacije neke dve uočene klase. U zavisnosti od toga treba izabrati način izvođenja.
- Ako je relacija između dve klase „A je deo od B“, izbor između privatnog izvođenja i članstva zavisí od manje važnih detalja: da li je potrebno redefinisati virtualne funkcije klase A, da li je unutar klase B potrebno konvertovati pokazivače, da li klasa B treba da sadrži jedan ili više primeraka klase A i slično.

## Glava 8

# Uvod u objektno modelovanje

## Apstraktni tipovi podataka

- Apstraktni tipovi podataka predstavljaju realizacije struktura podataka sa pridruženim protokolima (operacijama i definisanim načinom i redosledom pozivanja tih operacija). Na primer, *red* (engl. *queue*) je struktura elemenata koja ima operacije stavljanja i uzimanja elemenata u strukturu, pri čemu se elementi uzimaju po istom redosledu po kom su stavljani.
- Kada se realizuju strukture podataka (apstraktni tipovi podataka), najčešće nije bitno koji je tip elementa strukture, važan je samo skup operacija. Način realizacija tih operacija ne zavise od tipa elementa, već samo od tipa strukture.
- Za realizaciju apstraktnih tipova podataka kod kojih tip nije bitan, u jeziku C++ postoje *šabloni* (engl. *templates*). Šablon klase predstavlja definiciju čitavog skupa klase koje se razlikuju samo po tipu elementa i, eventualno, po dimenzijama. Šabloni klase se ponekad nazivaju i generičkim klasama.
- Konkretna klasa generisana iz šablona dobija se navođenjem stvarnog tipa elementa.
- Formalni argumenti šablona zadaju se u zaglavlju šablona:

```
template <class T>
class Queue {
public:
 Queue ();
 ~Queue ();
 void put (const T&);
 T get ();
};
//...
```
- Konkretna generisana klasa dobija se samo navođenjem imena šablona, uz definisanje stvarnih argumenata šablona. Stvarni argumenti šablona su tipovi i, eventualno, celobrojne dimenzije. Konkretna klasa se generiše na mestu navođenja, u fazi prevodenja. Na primer, red događaja može se kreirati na sledeći način:

```
class Event;
Queue<Event*> que;

que.put(e);
if (que.get()->isUrgent()) ...
```
- Generisanje je samo stvar automatskog generisanja parametrizovanog koda istog oblika, a nema nikakve veze sa izvršavanjem. Generisane klase nemaju nikakve posebne međusobne veze.

### Projektovanje apstraktnih tipova podataka

- Apstraktni tipovi podataka (strukture podataka) su veoma često korišćeni elementi svakog programa. Projektovanje biblioteke klasa koje realizuju standardne strukture podataka veoma je delikatan posao. Ovdje će biti prikazana konstrukcija dve česte linearne strukture podataka:

1. Kolekcija (engl. *collection*) je linearna, neuređena struktura elemenata koja ima samo operacije stavljanja elementa i izbacivanja datog elementa iz strukture. Redosled elemenata nije bitan, a elementi se mogu i ponavljati.

2. Red (engl. *queue*) je linearna, uređena struktura elemenata. Elementi su uređeni po redosledu stavljanja. Operacija uzimanja vraća element koji je prvi stavljen u strukturu.

- Važan koncept pridružen strukturama je pojam *iteratora* (engl. *iterator*). Iterator je objekat pridružen linearnoj strukturi koji služi za pristup redom elementima strukture. Iterator ima operacije za postavljanje na početak strukture, za pomeranje na sledeći element strukture, za pristup do tekućeg elementa na koji ukazuje i operaciju za ispitivanje da li je došao do kraja strukture. Za svaku strukturu može se napraviti proizvoljno mnogo objekata-iteratora i svaki od njih pamt svoju poziciju.

- Suština koncepta iteratora je da se obezbedi sekvencijalni pristup do elemenata agregatne strukture, bez izlaganja njene interne realizacije. S druge strane, loše je opterećivati sam interfejs date strukture operacijama koje obezbeđuju takav pristup. Zato se obezbeđuje posebna klasa iteratora pridružena datoj strukturi, koja je odgovorna za obilazak strukture, poznaje njenu internu predstavu, i za koju se onda može napraviti proizvoljan broj instanci koje nezavisno iteriraju. Ovakvo rešenje predstavlja projektni obrazac (engl. *design pattern*) *Iterator* (ili *Cursor*).

- Pri realizaciji biblioteke klasa za strukture podataka, bitno je razlikovati *protokol* strukture koji definiše njenu semantiku, od njene *implementacije*.

- Protokol strukture određuje značenje njenih operacija, potreban način ili redosled pozivanja itd.

- Implementacija se odnosi na način smeštanja elemenata u memoriju, organizaciju njihovih veze itd. Važan aspekt implementacije je da li je ona ograničena ili nije. Ograničena realizacija se oslanja na fiksno dimenzionisani niz elemenata, dok se neograničena realizacija odnosi na dinamičku strukturu (najčešće listu).

- Na primer, protokol reda izgleda otprilike ovako:

```
template <class T>
class Queue {
public:
 virtual IteratorQueue<T>* createIterator() const = 0;

 virtual void put (const T& t) = 0;
 virtual T get () = 0;
 virtual void clear () = 0;

 virtual const T& first () const = 0;
 virtual int isEmpty () const = 0;
 virtual int isFull () const = 0;
 virtual int length () const = 0;
 virtual int location (const T& t) const = 0;
};
```

- Da bi se korisniku obezbedile obe realizacije (ograničena i neograničena), postoje dve klase izvedene iz navedene apstraktne klase koja definiše interfejs reda. Jedna od njih podrazumeva ograničenu (engl. *bounded*) realizaciju, a druga neograničenu (engl. *unbounded*). Na primer:

```
template <class T, int N>
class QueueB : public Queue<T> {
public:
 QueueB () {}
 QueueB (const Queue<T>& q) :
 virtual -QueueB () {}

 Queue<T>& operator= (const Queue<T>& q);

 virtual IteratorQueue<T>* createIterator() const;

 virtual void put (const T& t);
 virtual T get ();
 virtual void clear ();

 virtual const T& first () const;
 virtual int isEmpty () const;
 virtual int isFull () const;
 virtual int length () const;
 virtual int location (const T& t) const;
};
```

- Treba obratiti pažnju na način kreiranja iteratora. Korisniku je dovoljan samo opšti, zajednički interfejs iteratora. Korisnik ne treba ništa da zna o specifičnostima realizacije iteratora i njegovoj vezi sa konkretnom izvedenom klasom reda. Zato je definisana osnovna apstraktna klasa iteratora, iz koje su izvedene klase za iteratore vezane za dve posebne realizacije reda:

```
template <class T>
class IteratorQueue {
public:
 virtual ~IteratorQueue () {}

 virtual void reset() = 0;
 virtual int next () = 0;

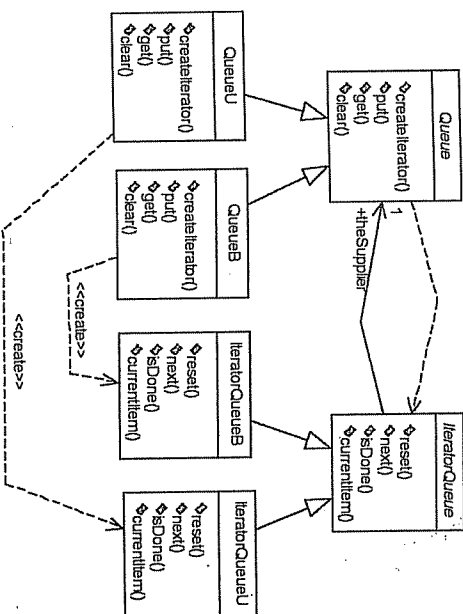
 virtual int isDone() const = 0;
 virtual const T* currentItem() const = 0;
};
```

- Izvedene klase reda praviće posebne, njima specifične iteratore koji se uklapaju u zajednički interfejs iteratora:

```
template <class T, int N>
IteratorQueue<T>* QueueB<T, N>::createIterator() const {
 return new IteratorQueueB<T, N>(this);
}
```

- Suština ovog rešenja je da se korisnici ovog podistema oslanjaju samo na apstraktne interfeise *Queue* i *IteratorQueue*, odnosno da ne znaju za specifičnosti konkretnih implementacija. Čak i samo pravljenje iteratora obavlja se preko apstraktnog interfejsa, tj. operacije *createIterator()*, koju definišu konkretne izvedene klase *QueueB* i *QueueU*, tako što prave konkretne specifične iteratore. Ovakav pristup znatno povećava fleksibilnost softvera.

- Ovo rešenje, tj. operacija `createIterator()` predstavlja projektni obrazac (engl. *design pattern*) *Factory Method* (ili *Virtual Constructor*), koji se sastoji u obezbeđivanju interfejsa za pravljenje objekta, pri čemu konkretne izvedene klase odlučuju koji konkretni objekat da naprave.
- Relacije između opisanih klasa prikazane su na sledećem dijagramu:



- Sama realizacija ograničene i neograničene strukture oslanja se na dve klase koje imaju sledeće interfejse:

```

template <class T>
class Unbounded {
public:
 Unbounded ();
 ~Unbounded ();
 Unbounded(T& operator= (const Unbounded(T&)&);
 void append (const T&);
 void insert (const T&, int at=0);
 void remove (const T&);
 void clear ();
};

int isEmpty () const;
int isFull () const;
int length () const;
const T& first () const;
const T& last () const;
const T& itemAt (int at) const;
T& itemAt (int at);
location (const T&) const;
};

```

```

template <class T, int N>
class Bounded {
public:
 Bounded ();
 Bounded (const Bounded<T,N>&);
 ~Bounded ();
};

```

```

Bounded<T,N>& operator= (const Bounded<T,N>&);

```

```

void append (const T&);
void insert (const T&, int at=0);
void remove (const T&);
void remove (int at=0);
void clear ();

```

```

int isEmpty () const;
int isFull () const;
int length () const;
const T& first () const;
const T& last () const;
const T& itemAt (int at) const;
T& itemAt (int at);
location (const T&) const;
int

```

- Definisana struktura može se koristiti kao u sledećem primeru:

```

class Event {
//...
};

typedef QueueB<Event*, MAXEV> EventQueue;
typedef IteratorQueue<Event*> Iterator;

//...
EventQueue que;
que.put (e);

Iterator* it=que.createIterator();
for (; it->isDone(); it->next())
 (*it->currentItem())->handle();
delete it;

```

- Promena orijentacije na ograničeni red veoma je jednostavna. Ako se želi neograničeni red, dovoljno je promeniti samo:

```

typedef QueueU<Event*> EventQueue;

```

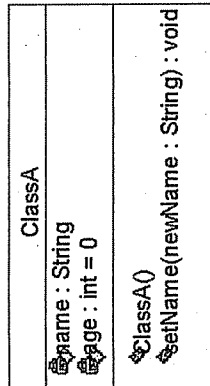
- Kompletna kôd za ovu biblioteku apstraktnih tipova podataka dat je u Praktikum.

## Modelovanje strukture – klase i osnovne relacije između klasa

- Klasa je osnovna jedinica strukturnog modela sistema.
- Klasa je veoma retko izolovana. Ona dobija smisao samo uz druge klase sa kojima je u relaciji. Osnovne relacije između klasa su: asocijacija, zavisnosti i nasleđivanje.

### Klasa, atributi i operacije

- Klasom se modeluje apstrakcija. Klasa je opis skupa objekata koji dele iste atribute, operacije, relacije i semantiku.
- Atribut je imenovano svojstvo entiteta. Njime se opisuje opseg vrednosti koje instance tog svojstva mogu da imaju.
- Operacija je implementacija usluge koja se može zatražiti od bilo kog objekta klase da bi se uticalo na ponašanje.
- Klasa se prikazuje pravougaonim simbolom u kome mogu postojati posebni odeljci za ime klase, atribute i operacije:

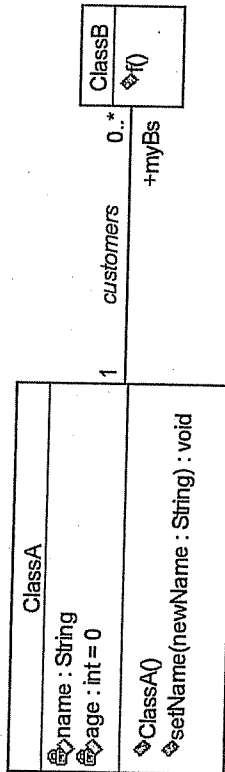


### Asocijacija

- Asocijacija (pridruživanje, engl. *association*) je relacija između klasa čiji su objekti na neki način strukturno povezani. Ta veza između objekata klase postoji određeno duže vreme, a ne samo tokom trajanja izvršavanja operacije jednog objekta koju poziva drugi objekat. Instanca asocijacije naziva se *vezom* (engl. *link*) i postoji između objekata datih klasa.
- Asocijacija se predstavlja punom linijom koja povezuje dve klase. Asocijacija može imati ime koje opisuje njeno značenje. Svaka strana u asocijaciji ima svoju *ulogu* (engl. *role*) koja se može naznačiti na strani date klase.
- Na svakoj strani asocijacije može se definisati kardinalnost (multiplikativnost, engl. *multiplicity*) pomoću sledećih oznaka:

- 1. tačno jedan
- \* proizvoljno mnogo
- 1..\* jedan i više
- 0..1 nula ili jedan
- 3..7 zadati opseg
- i slično.

#### Primer:



- Druge posebne karakteristike svake strane asocijacije je *navigabilnost* (engl. *navigability*): sposobnost da se sa te strane (od objekta sa jedne strane veze) dospe do druge strane (do objekta sa druge strane veze). Prema ovom svojstvu, asocijacija može biti simetrična ili asimetrična.
- Primer: prikazana asocijacija se na jeziku C++ na strani klase A realizuje na sledeći način, ukoliko postoji mogućnost navigacije prema klasi B:

```
class B;
class A {
public:

 // Funkcije za uspostavljanje veza asocijacije:
 void insert (B* b) { myBs.add(b); }
 void remove (B* b) { myBs.remove(b); }
private:
 Collection<B*> myBs;
};
```

- Na strani klase B, ukoliko postoji mogućnost navigacije prema klasi A, ova veza se uspostavlja pomoću konstruktora klase B (jer je kardinalnost tačno 1):

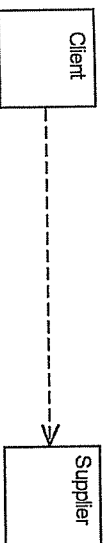
```
class A;
class B {
public:
 B(A* a) { myA=a; }

private:
 A* myA;
};
```

- Kod navedene realizacije na jeziku C++, potrebno je obratiti pažnju na sledeće. U deklaraciji interfejsa klase B nije potrebna potpuna definicija klase A, već samo prosta deklaracija `class A;`, jer je objekat B vezan za objekat klase A preko pokazivača. Samo u implementaciji neke složenije operacije potrebna je potpuna definicija klase A. Kako se implementacije ovih funkcija nalaze u modulu B.cpp, samo ovaj modul zavisi od modula sa interfejsom klase A, dok modul B.h ne zavisi. Na ovaj način se drastično smanjuje zavisnosti između modula i vreme prevodenja.

### Zavisnost

- Relacija zavisnosti (engl. *dependency*) postoji ako klasa A na neki način koristi usluge klase B. To može biti npr. odnos klijent-server (klasa A poziva funkcije klase B) ili odnos instancijalizacije (klasa A pravi objekte klase B).
- Za realizaciju ove relacije između klase A i B potrebno je da interfejs ili implementacija klase A „zna“ za definiciju klase B. Tako je klasa A zavisna od klase B.
- Oznaka:



- Značenje relacije može da se navede kao stereotip relacije na dijagramu, npr. <<call>> ili <<create>>.
- Ako klasa Client koristi usluge klase Supplier tako što poziva operacije objekata ove klase (odnos klijent-server), onda ona tipično „vidi“ ove objekte kao argumente svojih operacija. U ovom slučaju, za implementaciju na jeziku C++, interfejsu klase Client nije potrebna definicija klase Supplier, već samo njenoj implementaciji:

```

class Supplier;

class Client {
public:

 void afunction (Supplier*);
};

// Implementacija:
void Client::afunction (Supplier* s) {

 s->doSomething();
}

// Pri implementaciji na jeziku C++, ako je potrebno dobiti notaciju prenosa po vrednosti, a
// zadržati navedenu pogodnost slabije zavisnosti između modula, onda se argument prenosi
// preko reference na konstantu:

void Client::afunction (const Supplier& s) {
 s.doSomething();
}

// Ako klasa Client instancijalizuje klasu Supplier, onda je realizacija nalik na:
Supplier* Client::createSupplier (/some_arguments/) {
 return new Supplier (/some_arguments/);
}

```

### Nasledivanje (generalizacija/specijalizacija)

- Nasledivanje (engl. *inheritance*) predstavlja relaciju generalizacije, odnosno specijalizacije, zavisno u kom smeru se posmatra. Oznaka:



- Relacija generalizacije/specijalizacije (engl. *generalization/specialization*) predstavlja relaciju koja ima dve važne semantičke manifestacije:
  - (a) Nasledivanje: nasledena (izvedena) klasa implicitno poseduje (nasleduje) sve atribute, operacije i asocijacije osnovne klase (važi i tranzitivnost).
  - (b) Supstitucija (engl. *substitution*): objekat (instancija) izvedene klase može se naći svugde gde se očekuje objekat osnovne klase (važi i tranzitivnost). Za strukturni aspekt sistema ovo pravilo ima sledeću binu manifestaciju: ako u nekoj asocijaciji učestvuje osnovna klasa, onda u nekoj vezi kao instanci te asocijacije mogu učestvovati objekti svih klasa izvedenih (neposredno ili posredno) iz te klase.

- Realizacija:

```
class Derived : public Base {

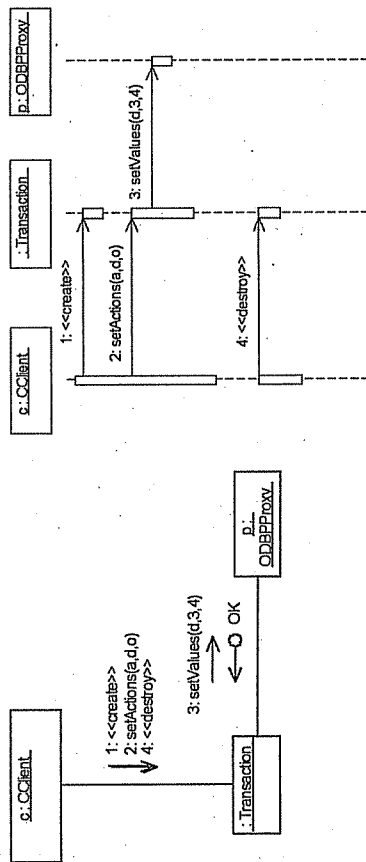
}
```

- Zbog ovako definisane semantike osnovnih relacija između klasa (pervenstveno asocijacije i nasledivanja), softverski sistemi (aplikacije) koji su modelovani objektno imaju jedno opšte svojstvo: njihova struktura u vreme izvršavanja može se apstraktno posmatrati kao *tipizirani graf*, jer se sastoji iz objekata (instanci klase) povezanih vezama (instancama asocijacija). Dakle, objekti predstavljaju čvorove, a veze grane jednog grafa. Pri tom, objekti kao čvorovi grafa imaju svoje tipove (to su klase čije su ovo instance), kao i veze koje su instance odgovarajućih asocijacija. Na stranama svake veze koja je instanca neke asocijacije nalaze se instance onih klasa koje povezuje ta asocijacija, ili klasa izvedenih iz njih (uključujući i tranzitivnost nasledivanja).

### Modelovanje ponašanja – interakcije

- Ponašanje sistema se uglavnom realizuje interakcijama između objekata. *Interakcija* (engl. *interaction*) je ponašanje koje se sastoji od skupa poruka koje se razmenjuju između objekata u nekom kontekstu da bi se ispunila neka svrha. *Poruka* (engl. *message*) je specifikacija komunikacije između objekata koja prenosi informaciju sa očekivanjem da će se pokrenuti neka aktivnost.
- *Dijagram interakcije* (engl. *interaction diagram*) opisuje ponašanje i odnosi se na neki kontekst – deo sistema čije ponašanje opisuje. To može biti neka funkcionalnost sistema ili neka operacija klase.
- Postoje dve vrste dijagrama interakcije: dijagram kolaboracije i dijagram sekvence. Ovi dijagrami predstavljaju dva različita pogleda na semantički istu stvar (istu interakciju), samo što naglašavaju različite aspekte te interakcije.

- *Dijagram kolaboracije* (engl. *collaboration diagram*) je dijagram interakcije koji naglašava strukturu organizaciju objekata koji razmenjuju poruke. Grafički, ovaj dijagram je skup čvorova koji predstavljaju objekte i skup linija koji ih povezuju i preko kojih idu poruke.
- *Dijagram sekvence* (engl. *sequence diagram*) je dijagram interakcije koji naglašava vremenski redosled poruka. Ovaj dijagram prikazuje objekte poredane po x osi, dok se poruke redaju kao horizontalne linije po y osi, pri čemu vreme raste nadole.



Dragan Milićev, Ljubica Lazarević, Jelena Marušić

# Objektno orijentisano programiranje na jeziku C++

## Praktikum



**Dragan Milićev**

# **Primeri**

## **Glava 1**

Pregled osnovnih koncepata OOP na jeziku C++

## **Glava 2**

Pregled osnovnih koncepata nasleđenih iz jezika C

## **Glava 3**

Elementi jezika C++ koji nisu objektno orijentisani

## **Glava 4**

Klase i preklapanje operatora

## **Glava 5**

Nasledivanje

## **Glava 6**

Uvod u objektno modelovanje

# Glava 1

## Pregled osnovnih koncepata OOP na jeziku C++

### Zadatak

Napisati kompletan kôd za primer apstrakcija Osoba, Zena i Maloletnik iz skripta.

### Rešenje

```
// Primer 1: Osnovni koncepti: klase, konstruktori,
// nasledjivanje, polimorfizam
```

```
#include <stdio.h>
```

```
// Klasa: Osoba
```

```
class Osoba {
public:
 Osoba(char* ime, int godine);
 virtual void kosi();
protected:
 char* ime;
 int god;
};
```

```
Osoba::Osoba (char* i, int g) {
 ime=i;
 god=(g>0 && g<=100)?g:0;
}
```

```
void Osoba::kosi() {
 printf("Ja sam %s i imam %d godina.\n",ime,god);
}
```

```
// Klasa: Maloletnik
```

```
class Maloletnik : public Osoba {
public:
 Maloletnik(char* ime, char* staratelj, int godine);
 void koOdgovara();
private:
 char* staratelj;
```

```
};
Maloletnik::Maloletnik (char* i, char* s, int g) : Osoba(i,g) {
 staratelj=s;
}
```

```
void Maloletnik::koodgovara () {
 printf("Ja sam %s i za mene odgovara %s.\n", ime, staratelj);
}
```

```
// Klasa: Zena
```

```
class Zena : public Osoba {
public:
 Zena(char* ime, char* devojacko, int godine);
 virtual void kosi();
```

```
private:
 char* devojacko;
};
```

```
Zena::Zena (char *i, char *d, int g) : Osoba(i,g) {
 devojacko=d;
}
```

```
void Zena::kosi () {
 printf("Ja sam %s, devojacko prezime %s.\n", ime, devojacko);
}
```

```
// Funkcija: ispitaaj
void ispitaaj (Osoba *hejTi) {
 hejTi->kosi();
}
```

```
// Glavni program
```

```
void main () {
 Osoba otac("Petar Petrovic",40);
 Zena majka("Milka Petrovic", "Mirovic", 35);
 Maloletnik dete("Milan Petrovic", "Petar Petrovic", 12);
 ispitaaj(otac);
 ispitaaj(majka);
 ispitaaj(dete);
 dete.koodgovara();
}
```

## Zadatak

Sačiniti jednostavnu realizaciju klase kompleksnih brojeva.

### Rešenje

```
// Primer 2: Osnovni koncepti
```

```
#include <stdio.h>
```

```
// Klasa: Complex
```

```
class Complex {
public:
 Complex(float real, float imag);
```

```
Complex add(Complex);
Complex sub(Complex);
```

```
float Re();
float Im();
```

```
void print();
```

```
private:
 float real, imag;
};
```

```
Complex::Complex (float r, float i) {real=r; imag=i;}
```

```
Complex Complex::add (Complex c) {
 return Complex(real+c.real, imag+c.imag);
}
```

```
Complex Complex::sub (Complex c) {
 return Complex(real-c.real, imag-c.imag);
}
```

```
float Complex::Re() {return real;}
float Complex::Im() {return imag;}
```

```
void Complex::print() {
 printf("(%f,%f)", real, imag);
}
```

```
// Glavni program
```

```
void main () {
 Complex c1(3.5, -17.5), c2(-3.5, 17.5), c3(0, 0);
 c3=c1.add(c2);
 c1=c2.sub(c3);
 printf("c1="); c1.print(); printf("\n");
 printf("c2="); c2.print(); printf("\n");
 printf("c3="); c3.print(); printf("\n");
}
```

## Zadaci za samostalan rad

1. Realizovati klasu Counter koja će imati operaciju inc. Svaki objekat ove klase treba da odbrjava pozive svoje operacije inc. Na početku života, vrednost brojača objekta postavlja se na nulu, a pri svakom pozivu operacije inc povećava se za jedan, i vraća se novodobijena vrednost.
2. Realizovati klasu koja predstavlja člana biblioteke. Svaki član biblioteke ima svoj članski broj, ime i prezime, i trenutno stanje računa za naplatu članarine. Ova klasa treba da ima operaciju za naplatu članarine, koja će sa računa člana skinuti odgovarajuću konstantnu sumu. Biblioteka poseduje i posebnu kategoriju počasnih članova, kojima se ne naplaćuje članarina. Napraviti niz pokazivača na objekte klase članova biblioteke, i definisati funkciju za naplatu članarine svim članovima. Ova funkcija treba da prolazi kroz niz članova i vrši naplatu pozivom operacije klase za naplatu, bez obzira na to što se u nizu mogu nalaziti i redovni i počasni članovi.

## Pregled osnovnih koncepata nasleđenih iz jezika C

### Zadatak

Napisati funkciju koja prebrojava različite elemente u datom celobrojnom nizu.

### Rešenje

```
// Theme: Basic Structural programming in C/C++
// Problem 1:
// A function that counts distinct elements of a given integer array
//
#include <iostream.h>

int countDistinct (int* array, int dim) {
 for (int count=0, i=0; i<dim; i++) {
 for (int found=0, j=0; (j<i) && (found==0); j++)
 if (array[i]==array[j]) found=1;
 if (found==0) count++;
 }
 return count;
}

void main () {
 int a1[5], a2[5], a3[5];
 a1[0]=0;
 a2[0]=1; a2[1]=2; a2[2]=1; a2[3]=3; a2[4]=2;
 a3[0]=1; a3[1]=1; a3[2]=1; a3[3]=1; a3[4]=1;

 cout<<"\n\n";
 cout<<"Number of distinct in a1: "<<countDistinct(a1,5)<<"\n";
 cout<<"Number of distinct in a2: "<<countDistinct(a2,5)<<"\n";
 cout<<"Number of distinct in a3: "<<countDistinct(a3,5)<<"\n";
}
```

## Zadatak

Napisati funkciju koja iz datog celobrojnog niza izbacuje elemente koji su jednaki zadatoj vrednosti.

### Rešenje

```
// Theme: Basic Structural programming in C/C++
// Problem 2:
// A function that deletes elements from a given integer array
// equal to the given value
//
#include <iostream.h>

void deleteEqual (int* array, int* dim, int value) {
 for (int i=0; i<*dim; i++)
 if (array[i]!=value) array[i++] = array[i];
 *dim=j;
}

void main () {
 int a1[1], a2[5], a3[5];
 int dim1=1, dim2=5, dim3=5;
 a1[0]=0;
 a2[0]=1; a2[1]=2; a2[2]=1; a2[3]=3; a2[4]=2;
 a3[0]=1; a3[1]=1; a3[2]=1; a3[3]=1; a3[4]=1;

 cout<<"\n\n";
 cout<<"Array a1:\n";
 for (int i=0; i<dim1; i++) cout<<a1[i]<<" ";
 cout<<"\n";
 deleteEqual(a1, &dim1, 3);
 cout<<"Array a1 after deletion of 3:\n";
 for (i=0; i<dim1; i++) cout<<a1[i]<<" ";

 cout<<"\n\n";
 cout<<"Array a2:\n";
 for (i=0; i<dim2; i++) cout<<a2[i]<<" ";
 cout<<"\n";
 deleteEqual(a2, &dim2, 2);
 cout<<"Array a2 after deletion of 2:\n";
 for (i=0; i<dim2; i++) cout<<a2[i]<<" ";

 cout<<"\n\n";
 cout<<"Array a3:\n";
 for (i=0; i<dim3; i++) cout<<a3[i]<<" ";
 cout<<"\n";
 deleteEqual(a3, &dim3, 1);
 cout<<"Array a3 after deletion of 1:\n";
 for (i=0; i<dim3; i++) cout<<a3[i]<<" ";
}
```

## Glava 3

# Elementi jezika C++ koji nisu objektno orijentisani

## Zadatak

Napisati funkciju koja sabira dva pozitivna cela broja u „neograničenoj tačnosti“. Brojevi su predstavljani kao nizovi decimalnih cifara.

### Rešenje

```
// Theme: Basic Structural programming in C/C++
// Problem 3:
// A function that calculates the sum of two positive integer numbers
// with infinite precision. The numbers are given as integer arrays of
// decimal digits.
//
#include <iostream.h>

void add (const int* op1, int len1, const int* op2, int len2,
 int* sum, int& lensum) {
 lensum=0;
 for (int i=0, carry=0; (i<len1) && (i<len2); i++) {
 int temp=op1[i]+op2[i]+carry;
 sum[lensum++]=temp%10;
 carry=temp/10;
 }
 for (; i<len1; i++) {
 int temp=op1[i]+carry;
 sum[lensum++]=temp%10;
 carry=temp/10;
 }
 for (; i<len2; i++) {
 int temp=op2[i]+carry;
 sum[lensum++]=temp%10;
 carry=temp/10;
 }
 if (carry>0) sum[lensum++]=carry;
}

void main () {
 int a1[1], a2[5], a3[6];
 int dim1=1, dim2=5, dim3=6;
 a1[0]=1;
 a2[0]=9; a2[1]=9; a2[2]=9; a2[3]=9; a2[4]=9;
}
```

```

cout<<"\n\n";
add(a1,dim1,a2,dim2,a3,dim3);
for (int i=dim1-1; i>=0; i--) cout<<a1[i]; cout<<" ";
for (i=dim2-1; i>=0; i--) cout<<a2[i]; cout<<" ";
for (i=dim3-1; i>=0; i--) cout<<a3[i];

cout<<"\n\n";
add(a2,dim2,a1,dim1,a3,dim3);
for (i=dim2-1; i>=0; i--) cout<<a2[i]; cout<<" ";
for (i=dim1-1; i>=0; i--) cout<<a1[i]; cout<<" ";
for (i=dim3-1; i>=0; i--) cout<<a3[i];

cout<<"\n\n";
add(a2,dim2,a2,dim2,a3,dim3);
for (i=dim2-1; i>=0; i--) cout<<a2[i]; cout<<" ";
for (i=dim2-1; i>=0; i--) cout<<a2[i]; cout<<" ";
for (i=dim3-1; i>=0; i--) cout<<a3[i];
}

```

## Zadatak

Napisati funkciju koja u datom nizu znakova pretvara prva slova reči u velika, a ostala u mala. Reči su rastavljene blanko znacima.

### Rešenje

```

// Theme: Basic Structural programming in C/C++
//
// Problem 4:
// A function that capitalizes words (first letters into capitals,
// others into small). Words are separated by blanks only.
//

```

```

#include <iostream.h>
#include <string.h>

```

```

char* capitalize (const char* text) {
 char* output=new char(strlen(text)+1);
 int first=1;

```

```

 for (const char* p=text; *p!='\0'; p++)
 if (*p==' ') { // separation of words
 *q++=*p; first=1;
 }
 else { // inside a word
 if (first) // capitalize
 if ((*p>='a') && (*p<='z')) *q++=*p-'a'+'A';
 else *q++=*p;
 // turn into small ones
 if ((*p>='A') && (*p<='Z')) *q++=*p-'A'+'a';
 else *q++=*p;
 first=0;
 }
 }

```

```

*q='\0';
return output;
}

void main () {
 char* text1="Please, capitalize All these words properly! ";
 char* text2=capitalize(text1);

 cout<<"\n\n";
 <<"Input text: "<<text1<<"\n";
 <<"Output text: "<<text2;

 delete [] text2;
}

```

## Zadatak

Napisati funkciju koja unutar datog niza znakova zamenjuje sve pojave zadatog podniza znakova drugim podnizom.

### Rešenje

```

// Theme: Basic Structural programming in C/C++
//
// Problem 5:
// A function that replaces all occurrences of a substring inside a string
//

```

```

#include <iostream.h>
#include <string.h>

```

```

const int MAXLEN = 1000; // maximal length of the result string

```

```

char* replace (const char *text, const char* what, const char* by) {
 char* output=new char[MAXLEN];
 char* q=output;

```

```

 int i=0, // position for comparison
 len=strlen(what);

 for (const char* p=text; *p!='\0';)
 if (p[i]!=what[i]) { // cancel matching
 if (q-output<MAXLEN) *q++=*p++;
 i=0;
 }
 else if (++i==len) { // matched: replace
 for (const char *r=by; *r!='\0'; r++)
 if (q-output<MAXLEN) *q++=*r;
 p+=len;
 i=0;
 }
 if (q-output<MAXLEN) *q++='\0';
 else output[MAXLEN-1]='\0';
 return output;
}

```

```

void main () {
 char* text1="Please, replace all aab with xx: aabababaaabaaxaab.
 char* text2=replace(text1,"aab","xx");

 cout<<"\n\n"
 <<"Input text: "<<text1<<"\n"
 <<"Output text: "<<text2;

 delete [] text2;
}

```

## Zadatak

Napisati skup funkcija koje operišu jednostruko ulančanim dinamičkim listama.

### Rešenje

```

// Theme: Basic Structural programming in C/C++
// Problem 6:
// A set of functions that operate upon a linked list
//
#include <iostream.h>

typedef int T; // type of a list element

struct ListElem {
 T cont; // contents
 ListElem* next; // link to the next
};

struct List {
 ListElem* head;
};

// Initialize list
void initList (List* lst) {
 if (lst==0) return;
 lst->head=0;
}

// Delete list
void delList (List* lst) {
 if (lst==0 || lst->head==0) return;
 for (ListElem* temp=lst->head->next;
 lst->head!=0;
 lst->head=temp, temp=temp->next;0)
 delete lst->head;

// Insert at front
void insert (List* lst, T t) {
 if (lst==0) return;
 ListElem* e = new ListElem;
 if (e==0) return;
 e->cont=t;
 e->next=lst->head;
 lst->head=e;
}

```

```

// Delete element at position 'at'
void delElem (List* lst, int at) {
 if (lst==0) return;
 int i=0;
 ListElem *temp=lst->head, *prev=0;
 for (; (temp!=0) && (i<at);
 prev=temp, temp=temp->next, i++);
 if (temp!=0) {
 if (prev!=0) prev->next=temp->next;
 else lst->head=temp->next;
 delete temp;
 }
}

```

```

// Iterator:
void forEach (List* lst, void (*f)(T)) {
 if (lst==0) return;
 for (ListElem* e=lst->head; e!=0; e=e->next)
 f(e->cont);
}

```

```

// Test function:
void print (int i) {
 cout<<i<<" ";
}

```

```

void main () {
 List lst;
 initList(&lst);

 insert(&lst,3); insert(&lst,2); insert(&lst,1);

 cout<<"\n\n";
 cout<<"List: "; foreach(&lst,&print); cout<<"\n";

 insert(&lst,0);
 cout<<"Insert 0: "; foreach(&lst,&print); cout<<"\n";

 delElem(&lst,2);
 cout<<"Delete 2: "; foreach(&lst,&print); cout<<"\n";

 delElem(&lst,0);
 cout<<"Delete 0: "; foreach(&lst,&print); cout<<"\n";

 delList(&lst);
}

```

## Zadaci za samostalan rad

3. Realizovati funkciju `strlen` koja prihvata pokazivač na niz znakova kao argument i kopira niz znakova na koji ukazuje taj argument u niz znakova alocirani u dinamičkoj memoriji, na koga će ukazivati pokazivač vraćen kao rezultat funkcije.

## Klase i preklapanje operatora

### Zadatak

Realizovati apstraktni tip steka čiji su elementi pokazivači na neki tip. Stek treba da bude implementiran pomoću dinamički alociranog niza, tako da se dinamički proširuje kada se napuni, ali ne mora i da se skuplja kada se isprazni. Pored toga, svi stekovi u sistemu treba da se prijavljuju zajedničkom „kontroloru“.

### Rešenje

```
// Primer 3: Realizacija dinamičkog steka
// File: stack.h

// Klasa Stack:
// Stek elemenata tipa (pokazivaca na) T.
// Stek se sam proširuje kada se napuni, ali se ne skuplja kada se
// isprazni.
// Svi stekovi se prijavljuju zajedničkom "kontroloru".

#ifndef _Stack
#define _Stack

typedef int T; // tip elemenata steka
const int incsize = 10; // korak uvećanja steka

class Stack {
public:
 Stack (int initialSize=incsize);
 Stack (const Stack&);
 ~Stack ();

 Stack& operator= (const Stack&);
 Stack& operator++ ();
 Stack& operator++ (int);
 Stack& operator+= (int s);
 friend ostream& operator<< (ostream&, const Stack&);

 void push (const T&);
 T& pop ();
 T& peek ();
 const T& () const { return *slots[nfull-1]; }

protected:
 void resize(int step=incsize);
 void release ();
```



```

private:
friend class SkIterator;

static Stack *head;
void link();
void unlink();

int size;
int nfull;
const T **slots;
};

// Klasa SkIterator:
class SkIterator {
public:
 SkIterator () : nextstk(Stack::head) {}
 ~SkIterator () {}
 Stack* operator () ();
 Stack* operator* () const { return *nextstk;}

 void print (ostream& os) const;
 friend ostream& operator<<(ostream& os, const SkIterator& it);

private:
 Stack *nextstk;
};

#endif

// Primer 3: Realizacija dinamičkog steka
// File: stack.cpp
#include <iostream.h>
#include "stack.h"
typedef const T *PT;

// Klasa Stack:
Stack *Stack::head = 0;

void Stack::resize (int by) {
 PT *newslots = new PT[size+by];
 if (slots) {
 for (int i=0; i<nfull; i++)
 newslots[i] = slots[i];
 delete [] slots;
 }
 slots = newslots;
}

void Stack::release () {
 delete [] slots;
 size = 0;
}

```

```

void Stack::link () {
 next = head;
 head = this;
}

void Stack::unlink () {
 if (head == this) {
 head = next;
 next = 0;
 return;
 }
 Stack *p = head;
 for (; (p->next != this) ; p = p->next);
 p->next = next;
 next = 0;
 return;
}

Stack Stack::operator= (const Stack &right) {
 if (&right == this) return *this;
 release();
 resize(right.size);
 for (int i=0; i<right.nfull; i++)
 slots[i] = right.slots[i];
 nfull = right.nfull;
}

Stack::Stack (int sz) : nfull(0), size(0), slots(0) {
 link();
 resize(sz);
}

Stack::~Stack () {
 release();
 unlink();
}

void Stack::push (const T &t) {
 if (nfull == size) resize(incsize);
 slots[nfull++] = &t;
}

T& Stack::pop () {
 return (T&) *slots[--nfull];
}

// Funkcija za ispis steka:
ostream& operator<<(ostream &os, const Stack &s) {
 os<<" ";
 for (int i=0; i<s.nfull; i++)
 os<<"s.slots["<<i<<" ";
 return os<<" ";
}

```

```
// Klasa StkIterator:
Stack* StkIterator::operator() () {
 Stack *temp = nextstk;
 if (nextstk) nextstk=nextstk->next;
 return temp;
}

void StkIterator::print (ostream &os) const {
 for (Stack *p=Stack::head; p; p=p->next) os<<"p<<"<<" ";
}

// Funkcija za ispis svih stekova:
ostream& operator<< (ostream &os, const StkIterator &sc) {
 sc.print(os);
 return os;
}
```

## Zadatak

Realizovati apstraktni tip kompleksnog broja.

### Rešenje

```
// Primer 4: Klasa kompleksnih brojeva
// File: complex.h
#ifndef _Complex
#define _Complex

class Complex {
public:
 Complex (double real=0, double imag=0);

 friend Complex operator+ (const Complex&, const Complex&);
 friend Complex operator- (const Complex&, const Complex&);
 friend Complex operator* (const Complex&, const Complex&);
 friend Complex operator/ (const Complex&, const Complex&);

 Complex& operator+= (const double);
 Complex& operator-= (const double);
 Complex& operator*= (const double);
 Complex& operator/= (const double);
 Complex& operator++= (const Complex&);
 Complex& operator--= (const Complex&);
 Complex& operator*= (const Complex&);
 Complex& operator/= (const Complex&);

 Complex operator+ () const;
 Complex operator- () const;

 friend int operator== (const Complex&, const Complex&);
 friend int operator!= (const Complex&, const Complex&);

 friend double re (const Complex&);
 friend double im (const Complex&);
 friend Complex conj (const Complex&);

 friend ostream& operator<< (ostream&, const Complex&);
 friend istream& operator>> (istream&, Complex&);
};
```

```
private:
 double real, imag;
};

// Inline functions:
inline Complex::Complex (double r, double i) : real(r), imag(i) {}
inline Complex operator+ (const Complex& c1, const Complex& c2)
{ return Complex(c1.real+c2.real, c1.imag+c2.imag); }
inline Complex operator- (const Complex& c1, const Complex& c2)
{ return Complex(c1.real-c2.real, c1.imag-c2.imag); }

inline Complex& Complex::operator+= (const double d)
{ real+=d; return *this; }
inline Complex& Complex::operator-= (const double d)
{ real-=d; return *this; }
inline Complex& Complex::operator*= (const double d)
{ real*=d; imag*=imag; return *this; }
inline Complex& Complex::operator/= (const double d)
{ real/=d; imag/=imag; return *this; }

inline Complex& Complex::operator+= (const Complex &c)
{ real+=c.real; imag+=c.imag; return *this; }
inline Complex& Complex::operator-= (const Complex &c)
{ real-=c.real; imag-=c.imag; return *this; }
inline Complex& Complex::operator*= (const Complex &c)
{ return *this*=this*c; }
inline Complex& Complex::operator/= (const Complex &c)
{ return *this/=this/c; }

inline Complex Complex::operator+ () const { return *this; }
inline Complex Complex::operator- () const { return Complex(-real, -imag); }

inline int operator== (const Complex& c1, const Complex& c2)
{ return (c1.real==c2.real) && (c1.imag==c2.imag); }
inline int operator!= (const Complex& c1, const Complex& c2)
{ return !(c1==c2); }

inline double re (const Complex& c) { return c.real; }
inline double im (const Complex& c) { return c.imag; }
inline Complex conj (const Complex& c) { return Complex(-c.real, c.imag); }

#endif

// Primer 4: Klasa kompleksnih brojeva
// File: complex.cpp
#include <iostream.h>
#include "complex.h"
```

```

Complex operator* (const Complex& c1, const Complex& c2)
{ return Complex(c1.real*c2.real-c1.imag*c2.imag,
 c1.real*c2.imag+c1.imag*c2.real); }

Complex operator/ (const Complex& c1, const Complex& c2)
{ double r=c2.real*c2.real+c2.imag*c2.imag;
 return Complex((c1.real*c2.real+c1.imag*c2.imag)/r,
 (c2.real*c1.imag-c2.imag*c1.real)/r); }

ostream& operator<< (ostream& fos, const Complex& kc)
{ return os<<"<<c.real<<"<<c.imag<<" "; }

istream& operator>> (istream& fis, Complex& kc)
{ return is>>c.real>>c.imag; }

```

## Zadatok

Realizovati apstraktni tip koji predstavlja niz znakova (engl. *string*). Ovaj tip treba da bude implementiran tako da se niz znakova dinamički alokira u memoriji. Posebno svojstvo implementacije je da se objekti-stringovi koji imaju isti sadržaj (prilikom kopiranja objekata) vezuju za isti fizički niz znakova u dinamičkoj memoriji, a da se ti nizovi ne kopiraju sve dok su isti.

### Rešenje

Potrebno je koristiti sistem brojanja referenci na implementacioni dinamički niz znakova.

```

// Primer 5: Klasa DString koja broji korišćenje zajedničkog teksta
// File: dstring.h
#include <iostream.h>
#include <string.h>

class DString {
public:
 DString ();
 DString (const char*);
 DString (const DString&);
 ~DString ();

 DString& operator= (const DString&);
 DString& operator= (const char*);

 friend int operator==(const DString&, const char*);
 friend int operator!=(const DString&, const DString&);
 friend int operator==(const DString&, const char*);
 friend int operator!=(const DString&, const DString&);

 char& operator[] (const int);

 int count () const;
 int len () const;

 friend istream& operator>> (istream&, DString&);
 friend ostream& operator<< (ostream&, const DString&);

protected:
 void release();

private:
 struct Text {
 char *store;
 int usage;
 };
 Text *txt;
};

```

```

// Inline functions:
inline DString::DString (const DString& ks)
{ txt=s.txt; txt->usage++; }

inline DString::~DString () { release(); }

inline int DString::count () const { return txt->usage; }

inline int DString::len () const { return strlen(txt->store); }

inline int operator==(const DString& ks, const char *c)
{ return strcmp(s.txt->store,c)==0; }

inline int operator==(const DString& ks1, const DString& ks2)
{ return strcmp(s1.txt->store,s2.txt->store)==0; }

inline int operator!=(const DString& ks, const char *c)
{ return strcmp(s.txt->store,c)!=0; }

inline int operator!=(const DString& ks1, const DString& ks2)
{ return strcmp(s1.txt->store,s2.txt->store)!=0; }

inline char& DString::operator[] (const int i) { return txt->store[i]; }

// Primer 5: Klasa DString koja broji korišćenje zajedničkog teksta
// File: dstring.cpp
#include "dstring.h"

void DString::release () {
 if (--txt->usage==0) {
 if (txt->store) delete [] txt->store;
 delete txt;
 }
}

DString::DString () {
 txt=new Text;
 txt->store=0;
 txt->usage=1;
}

DString::DString (const char *c) {
 txt=new Text;
 txt->usage=1;
 txt->store=new char [strlen(c)+1];
 strcpy(txt->store,c);
}

DString& DString::operator= (const char *c) {
 if (txt->usage>1) {
 txt->usage--;
 txt=new Text;
 }
 else if (txt->usage==1) delete [] txt->store;
 txt->store=new char [strlen(c)+1];
 strcpy(txt->store,c);
 txt->usage=1;
 return *this;
}

```

```

DString& DString::operator= (const DString &s) {
 if (this!=&s) {
 release();
 txt=s.txt;
 txt->usage++;
 }
 return *this;
}

ostream& operator<< (ostream &os, const DString &s) {
 { return os<<s.txt->store<< "["<<s.txt->usage<<"]"; }

istream& operator>> (istream &is, DString &s) {
 static const int MAXLEN = 256;
 static char buff[MAXLEN];
 is.get(buff, MAXLEN);
 s=buff;
 return is;
}

```

## Zadatak

Projektovati apstraktni tip podataka za predstavljanje vremena tokom dana. U glavnom programu demonstrirati upotrebu ovog tipa.

## Rešenje

```

class Time {
public:
 Time (int hour=0, int min=0, int sec=0);

 friend int operator== (const Time&, const Time&);
 friend int operator!= (const Time&, const Time&);
 friend int operator< (const Time&, const Time&);
 friend int operator> (const Time&, const Time&);
 friend int operator<= (const Time&, const Time&);
 friend int operator>= (const Time&, const Time&);

 int getHour () const { return h; }
 int getMin () const { return m; }
 int getSec () const { return s; }

 void setTime (int hour, int min, int sec);

 void incHour ();
 void incMin ();
 void incSec ();
 void decHour ();
 void decMin ();
 void decSec ();

 void addTime (const Time& interval);
 void addTime (int hour, int min, int sec);
 void subTime (const Time& interval);
 void subTime (int hour, int min, int sec);

protected:
 static void checkTime (int& hour, int& min, int& sec);

private:
 int h,m,s;
};

```

```

void Time::checkTime (int& hour, int& min, int& sec) {
 if (sec>59) {
 min+=(sec/60);
 sec%=60;
 }
 if (min>59) {
 hour+=(min/60);
 min%=60;
 }
 if (hour>23) hour%=24;
}

Time::Time (int hour, int min, int sec) {
 setTime(hour,min,sec);
}

void Time::setTime (int hour, int min, int sec) {
 checkTime(hour,min,sec);
 h=hour; m=min; s=sec;
}

int operator== (const Time& t1, const Time& t2) {
 return (t1.h==t2.h)&&(t1.m==t2.m)&&(t1.s==t2.s);
}

int operator!= (const Time& t1, const Time& t2) {
 return !(t1==t2);
}

int operator< (const Time& t1, const Time& t2) {
 if (t1.h<t2.h) return 1;
 if (t1.h>t2.h) return 0;
 if (t1.m<t2.m) return 1;
 if (t1.m>t2.m) return 0;
 if (t1.s<t2.s) return 1;
 return 0;
}

int operator> (const Time& t1, const Time& t2) {
 return (t2<t1);
}

int operator<= (const Time& t1, const Time& t2) {
 return (t1<t2)|| (t1==t2);
}

int operator>= (const Time& t1, const Time& t2) {
 return (t1>t2)|| (t1==t2);
}

void Time::incHour () {
 h++;
 if (h>23) h=0;
}

void Time::incMin () {
 m++;
 if (m>59) {
 m=0;
 incHour();
 }
}

```

```

void Time::incSec () {
 s++;
 if (s>59) {
 s=0;
 incMin();
 }
}

void Time::dechour () {
 h--;
 if (h<0) h=23;
}

void Time::decMin () {
 m--;
 if (m<0) {
 m=59;
 dechour();
 }
}

void Time::decSec () {
 s--;
 if (s<0) {
 s=59;
 decMin();
 }
}

void Time::addTime (const Time& t) {
 addTime(t.h,t.m,t.s);
}

void Time::addTime (int hour, int min, int sec) {
 h+=hour; m+=min; s+=sec;
 checkTime(h,m,s);
}

void Time::subTime (const Time& t) {
 subTime(t.h,t.m,t.s);
}

void Time::subTime (int hour, int min, int sec) {
 checkTime(hour,min,sec);
 if (s<sec) {
 decMin();
 s+=60;
 }
 s-=sec;
 if (m<min) {
 dechour();
 m+=60;
 }
 m-=min;
 if (h<hour) {
 h+=24;
 h-=hour;
 }
}

```

```

#include <iostream.h>

void main () {
 Time t1(23,45,59), t2(4,27,56);
 cout<<(t1<t2)<<"\n";
 t1.addTime(t2);
 cout<<t1.getHour()<<"<<t1.getMin()<<"<<t1.getSec()<<"\n";
 t1.subTime(t2);
 cout<<t1.getHour()<<"<<t1.getMin()<<"<<t1.getSec()<<"\n";
}

```

## Zadatak

Projektovati apstraktni tip podataka za predstavljanje datuma.

### Rešenje

```

class Date {
public:
 Date (int year=1980, int month=1, int day=1);

 friend int operator== (const Date&, const Date&);
 friend int operator!= (const Date&, const Date&);
 friend int operator< (const Date&, const Date&);
 friend int operator> (const Date&, const Date&);
 friend int operator<= (const Date&, const Date&);
 friend int operator>= (const Date&, const Date&);

 static int isLeapYear (int year);
 static int getDaysInMonth (int year, int month);

 int getYear () const { return y; }
 int getMonth() const { return m; }
 int getDay () const { return d; }
 int getWeekDay () const; // starting from Sunday

 void setDate(int year, int month, int day);

 void incYear ();
 void incMonth ();
 void incDay ();
 void decYear ();
 void decMonth ();
 void decDay ();

 void addDays (int numofDays);
 void subDays (int numofDays);

protected:
 static void checkDate (int& year, int& month, int& day);
 int getNumofDays () const; // returns number of days from 1-1-1980
 void countWeekDay ();

private:
 int y,m,d,wd;
 static int daysInMonth[13];
};

```

```

int Date::daysInMonth[] = {0,31,28,31,30,31,30,31,31,30,31,30,31};

int Date::isLeapYear (int year) {
 if (year%4 != 0) return 0;
 if (year%400 == 0) return 1;
 if (year%100 == 0) return 0;
 return 1;
}

int Date::getDaysInMonth (int year, int month) {
 if (month!=2) return daysInMonth[month];
 if (isLeapYear(year)) return 29;
 return 28;
}

int Date::getNumOfDays () const {
 Date d0(1980,1,1);
 if (d0 == *this) return 0;

 int flag=(d0<*this);
 Date d1=flag ? d0 : (*this);
 Date d2=flag ? (*this) : d0;
 for (int days=0; d1<d2; d1.incDay())
 days++;

 if (flag) return days;
 else return -days;
}

void Date::countWeekDay () {
 int days=getNumOfDays();
 if (days>=0) {
 days%=7;
 days=(days+3)%7;
 } else {
 days=-days;
 days%=7;
 days=(10-days)%7;
 }
 if (days==0) days=7;
 wd=days;
}

void Date::checkDate (int& year, int& month, int& day) {
 if (day>getDaysInMonth(year,month)) day=1;
 if (month>12) month=1;
}

Date::Date (int year, int month, int day) {
 setDate(year,month,day);
}

void Date::setDate (int year, int month, int day) {
 checkDate(year,month,day);
 y=year; m=month; d=day;
 wd=0;
}

int Date::getWeekDay () const {
 if (wd==0)
 ((Date* const)this)->countWeekDay();
 return wd;
}

```

```

int operator== (const Date& d1, const Date& d2) {
 return (d1.y==d2.y)&&(d1.m==d2.m)&&(d1.d==d2.d);
}

int operator!= (const Date& d1, const Date& d2) {
 return !(d1==d2);
}

int operator< (const Date& d1, const Date& d2) {
 if (d1.y<d2.y) return 1;
 if (d1.y>d2.y) return 0;
 if (d1.m<d2.m) return 1;
 if (d1.m>d2.m) return 0;
 if (d1.d<d2.d) return 1;
 return 0;
}

int operator> (const Date& d1, const Date& d2) {
 return (d2<d1);
}

int operator<= (const Date& d1, const Date& d2) {
 return (d1<d2)|| (d1==d2);
}

int operator>= (const Date& d1, const Date& d2) {
 return (d1>d2)|| (d1==d2);
}

void Date::incYear () {
 y++;
}

void Date::incMonth () {
 m++;
 if (m>12) {
 m=1;
 incYear();
 }
}

void Date::incDay () {
 d++;
 if (d>getDaysInMonth(y,m)) {
 d=1;
 incMonth();
 }
}

void Date::decYear () {
 y--;
}

void Date::decMonth () {
 m--;
 if (m<1) {
 decYear();
 m=12;
 }
}

void Date::decDay () {
 d--;
 if (d<1) {
 decMonth();
 d=getDaysInMonth(y,m);
 }
}

```

```
void Date::addDays (int days) {
 for (int i=0; i<days; i++) incDay();
}

void Date::subDays (int days) {
 for (int i=0; i<days; i++) decDay();
}
```

## Zadatok

Realizovati apstrakciju časovnika koji sadrži tekuci datum i vreme. Časovnik se može očitavati i navijati. Časovnik treba da prati tekuće vreme tako što će mu neki spoljni mehanizam generisati otkucanje sekundi. U sistemu treba da postoji samo jedna instanca časovnika. Treba onemogućiti pravljenje više od jedne instance časovnika i obezbediti da svi delovi sistema mogu pristupati časovniku na tačno definisan način (odgovarajućim imenovanjem). U glavnom programu prikazati upotrebu ove apstrakcije.

### Rešenje

Zahtevi navedeni u zadatku:

1. mora postojati tačno jedna instanca date klase i mora se obezbediti zaštita od pravljenja više instanci;
  2. svi ostali delovi programa treba da pristupaju toj instanci unapred definisanim imenovanjem, tako da se ne mora pamtiti posebno ime date instance;
- ispunjavaju se projektnim obrascem (engl. *design pattern*) *Singleton*.

```
class Clock {
public:
 static Clock* Instance (); // Singleton design pattern

 Date getDate () const { return d; }
 Time getTime () const { return t; }

 void wind (const Date&, const Time&);
 void tick ();

protected:
 Clock () {} // Protected due to the Singleton design pattern

private:
 Date d;
 Time t;
};

Clock* Clock::Instance () {
 static Clock instance;
 return &instance;
}

void Clock::wind (const Date& dt, const Time& tm) {
 d=dt; t=tm;
}
```

```
void Clock::tick () {
 t.incSec();
 if (t==Time(0,0,0)) d.incDay();
}
```

```
#include <iostream.h>
```

```
void print (const Time& t) {
 cout<<t.getHour()<<": "<<t.getMin()<<": "<<t.getSec()<<"\n";
}
```

```
void main () {
 Clock::Instance()->wind(Date(1998,3,22),Time(20,14,00));
```

```
 for (int i=0; i<10000; i++) {
 Clock::Instance()->tick();
 Time t=Clock::Instance()->getTime();
 print(t);
 }
}
```

## Zadaci za samostalan rad

4. Realizovati klasu čiji će objekti služiti za izdvajanje reči iz teksta koji je dat kao niz znakova. Jednom rečju smatra se niz znakova bez blanko znaka. Klasa treba da sadrži pokazivač na niz znakova koji predstavlja ulazni tekst, i koji će biti inicijalizovan u konstruktoru. Klasa treba da sadrži i funkciju koja, pri svakom pozivu, vraća pokazivač na dinamički niz znakova u kojem je izdvojena naredna reč teksta. Kada naiđe na kraj teksta, ova funkcija treba da vrati nula-pokazivač. U glavnom programu isprobati upotrebu ove klase, na nekoliko objekata koji deluju na isti globalni niz znakova.
5. Realizovati klasu LongInt koja predstavlja cele brojeve neograničene tačnosti (protizvoljan broj decimalnih cifara). Obezbediti operacije sabiranja i oduzimanja, kao i sve ostale očekivane operacije, uključujući i operacije za ulaz/izlaz. U glavnom programu napraviti instance ovog tipa i izvršiti operacije nad njima. Brojeve interno predstavljati kao nizove znakova-cifara.
6. Realizovati klasu View koja će predstavljati apstrakciju svih vrsta entiteta koji se mogu pojaviti na ekranu monitora u nekom korisničkom interfejsu (prozor, meni, dijalog, itd.). Sve klase koje će realizovati pojedine entitete korisničkog interfejsa biće izvedene iz ove klase. Ova klasa treba da ima apstraktnu funkciju draw, koja će biti zadužena za iscrtavanje entiteta pri osvežavanju (ažuriranju) izgleda ekrana (kada se nešto na ekranu promeni), i koju ne treba realizovati. Svaki objekat ove klase će se, pri nastanku, „prijavljivati“ u zajedničku listu svih objekata na ekranu. Klasa View treba da sadrži statičku funkciju članicu refresh koja će prolaziti kroz tu listu i pozivati funkciju draw svakog objekta, kako bi se izgled ekrana osvežio. Redosled objekata u listi predstavlja redosled iscrtavanja, čime se dobija efekat preklapanja na ekranu. Zbog toga klasa View treba da ima funkciju članicu setFocus, koja će dati objekat postaviti na kraj liste (tako se daje fokus tom entitetu).

## Nasleđivanje

### Zadatak

Projektovati apstrakcije i mehanizme koji omogućuju da se tekuće vreme časovnika očitava i zadaje u tekstualnom formatu prilagođenom specifičnostima datog podneblja. Treba obezbediti da se vreme i datum mogu pisati u različitim formatima, npr. „22:14:00“ ili „10:14:00 pm“, i „Nedelja, 22.3.1998.“ ili „Sunday, 3/22/1998“.

### Rešenje

Prepostavlja se da se nizovi znakova predstavljaju tipom String. Interfejs klase Clock dopunjuje se tako da izgleda ovako:

```
class Clock {
public:
 static Clock* Instance ();

 Date getDate () const { return d; }
 Time getTime () const { return t; }

 String getDateTime () const;
 String getDate () const;
 String getTime () const;

 void wind (const Date&, const Time&);

 void wind (const String& date&time);
 void setDate (const String& date);
 void setTime (const String& time);
};
```

Odgovornost za specifičnosti formata datuma i vremena i njegove transformacije treba enkapsulirati u objekte kojima se pristupa preko sledeće apstraktno osnovne klase:

```
enum IntlCode { US, 'UTC', ... }; // and other national codes

class International {
public:
 static International* Current ();
 static void setCurrent (IntlCode);

 virtual int checkDateTime (const String& date&time) const = 0;
 virtual int checkDate (const String&) const = 0;
 virtual int checkTime (const String&) const = 0;

 String getDateDateFormat () const;
 virtual String getDateFormat () const = 0;
 virtual String getTimeFormat () const = 0;
```



```

virtual String getDate(const Date& d) const { return d.getDate(); }
virtual Date getDate(const String& s) const { return Date(s); }
virtual String getTime(const Time& t) const { return t.getTime(); }
virtual Time getTime(const String& s) const { return Time(s); }

String getDate(const Date& d, const Time& t) const {
 return getDate(d) + " " + getTime(t); }
int getDate(const String& s, const Date& d, const Time& t) const {
 return getDate(s, d, t).getDate(); }
virtual String extractDate(const String& s, const Date& d, const Time& t) const {
 return extractDate(s, d, t).getDate(); }
virtual String extractTime(const String& s, const Date& d, const Time& t) const {
 return extractTime(s, d, t).getTime(); }
virtual String weekday(const String& s, const Date& d, const Time& t) const {
 return weekday(s, d, t).weekday(); }
virtual int weekday(const String& s, const Date& d, const Time& t) const {
 return weekday(s, d, t).weekday(); }

protected:
 International () {}

private:
 static International* current;
};

```

Konkretno izvedene klase realiziraju specifičnosti podneblja preko virtuelnih funkcija koje su u osnovnoj klasi apstraktne. Funkcije koje nisu označene kao virtuelne, zapravo se svedu na tačno definisan redosled poziva ostalih apstraktnih funkcija koje su ostavljene kao „kukice“ (engl. *hook methods*). Na ovaj način se u osnovnoj klasi strogo definiše redosled koraka izvršenja neke operacije, dok izvedena klasa ima mogućnost da definiše specifičnosti pojedinih koraka. Ovakav projektni obrazac naziva se *Template Method*. U ovom primeru, nevirtuelne funkcije realizuju se na sledeći način:

```

String International::getDate(const String& s, const Date& d, const Time& t) const {
 return getDate(s, d, t).getDate(); }
String International::extractDate(const String& s, const Date& d, const Time& t) const {
 return extractDate(s, d, t).extractDate(); }
String International::extractTime(const String& s, const Date& d, const Time& t) const {
 return extractTime(s, d, t).extractTime(); }
String International::weekday(const String& s, const Date& d, const Time& t) const {
 return weekday(s, d, t).weekday(); }
int International::getDate(const String& s, const Date& d, const Time& t) const {
 return getDate(s, d, t).getDate(); }
int International::extractDate(const String& s, const Date& d, const Time& t) const {
 return extractDate(s, d, t).extractDate(); }
int International::extractTime(const String& s, const Date& d, const Time& t) const {
 return extractTime(s, d, t).extractTime(); }
int International::weekday(const String& s, const Date& d, const Time& t) const {
 return weekday(s, d, t).weekday(); }

```

Realizacije apstraktnih funkcija zavise od nacionalnog područja. Na primer, za naše nacionalno područje, realizacija je sledeća:

```

class NationalYU : public International {
public:
 virtual int checkDate(const Date& d) const { return d.checkDate(); }
 virtual int checkTime(const Time& t) const { return t.checkTime(); }
 virtual int checkDate(const String& s, const Date& d, const Time& t) const {
 return checkDate(s, d, t).checkDate(); }
 virtual int checkTime(const String& s, const Date& d, const Time& t) const {
 return checkTime(s, d, t).checkTime(); }
 virtual String getDate(const Date& d, const Time& t) const {
 return getDate(d, t).getDate(); }
 virtual String extractDate(const String& s, const Date& d, const Time& t) const {
 return extractDate(s, d, t).extractDate(); }
 virtual String extractTime(const String& s, const Date& d, const Time& t) const {
 return extractTime(s, d, t).extractTime(); }
 virtual String weekday(const String& s, const Date& d, const Time& t) const {
 return weekday(s, d, t).weekday(); }
 virtual int weekday(const String& s, const Date& d, const Time& t) const {
 return weekday(s, d, t).weekday(); }

private:
 static String daysOfWeek[8];
};

String NationalYU::daysOfWeek[] = {
 "",
 "Nedelja",
 "Ponedeljak",
 "Utorak",
 "Sreda",
 "Cetvrtak",
 "Petak",
 "Subota"
};

int NationalYU::checkDate(const String& s, const Date& d) const {
 return checkDate(s, d).checkDate(); }
int NationalYU::checkTime(const String& s, const Time& t) const {
 return checkTime(s, t).checkTime(); }
int NationalYU::checkDate(const String& s, const Date& d, const Time& t) const {
 return checkDate(s, d, t).checkDate(); }
int NationalYU::checkTime(const String& s, const Date& d, const Time& t) const {
 return checkTime(s, d, t).checkTime(); }
String NationalYU::getDate(const Date& d, const Time& t) const {
 return getDate(d, t).getDate(); }
String NationalYU::extractDate(const String& s, const Date& d, const Time& t) const {
 return extractDate(s, d, t).extractDate(); }
String NationalYU::extractTime(const String& s, const Date& d, const Time& t) const {
 return extractTime(s, d, t).extractTime(); }
String NationalYU::weekday(const String& s, const Date& d, const Time& t) const {
 return weekday(s, d, t).weekday(); }
int NationalYU::getDate(const String& s, const Date& d, const Time& t) const {
 return getDate(s, d, t).getDate(); }
int NationalYU::extractDate(const String& s, const Date& d, const Time& t) const {
 return extractDate(s, d, t).extractDate(); }
int NationalYU::extractTime(const String& s, const Date& d, const Time& t) const {
 return extractTime(s, d, t).extractTime(); }
int NationalYU::weekday(const String& s, const Date& d, const Time& t) const {
 return weekday(s, d, t).weekday(); }

```

```

String s4=String::concat(s3,String::intToString(d.getMonth(),2));
String s5=String::concat(s4,".");
String s6=String::concat(s5,String::intToString(d.getYear(),4));
String s7=String::concat(s6,".");
return s7;
}

Date NationalYU::getDate(const String& s) const {
 //....
}

String NationalYU::getTime(const Time& t) const {
 String s1=String::intToString(t.getHour(),2);
 String s2=String::concat(s1,".");
 String s3=String::concat(s2,String::intToString(t.getMin(),2));
 String s4=String::concat(s3,".");
 String s5=String::concat(s4,String::intToString(t.getSec(),2));
 return s5;
}

Time NationalYU::getTime(const String& s) const {
 //....
}

String NationalYU::extractDate(const String& dateAndTime) const {
 //....
}

String NationalYU::extractTime(const String& dateAndTime) const {
 //....
}

String NationalYU::weekDay (int day) const {
 //....
}

int NationalYU::weekDay (const String& s) const {
 for (int i=1; i<8; i++)
 if (isSubstString(String::capit(daysOfWeek[i]),String::capit(s)))
 return i;
 return 0;
}

```

Sve specifičnosti sakrivene su od korisnika (u ovom slučaju klase Clock) iza interfejsa predstavljenog osnovnom klasom International. Preko mehanizma statičke funkcije Current() korisnik pristupa do konkretnog objekta koji se ponaša specifično, ne znajući, zapravo, o kom se objektu radi:

```

const int numOfNat = 2;
NationalUS natUS;
NationalYU natYU;
// etc.

International* Intl[numOfNat] = { &natUS, &natYU };

International* International::current=Intl[0];

```

```

International* International::Current () {
 return current;
}

void International::setCurrent (IntlCode cd) {
 if (cd<0 || cd>numOfNat) return;
 current=Intl[cd];
}

```

Realizacija klase Clock izgleda ovako:

```

String Clock::getTime () const {
 return International::Current()->getTime(d,t);
}

String Clock::getDate () const {
 return International::Current()->getDate(d);
}

String Clock::getTime () const {
 return International::Current()->getTime(t);
}

void Clock::wind (const String& dt) {
 if (!International::Current()->checkDate(dt)) return;
 International::Current()->getTime(dt,d,t);
}

void Clock::setDate (const String& ds) {
 if (!International::Current()->checkDate(ds)) return;
 d=International::Current()->getDate(ds);
}

void Clock::setTime (const String& ts) {
 if (!International::Current()->checkTime(ts)) return;
 t=International::Current()->getTime(ts);
}

```

## Zadatak

Realizovati apstrakciju budilnika. Korisnik može da navije budilnik na određeno vreme. U to vreme budilnik treba da pozove operaciju objekta korisnika budilnika.

### Rešenje

Budilnik treba da, kada dođe trenutak buđenja, „probudi“ korisnika pozivom neke njegove operacije. Objekat budilnik treba da sadrži pokazivač na objekat korisnika i da u datom trenutku pozove njegovu odgovarajuću funkciju. Ovaj pokazivač treba da inicijalizuje korisnik kada pravi budilnik. Prema tome, svaki korisnik budilnika treba da poseduje sledeći interfejs:

```

class Wakeable {
public:
 virtual void wakeUp () = 0;
};

```

Klasa korisnika budilnika treba da bude izvedena iz klase Wakeable i da redefiniše apstraktnu funkciju wakeUp()

Klasa budilnika treba da ima sledeći izgled:

```
class AlarmClock {
public:
 AlarmClock (wakeable* toWakeUp);
 void wind (Time alarmTime);
 void wind (Date alarmDate, Time alarmTime);

private:
 wakeable* myWakeable;
};

Korisnik budilnika pravi budilnik i pri tome mu dostavlja pokazivač na sebe, kako bi ga ovaj probudio u odgovarajuće vreme. Ovaj pokazivač budilnik čuva u atributu myWakeable. „Navijanje“ budilnika obavljaju funkcije wind (Time), koja navija budilnik na prvi naližak datog vremena (u tekucem danu ili sutradan) i wind (Date, Time) koja navija budilnik na zadati datum i vreme. Prema tome, korisnik budilnika može da izgleda ovako:

const Time hardWorkersWakeUp(5,0,0);

class HardWorker : public Wakeable {
public:
 HardWorker ();
 ~HardWorker ();

 void morningProcedure ();
 void eveningProcedure ();
 virtual void wakeUp ();

protected:
 void goToWork ();
 void takeDinner ();
 void makeBed ();

private:
 AlarmClock* myAlarmClock;
};

HardWorker::HardWorker () {
 //...
 myAlarmClock = new AlarmClock(this);
}

HardWorker::~HardWorker () {
 //...
 delete myAlarmClock;
}

void HardWorker::eveningProcedure () {
 takeDinner();
 makeBed();
 myAlarmClock->wind(hardWorkersWakeUp);
}

void HardWorker::wakeUp () {
 morningProcedure ();
 goToWork();
}
```

Ostaje da se realizuje mehanizam kojim će svi budilnici koji postoje u sistemu biti obavestavani o protoku vremena, da bi u odgovarajuće vreme mogli da probude svoje korisnike. Potreban je jedan objekat (*Singleton*) koji će imati spisak svih budilnika i koji će na svaki otkucaj sata (svake sekunde ili minuta, u zavisnosti od potrebne rezolucije) obavestavati sve budilnike da treba da provere svoje vreme buđenja. Svaki budilnik će se, prilikom nastanka, prijaviti ovom kontroleru budilnika, a prilikom ukidanja odjaviti. Kontroler će na svaki otkucaj sata proći kroz spisak budilnika i pozvati funkciju budilnika koja treba da proveri buđenje. Ovakav mehanizam obavestavanja svih objekata zainteresovanih za neku promenu predstavlja projektni obrazac *Observer*. Klasa *Singleton* kontrolera budilnika izgleda ovako:

```
class AlarmController {
public:
 static AlarmController* Instance ();
 ~AlarmController ();

 void sign (AlarmClock*);
 void unsign (AlarmClock*);

 void tick ();

protected:
 AlarmController ();

private:
 Collection<AlarmClock*> myClocks;
 IteratorCollection<AlarmClock*>* it;
};

AlarmController* AlarmController::Instance () {
 static AlarmController instance;
 return &instance;
}

AlarmController::AlarmController () {
 it=myClocks.createIterator();
}

AlarmController::~AlarmController () {
 delete it;
}

void AlarmController::sign (AlarmClock* clk) {
 myClocks.add(clk);
}

void AlarmController::unsign (AlarmClock* clk) {
 myClocks.remove(clk);
}

void AlarmController::tick () {
 for (it->reset(); it->isDone(); it->next()) {
 AlarmClock* clk = *it->currentItem();
 clk->checkTime();
 }
}
```

Kolekcija myClocks predstavlja spisak svih prijavljenih budilnika. U petlji funkcije tick() prolazi se iteratorom kroz ovu kolekciju i za svaki njen element poziva se funkcija budilnika checkTime().

Preostali deo klase AlarmClock treba da obezbedi da se budilnik prilikom nastanka prijavljuje, a prilikom nestanka odjavljuje kontroleru. To obezbeđuju konstruktor i destruktor:

```
class AlarmClock {
public:
 AlarmClock (Wakeable* toWakeUp);
 ~AlarmClock ();

private:
 Wakeable* myWakeable;
};

AlarmClock::AlarmClock (Wakeable* wa) : myWakeable(wa) {
 AlarmController::Instance()->sign(this);
}

AlarmClock::~AlarmClock () {
 AlarmController::Instance()->unsign(this);
}
```

#### Mehanizam navijanja realizuju funkcije wind():

```
class AlarmClock {
public:
 AlarmClock (Wakeable* toWakeUp);

 void wind (Time alarmTime);
 void wind (Date alarmDate, Time alarmTime);

private:
 int isWound;
 Date wad;
 Time wat;
};

AlarmClock::AlarmClock (Wakeable* wa)
 : myWakeable(wa), isWound(0),
 wad(Clock::Instance()->getDate()), wat(Clock::Instance()->getTime())
{
 AlarmController::Instance()->sign(this);
}

void AlarmClock::wind (Time alarmTime) {
 Time currentTime = Clock::Instance()->getTime();
 Date alarmDate = Clock::Instance()->getDate();
 if (alarmTime < currentTime) alarmDate.incdDay();
 wind(alarmDate, alarmTime);
}

void AlarmClock::wind (Date alarmDate, Time alarmTime) {
 wad=alarmDate;
 wat=alarmTime;
 isWound=1;
}
```

Funkcija checkTime() proverava tekuće vreme i vreme na koje je budilnik navijen i obavija buđenje ako je potrebno:

```
void AlarmClock::checkTime () {
 if (isWound==0) return;
 Time currentTime = Clock::Instance()->getTime();
 Date currentDate = Clock::Instance()->getDate();
 if (wat==currentTime && wad==currentDate) {
 isWound=0;
 myWakeable->wakeUp();
 }
}
```

#### Najzad, kompletna klasa AlarmClock izgleda ovako:

```
class AlarmClock {
public:
 AlarmClock (Wakeable* toWakeUp);
 ~AlarmClock ();

 void wind (Time alarmTime);
 void wind (Date alarmDate, Time alarmTime);

protected:
 friend class AlarmController;
 void checkTime ();

private:
 Wakeable* myWakeable;

 int isWound;
 Date wad;
 Time wat;
};

AlarmClock::AlarmClock (Wakeable* wa)
 : myWakeable(wa), isWound(0),
 wad(Clock::Instance()->getDate()), wat(Clock::Instance()->getTime())
{
 AlarmController::Instance()->sign(this);
}

AlarmClock::~AlarmClock () {
 AlarmController::Instance()->unsign(this);
}

void AlarmClock::wind (Time alarmTime) {
 Time currentTime = Clock::Instance()->getTime();
 Date alarmDate = Clock::Instance()->getDate();
 if (alarmTime < currentTime) alarmDate.incdDay();
 wind(alarmDate, alarmTime);
}

void AlarmClock::wind (Date alarmDate, Time alarmTime) {
 wad=alarmDate;
 wat=alarmTime;
 isWound=1;
}

void AlarmClock::checkTime () {
 if (isWound==0) return;
 Time currentTime = Clock::Instance()->getTime();
 Date currentDate = Clock::Instance()->getDate();
 if (wat==currentTime && wad==currentDate) {
 isWound=0;
 myWakeable->wakeUp();
 }
}
```

## Zadaci za samostalni rad

7. Realizovati klasu `Screen` koja apstrahuje fizički ekran i ima operacije za brisanje ekrana, ispis jednog znaka na odgovarajuću poziciju na ekranu i ispis niza znakova u oblast definisanu pravougaonikom na ekranu (red po red). Realizovati ovu klasu za postojeće tekstualno okruženje (npr. terminalni režim rada aplikacije). Koristeći ovu klasu, realizovati klasu `Window` koja predstavlja prozor, kao izvedenu klasu klase `View` iz zadatka 6. Prozor treba da ima mogućnosti dobijanja fokusa, pomeranja, promene veličine, minimiziranja, maksimiziranja, kao i otvaranja i zatvaranja. Definirati i virtualnu funkciju `draw`. U glavnom programu napraviti nekoliko prozora i višiti operacije nad njima.

## Glava 6

### Uvod u objektno modelovanje

#### Apstraktni tipovi podataka

Kompletan kôd za projektovanu biblioteku apstraktnih tipova podataka dat je u nastavku.

Datoteka `unbound.h`:

```
// Project: Abstract Data Types
// Module: Unbound
// File: unbound.h
// Date: 3.11.1996.
// Author: Dragan Milicev
// Contents:
// Class templates: Unbounded
// IteratorUnbounded

#ifndef UNBOUND_
#define UNBOUND_

// class template unbounded
// =====

template <class T>
class Unbounded {
public:
 Unbounded ();
 Unbounded (const Unbounded<T>&);
 ~Unbounded ();

 Unbounded<T>& operator= (const Unbounded<T>&);

 void append (const T&);
 void insert (const T&, int at=0);
 void remove (const T&);
 void remove (int at=0);
 void clear ();

 int isEmpty () const;
 int isFull () const;
 int length () const;
 const T& first () const;
 const T& last () const;
 const T& itemAt (int at) const;
 T& itemAt (int at);
 int location (const T& const;

protected:
 void copy (const Unbounded<T>&);
```

```

private:
 friend class IteratorUnbounded<T>;
 friend struct Element<T>;

 void remove (Element<T>*&);

 Element<T>* head;
 int size;
};

template <class T>
struct Element {
 T t;
 Element<T>* prev, *next;
 Element (const T&);
 Element (const T&, Element<T>* next);
 Element (const T&, Element<T>* prev, Element<T>* next);
};

template <class T>
Element<T>::Element (const T& e) : t(e), prev(0), next(0) {}

template <class T>
Element<T>::Element (const T& e, Element<T>* n)
 : t(e), prev(0), next(n) {
 if (n!=0) n->prev=this;
}

template <class T>
Element<T>::Element (const T& e, Element<T>* p, Element<T>* n)
 : t(e), prev(p), next(n) {
 if (n!=0) n->prev=this;
 if (p!=0) p->next=this;
}

template <class T>
void Unbounded<T>::remove (Element<T>* e) {
 if (e==0) return;
 if (e->next!=0) e->next->prev=e->prev;
 if (e->prev!=0) e->prev->next=e->next;
 else head=e->next;
 delete e;
 size--;
}

template <class T>
void Unbounded<T>::copy (const Unbounded<T>& r) {
 size=0;
 for (Element<T>* cur=r.head; cur!=0; cur=cur->next) append(cur->t);
}

```

```

template <class T>
void Unbounded<T>::clear () {
 for (Element<T>* cur=head, *temp=0; cur!=0; cur=temp) {
 temp=cur->next;
 delete cur;
 }
 head=0;
 size=0;
}

template <class T>
int Unbounded<T>::isEmpty () const { return size==0; }

template <class T>
int Unbounded<T>::isFull () const { return 0; }

template <class T>
int Unbounded<T>::length () const { return size; }

template <class T>
const T& Unbounded<T>::first () const { return itemAt(0); }

template <class T>
const T& Unbounded<T>::last () const { return itemAt(length()-1); }

template <class T>
const T& Unbounded<T>::itemAt (int at) const {
 static T except;
 if (isEmpty()) return except; // Exception!
 if (at>length()) at=length()-1;
 if (at<0) at=0;
 int i=0;
 for (Element<T>* cur=head; i<at; cur=cur->next, i++);
 return cur->t;
}

template <class T>
T& Unbounded<T>::itemAt (int at) {
 static T except;
 if (isEmpty()) return except; // Exception!
 if (at>length()) at=length()-1;
 if (at<0) at=0;
 int i=0;
 for (Element<T>* cur=head; i<at; cur=cur->next, i++);
 return cur->t;
}

template <class T>
int Unbounded<T>::location (const T& e) const {
 int i=0;
 for (Element<T>* cur=head; cur!=0; cur=cur->next, i++)
 if (cur->t==e) return i;
 return -1;
}

```

```

template<class T>
void Unbounded<T>::append (const T& t) {
 if (head==0) head=new Element<T>(t);
 else {
 for (Element<T> *cur=head; cur->next!=0; cur=cur->next);
 new Element<T>(t,cur,0);
 size++;
 }
}

template<class T>
void Unbounded<T>::insert (const T& t, int at) {
 if ((at>size)|| (at<0)) return;
 if (at==0) head=new Element<T>(t,head);
 else if (at==size) { append(t); return; }
 else {
 int i=0;
 for (Element<T> *cur=head; i<at; cur=cur->next, i++);
 new Element<T>(t,cur->prev,cur);
 size++;
 }
}

template<class T>
void Unbounded<T>::remove (int at) {
 if ((at>size)|| (at<0)) return;
 int i=0;
 for (Element<T> *cur=head; i<at; cur=cur->next, i++);
 remove(cur);
}

template<class T>
void Unbounded<T>::remove (const T& t) {
 remove (location(t));
}

template<class T>
Unbounded<T>::Unbounded () : size(0), head(0) {}

template<class T>
Unbounded<T>::Unbounded (const Unbounded<T>& r) : size(0), head(0) {
 copy(r);
}

template<class T>
Unbounded<T>::operator= (const Unbounded<T>& r) {
 clear();
 copy(r);
 return *this;
}

template<class T>
Unbounded<T>::~Unbounded () { clear(); }

```

```

////////////////////////////////////
// class template IteratorUnbounded
////////////////////////////////////

template <class T>
class IteratorUnbounded {
public:
 IteratorUnbounded (const Unbounded<T>*&);
 IteratorUnbounded (const IteratorUnbounded<T>&);
 ~IteratorUnbounded ();

 IteratorUnbounded<T>& operator= (const IteratorUnbounded<T>&);
 int operator== (const IteratorUnbounded<T>&);
 int operator!= (const IteratorUnbounded<T>&);

 void reset();
 int next ();

 int isDone() const;
 const T* currentItem() const;

private:
 const Unbounded<T>* theSupplier;
 Element<T>* cur;
};

template <class T>
IteratorUnbounded<T>::IteratorUnbounded (const Unbounded<T>* ub) :
 theSupplier(ub), cur(theSupplier->head) {}

template <class T>
IteratorUnbounded<T>::IteratorUnbounded (const IteratorUnbounded<T>& r) :
 theSupplier(r.theSupplier), cur(r.cur) {}

template <class T>
IteratorUnbounded<T>& IteratorUnbounded<T>::operator= (const
 IteratorUnbounded<T>& r) {
 theSupplier=r.theSupplier;
 cur=r.cur;
 return *this;
}

template <class T>
IteratorUnbounded<T>::~IteratorUnbounded () {}

template <class T>
int IteratorUnbounded<T>::operator== (const IteratorUnbounded<T>& r) {
 return (theSupplier==r.theSupplier)&&(cur==r.cur);
}

template <class T>
int IteratorUnbounded<T>::operator!= (const IteratorUnbounded<T>& r) {
 return !(*this==r);
}

```

```

template <class T>
void IteratorUnbounded<T>::reset () {
 cur=theSupplier->head;
}

template <class T>
int IteratorUnbounded<T>::next () {
 if (cur!=0) cur=cur->next;
 return !isDone();
}

template <class T>
int IteratorUnbounded<T>::isDone () const {
 return (cur==0);
}

template <class T>
const T* IteratorUnbounded<T>::currentItem () const {
 if (isDone()) return 0;
 else return &(cur->t);
}

#endif

Datoteka bound.h:
// Project: Abstract Data Types
// Module: Bound
// File: bound.h
// Date: 3.11.1996.
// Author: Dragan Milicev
// Contents:
// Class templates: Bounded
// IteratorBounded

#ifndef _BOUND_
#define _BOUND_

////////////////////////////////////
// class template Bounded
////////////////////////////////////

template <class T, int N>
class Bounded {
public:
 Bounded ();
 Bounded (const Bounded<T,N>&);
 ~Bounded ();

 Bounded<T,N>& operator= (const Bounded<T,N>&);

 void append (const T&);
 void insert (const T&, int at=0);
 void remove (const T&);
 void remove (int at=0);
 void clear ();

```

```

 isEmpty () const;
 isFull () const;
 length () const;
 const T& first () const;
 const T& last () const;
 const T& itemAt (int at) const;
 T& itemAt (int at);
 int location (const T&) const;

protected:
 void copy (const Bounded<T,N>&);

private:
 T dep[N];
 int size;
};

template<class T, int N>
void Bounded<T,N>::copy (const Bounded<T,N>& r) {
 size=0;
 for (int i=0; i<r.size; i++) append(r.itemAt(i));
}

template<class T, int N>
void Bounded<T,N>::clear () {
 size=0;
}

template<class T, int N>
int Bounded<T,N>::isEmpty () const { return size==0; }

template<class T, int N>
int Bounded<T,N>::isFull () const { return size==N; }

template<class T, int N>
int Bounded<T,N>::length () const { return size; }

template<class T, int N>
const T& Bounded<T,N>::first () const { return itemAt(0); }

template<class T, int N>
const T& Bounded<T,N>::last () const { return itemAt(length()-1); }

template<class T, int N>
const T& Bounded<T,N>::itemAt (int at) const {
 static T except;
 if (isEmpty()) return except; // Exception!
 if (at>length()) at=length()-1;
 if (at<0) at=0;
 return dep[at];
}

```



```

template<class T, int N>
T& Bounded<T,N>::itemat (int at) {
 static T except;
 if (isEmpty()) return except; // Exception!
 if (at>length()) at=length()-1;
 if (at<0) at=0;
 return deplat[];
}

template<class T, int N>
int Bounded<T,N>::location (const T& e) const {
 for (int i=0; i<size; i++) if (depl[i]==e) return i;
 return -1;
}

template<class T, int N>
void Bounded<T,N>::append (const T& t) {
 if (isFull()) return;
 depl[size++]=t;
}

template<class T, int N>
void Bounded<T,N>::insert (const T& t, int at) {
 if (isFull()) return;
 if ((at>size)|| (at<0)) return;
 for (int i=size-1; i>=at; i--) depl[i+1]=depl[i];
 depl[at]=t;
 size++;
}

template<class T, int N>
void Bounded<T,N>::remove (int at) {
 if ((at>size)|| (at<0)) return;
 for (int i=at+1; i<size; i++) depl[i-1]=depl[i];
 size--;
}

template<class T, int N>
void Bounded<T,N>::remove (const T& t) {
 remove(location(t));
}

template<class T, int N>
Bounded<T,N>::Bounded () : size(0) {}

template<class T, int N>
Bounded<T,N>::Bounded (const Bounded<T,N>& r) : size(0) {
 copy(r);
}

template<class T, int N>
Bounded<T,N>& Bounded<T,N>::operator= (const Bounded<T,N>& r) {
 clear();
 copy(r);
 return *this;
}

template<class T, int N>
Bounded<T,N>::~Bounded () { clear(); }

```

```

////////////////////////////////////
// class template IteratorBounded
////////////////////////////////////

template<class T, int N>
class IteratorBounded {
public:
 IteratorBounded (const Bounded<T,N>*);
 IteratorBounded (const IteratorBounded<T,N>&);
 ~IteratorBounded ();

 IteratorBounded<T,N>& operator= (const IteratorBounded<T,N>&);
 int operator!= (const IteratorBounded<T,N>&);
 int operator! = (const IteratorBounded<T,N>&);

 void reset();
 int next ();

 int isDone() const;
 const T* currentItem() const;

private:
 const Bounded<T,N>* theSupplier;
 int cur;
};

template<class T, int N>
IteratorBounded<T,N>::IteratorBounded (const Bounded<T,N>* b)
: theSupplier(b), cur(0) {}

template<class T, int N>
IteratorBounded<T,N>::IteratorBounded (const IteratorBounded<T,N>& r)
: theSupplier(r.theSupplier), cur(r.cur) {}

template<class T, int N>
IteratorBounded<T,N>& IteratorBounded<T,N>::operator= (const
 IteratorBounded<T,N>& r) {
 theSupplier=r.theSupplier;
 cur=r.cur;
 return *this;
}

template<class T, int N>
IteratorBounded<T,N>::~~IteratorBounded () {}

template<class T, int N>
int IteratorBounded<T,N>::operator!= (const IteratorBounded<T,N>& r) {
 return (theSupplier!=r.theSupplier) && (cur!=r.cur);
}

template<class T, int N>
int IteratorBounded<T,N>::operator! = (const IteratorBounded<T,N>& r) {
 return !(*this==r);
}

```

```

template <class T, int N>
void IteratorBounded<T,N>::reset () {
 cur=0;
}

template <class T, int N>
int IteratorBounded<T,N>::next () {
 if (!isDone()) cur++;
 return !isDone();
}

template <class T, int N>
int IteratorBounded<T,N>::isDone () const {
 return (cur>=theSupplier->length());
}

template <class T, int N>
const T* IteratorBounded<T,N>::currentItem () const {
 if (isDone()) return 0;
 else return &theSupplier->itemAt(cur);
}

#endif

// Project: Abstract Data Types
// Module: Collection
// File: collect.h
// Date: 5.11.1996.
// Author: Dragan Milicev
// Contents:
// Class templates: Collection
// CollectionB
// CollectionU
// IteratorCollection
// IteratorCollectionB
// IteratorCollectionU
//
// #ifndef _COLLECT_
// #define _COLLECT_
//
// #include "bound.h"
// #include "unbound.h"

```

#### Datoteka collect.h:

```

// class template Collection
//
template <class T>
class Collection {
public:
 virtual ~Collection () {}

 CollectionB () {}
 CollectionB (const Collection<T>&);
 virtual ~CollectionB () {}

 Collection<T>& operator= (const Collection<T>&);
 virtual IteratorCollection<T>* createIterator () const;

 virtual void add (const T& t);
 virtual void remove (const T& t);
 virtual void remove (int at);
 virtual void clear () {}
};

// class template CollectionB
//
template <class T, int N>
class CollectionB : public Collection<T> {
public:
 CollectionB () {}
 CollectionB (const Collection<T>&);
 virtual ~CollectionB () {}

 Collection<T>& operator= (const Collection<T>&);
 virtual IteratorCollection<T>* createIterator () const;

 virtual void add (const T& t);
 virtual void remove (const T& t);
 virtual void remove (int at);
 virtual void clear () {}
};

// class template CollectionU
//
template <class T>
class CollectionU {
public:
 virtual ~Collection () {}

 CollectionB () {}
 CollectionB (const Collection<T>&);
 virtual ~CollectionB () {}

 Collection<T>& operator= (const Collection<T>&);
 virtual IteratorCollection<T>* createIterator () const;

 virtual void add (const T& t);
 virtual void remove (const T& t);
 virtual void remove (int at);
 virtual void clear () {}
};

```

```

virtual int isEmpty() const { return rep.isEmpty(); }
virtual int isFull() const { return rep.isFull(); }
virtual int length() const { return rep.length(); }
virtual int location(const T& t) const { return rep.location(t); }

private:
friend class IteratorCollectionB<T,N>;
Bounded<T,N> rep;
};

// class template IteratorCollectionB
// class template IteratorCollectionB
// class template IteratorCollectionB

template<class T, int N>
class IteratorCollectionB : public IteratorCollection<T>,
private IteratorBounded<T,N> {
public:
 IteratorCollectionB(const CollectionB<T,N>* c)
 : IteratorBounded<T,N>(c->rep) {}

 virtual ~IteratorCollectionB() {}

 virtual void reset() { IteratorBounded<T,N>::reset(); }
 virtual int next() { return IteratorBounded<T,N>::next(); }
 virtual int isDone() const { return IteratorBounded<T,N>::isDone(); }
 virtual const T* currentItem() const
 { return IteratorBounded<T,N>::currentItem(); }
};

template<class T, int N>
CollectionB<T,N>::CollectionB(const Collection<T>& r) {
 copy(r);
}

template<class T, int N>
Collection<T>& CollectionB<T,N>::operator=(const Collection<T>& r) {
 return Collection<T>::operator=(r);
}

template<class T, int N>
IteratorCollectionB<T,N>::createIterator() const {
 return new IteratorCollectionB<T,N>(this);
}

// class template CollectionB
// class template CollectionB
// class template CollectionB

template<class T>
class CollectionB : public Collection<T> {
public:
 CollectionB() {}
 CollectionB(const Collection<T>& r);
 virtual ~CollectionB() {}

 Collection<T>& operator=(const Collection<T>& r);
 virtual IteratorCollection<T>* createIterator() const;
};

```

```

virtual void add(const T& t) { rep.append(t); }
virtual void remove(const T& t) { rep.remove(t); }
virtual void remove(int ac) { rep.remove(ac); }
virtual void clear() { rep.clear(); }

virtual int isEmpty() const { return rep.isEmpty(); }
virtual int isFull() const { return rep.isFull(); }
virtual int length() const { return rep.length(); }
virtual int location(const T& t) const { return rep.location(t); }

private:
friend class IteratorCollectionU<T>;
Unbounded<T> rep;
};

// class template IteratorCollectionU
// class template IteratorCollectionU
// class template IteratorCollectionU

template<class T>
class IteratorCollectionU : public IteratorCollection<T>,
private IteratorUnbounded<T> {
public:
 IteratorCollectionU(const CollectionU<T>* c)
 : IteratorUnbounded<T>(c->rep) {}

 virtual ~IteratorCollectionU() {}

 virtual void reset() { IteratorUnbounded<T>::reset(); }
 virtual int next() { return IteratorUnbounded<T>::next(); }
 virtual int isDone() const { return IteratorUnbounded<T>::isDone(); }
 virtual const T* currentItem() const
 { return IteratorUnbounded<T>::currentItem(); }
};

template<class T>
CollectionU<T>::CollectionU(const Collection<T>& r) {
 copy(r);
}

template<class T>
Collection<T>& CollectionU<T>::operator=(const Collection<T>& r) {
 return Collection<T>::operator=(r);
}

template<class T>
IteratorCollectionU<T>::createIterator() const {
 return new IteratorCollectionU<T>(this);
}

#endit

```

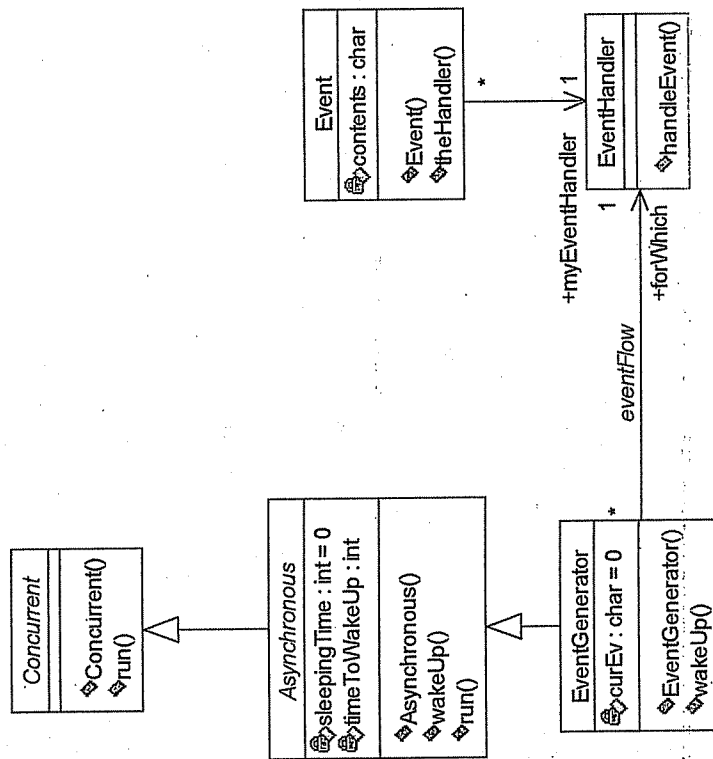
## Zadatak

Realizovati jednostavan sistem za simulaciju orijentisan na događaje. U jezgri sistema leži apstrakcija `Concurrent`, koja predstavlja apstraktnu osnovnu klasu za klase čiji se objekti konkurentno izvršavaju u sistemu. Posebna vrsta konkurentnih objekata su tzv. asinhroni objekti, predstavljeni klasom `Asynchronous`, koji povremeno obavljaju neki posao.

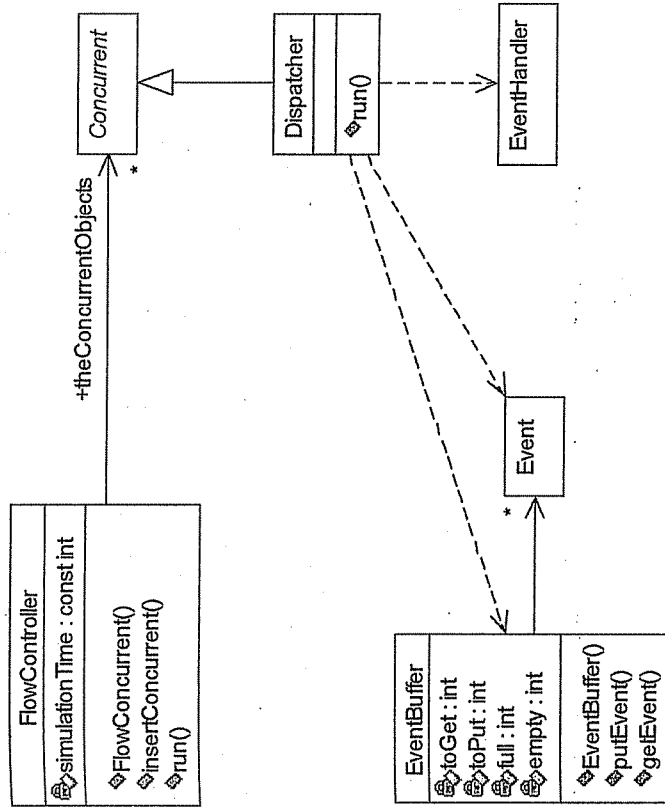
Posebna vrsta ovakvih objekata su generatori događaja, predstavljeni klasom EventGenerator. Svaki generator događaja vezan je za jednog primaoca događaja, predstavljenog klasom EventHandler, kome generiše događaje. Događaji se slažu u bafer događaja, objekat klase EventBuffer, dok jedan konkurentni objekat Dispatcher uzima događaje iz ovog bafera i prosleđuje ih primaocima kojima su namenjeni.

### Rešenje

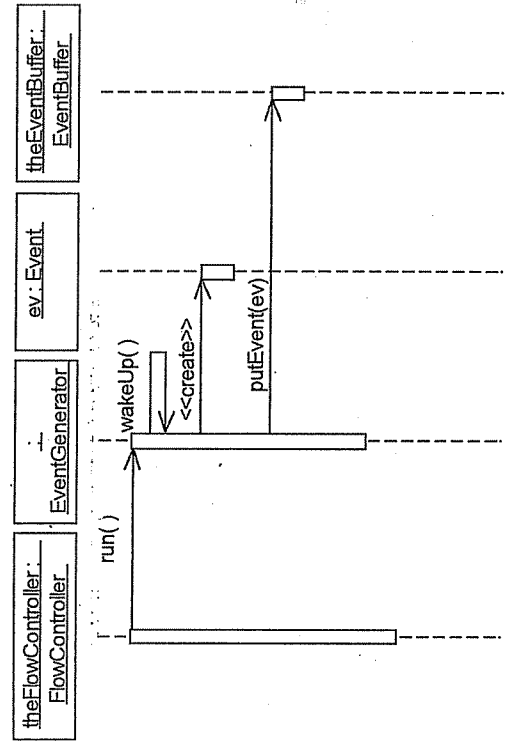
Dijagram klasa koji prikazuje ključne apstrakcije u sistemu:

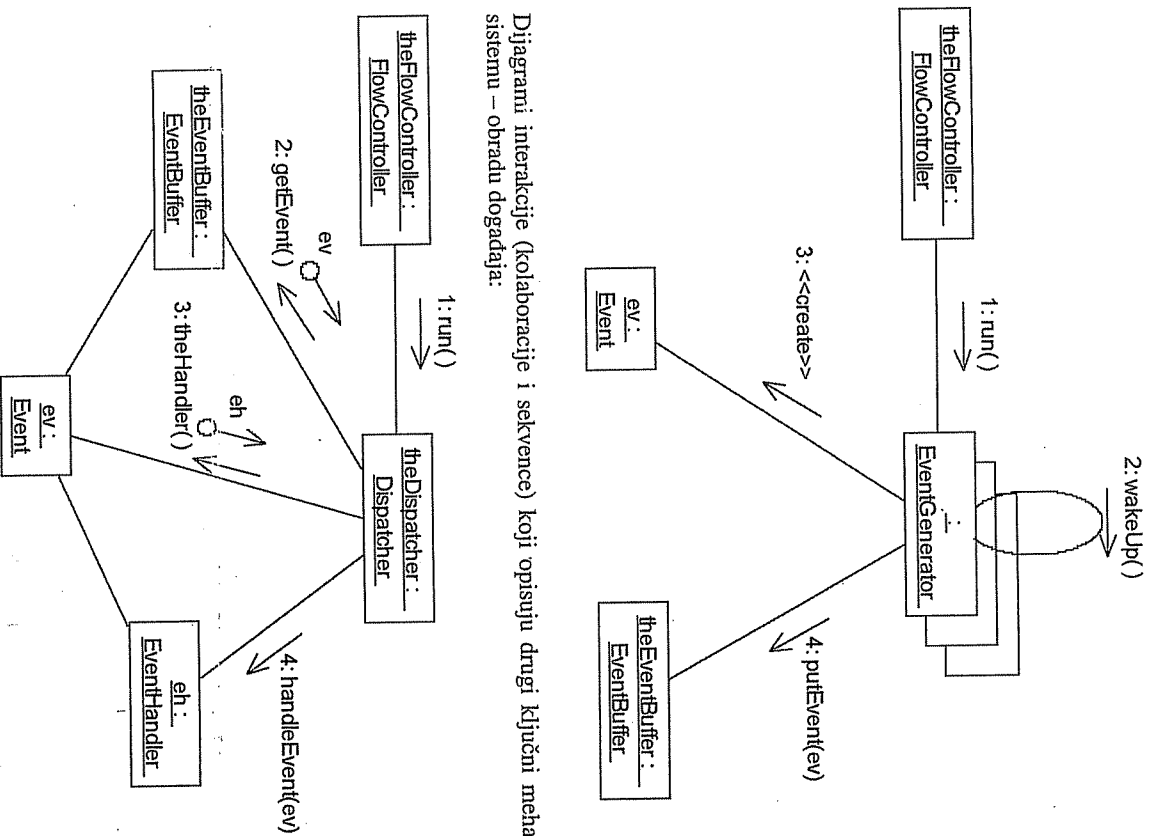


Dijagram klasa koji prikazuje implementacione klase:

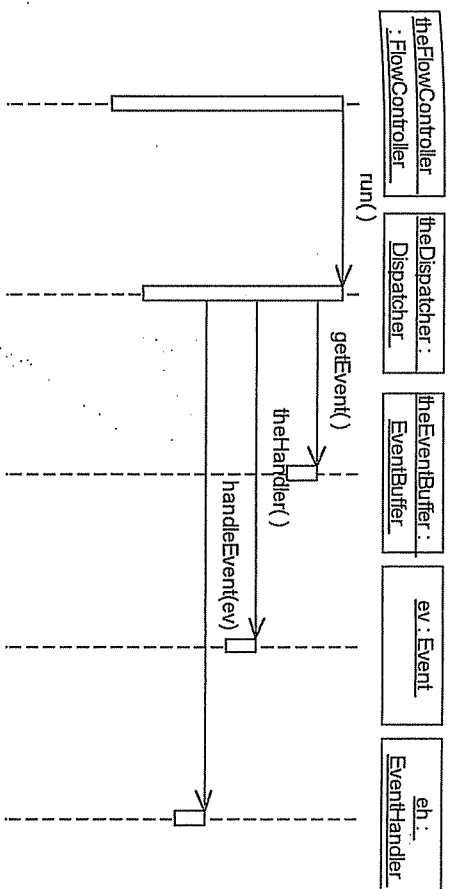


Dijagrami interakcije (sekvence i kolaboracije) koji opisuju prvi ključni mehanizam u sistemu – generisanje događaja:





Dijagrami interakcije (kolaboracije i sekvence) koji opisuju drugi ključni mehanizam u sistemu – obradu događaja:



#### Izvorni kôd:

```

// Project: simpleev.py
// Description: Simple Event-Driven System
// Model: Main (the only one)
// File: simpleev.cpp
// Version: 1.0
// Author: Dragan Milicev
// Comments: A simple OO design/C++ example

// *****
// Constants:
const int tEvgen = 5; // time constant for EventGenerator
const int maxEvgens = 10; // maximum number of EventGenerators
const int maxEvhands = 10; // maximum number of EventHandlers
const int maxConcur = maxEvgens+1; // maximum number of Concurrent objects
const int maxEvents = 10; // maximum number of Events

// *****
// Class definitions:
class Concurrent {
public:
 Concurrent ();
 virtual void run () = 0;
};

class Asynchronous : public Concurrent {
public:
 Asynchronous (int timeToWakeUp);
 virtual void run ();
protected:
 virtual void wakeup () = 0;
};

```

```

private:
 int sleepingTime;
 const int timeToWakeUp;
};

class EventHandler; // forward

class EventGenerator : public Asynchronous {
public:
 EventGenerator (EventHandler* forWhich);
protected:
 virtual void wakeUp ();
private:
 EventHandler* forWhich;
 char curEv;
};

class Dispatcher : public Concurrent {
public:
 virtual void run ();
};

class FlowController {
public:
 FlowController (int simulationTime);
 void insertConcurrent (Concurrent* toInsert);
 void run ();
private:
 Concurrent* theConcurrentObjects [maxConcur];
 int numOfConcurrentObjects;
 const int simulationTime;
};

class Event {
public:
 Event (EventHandler* theHandler);
private:
 EventHandler* myEventHandler;
 char contents;
};

class EventBuffer {
public:
 EventBuffer ();
 void putEvent (Event* toPut);
 Event* getEvent ();
};

```

```

private:
 theEventQueue [maxEvents];
 int toGet, toPut;
 int full, empty;
};

class EventHandler {
public:
 void handleEvent (Event* toHandle);
};

// *****
// Static objects definitions:
// *****

FlowController theFlowController (100000);
Dispatcher theDispatcher;
EventBuffer theEventBuffer;

// Event handlers:
EventHandler EventHandler1, EventHandler2, EventHandler3;

// Event generators:
EventGenerator EventGenerator1 (&EventHandler1),
EventGenerator2 (&EventHandler1),
EventGenerator3 (&EventHandler2),
EventGenerator4 (&EventHandler2),
EventGenerator5 (&EventHandler3);

// *****
// Class implementations:
// *****

Concurrent::Concurrent () { theFlowController.insertConcurrent(this); }

Asynchronous::Asynchronous (int t) : timeToWakeUp(t), sleepingTime(0) {}

void Asynchronous::run () {
 if (++sleepingTime==timeToWakeUp) {
 wakeUp();
 sleepingTime=0;
 }
}

EventGenerator::EventGenerator (EventHandler* fw) : Asynchronous (timeToWakeUp(fw),
 forWhich(fw), curEv(0) {}

void EventGenerator::wakeUp () {
 theEventBuffer.putEvent (new Event (forWhich, curEv));
 curEv= (curEv+1) %128;
}

```

```
void Dispatcher::run () {
 Event* ev=theEventBuffer.getEvent();
 if (ev!=0) ev->theHandler()->handleEvent(ev);
}
```

```
FlowController::FlowController (int s) : simulationTime(s),
 numOfConcurrentObjects(0) {}
```

```
void FlowController::insertConcurrent (Concurrent* c) {
 if (numOfConcurrentObjects<maxConcur
 theConcurrentObjects[numOfConcurrentObjects++]=c;
};
```

```
void FlowController::run () {
 for (int t=0; t<simulationTime; t++)
 for (int i=0; i<numOfConcurrentObjects; i++)
 theConcurrentObjects[i]->run();
}
```

```
Event::Event (EventHandler* fw, char c) : myEventHandler(fw), contents(c) {}

EventHandler* Event::theHandler () { return myEventHandler; }
```

```
EventBuffer::EventBuffer () : toGet(0), toPut(0), full(0), empty(1) {}
```

```
void EventBuffer::putEvent (Event* e) {
 if (full==1) { // buffer full!
 delete e; return;
 }
 theEventQueue[toPut]=e;
 toPut=(toPut+1)%maxEvents;
 if (toPut==toGet) full=1;
 empty=0;
}
```

```
Event* EventBuffer::getEvent () {
 if (empty==1) return 0; // buffer empty!
 Event* e=theEventQueue[toGet];
 toGet=(toGet+1)%maxEvents;
 if (toGet==toPut) empty=1;
 full=0;
 return 0;
}
```

```
void EventHandler::handleEvent (Event* e) {
 // here is the place for event handling!
 delete e;
}
```

```
// *****
// Main program:
void main () {
 theFlowController.run();
}
```

Ljubica Lazarević

# Zadaci za vežbu

Glava 7

Elementi jezika C++ koji nisu objektno orijentisani

Glava 8

Klase i preklapanje operatora

Glava 9

Nasledivanje

## Elementi jezika C++ koji nisu objektno orijentisani

### Zadatak 1

Realizovati potrebne funkcije za sortiranje datog niza brojeva partijskom metodom (engl. *Quick Sort*).

#### Uputstvo

Ideja *Quick Sort* metode sortiranja nizova je u tome da se dati niz podeli tako da se levo od odgovarajućeg, unapred određenog elementa niza (engl. *pivot point*)  $a[k]$ ,  $0 \leq k \leq n-1$ , ( $n$  je broj elemenata polaznog niza), nađu elementi koji nisu veći od njega, a desno oni elementi koji nisu manji od datog elementa. Pošto je u tom slučaju izabrani element  $a[k]$  baš na mestu na kom i treba da se nađe u sortiranom nizu, levo i desno od njega su dobijene particije niza od  $k$ , tj.  $n-k-1$  elemenata. Pri tom, dobijene particije nisu sortirane, tako da je potrebno da se one međusobno nezavisno sortiraju. Postupak deobe niza se ponavlja rekursivno na dobijene particije sve dok se ne dobiju particije sa po jednim elementom.

Postoji više mogućnosti za izbor pivot elementa, a jedna od njih je da to bude poslednji element niza. Na početku formiranja particija uoč se ta referentna vrednost, a zatim se, polazeći od početka niza, traži prvi element čija vrednost nije manja od referentne. Za ovo pretraživanje koristi se brojač  $i$ . Zatim se, polazeći od kraja niza, traži prvi element čija vrednost nije veća od referentne. Za ovo pretraživanje koristi se brojač  $j$ . Ako je  $i < j$ , elementi  $a[i]$  i  $a[j]$  međusobno menjaju mesta i postupak formiranja particije se nastavlja ka sredini niza. Ako se opisano pomeranje brojača  $i$  i  $j$  završi tako da je  $i \geq j$ , referentni element i element  $a[i]$  menjaju mesta. Time je referentni element došao na svoje mesto i završeno je formiranje particije.

Pažljivim izborom referentne vrednosti, moguće je postići efikasnost metode  $O(n \log n)$ , dok linearne metode sortiranja imaju kompleksnost  $O(n^2)$ . Kao što se može primetiti, ova razlika značajnije dolazi do izražaja pri velikim vrednostima dimenzije ulaznog niza  $n$ .



## Rešenje

```

// Modul sort.h
// Deklaracija funkcije
#ifndef _sort_h
#define _sort_h

extern void gsort (int a[], int n);

#endif

// Modul sort.cpp
// Pomocna funkcija za medjusobnu zamenu dva elementa
inline void swap (int& x, int& y) { int z=x; x=y; y=z; }

// Razbija niz a od n elemenata na dve particije
// Vraca indeks elementa koji deli niz na particije
int divide (int a[], int n) {
 int b=a[n-1], i=0, j=n-1;
 while (i<j){
 while (a[i]<b && i<=n-1) i++;
 while (a[j]>=b && j>=0) j--;
 if (i<j) swap(a[i],a[j]);
 }
 swap(a[i],a[n-1]);
 return i;
}

// Sortira niz a od n elemenata tako sto ga prvo podeli na dve particije,
// a zatim sortira dobijene particije rekurzivnim pozivima
void gsort (int a[], int n){
 if (n>1){
 int i=divide(a,n);
 gsort(a,i);
 gsort(a+i+1,n-i-1);
 }
}

// Modul sort_main.cpp
// Testiranje funkcija definisanih u sort.cpp
#include <iostream.h>
#include "sort.h"

```

```

void main () {
 int n;
 cout << "Duzina niza je: ";
 cin >> n;
 int* a=new int[n];
 if (a){
 cout<<"unesite pocetni niz:\n";
 for (int i=0; i<n; i++) cin >> a[i];
 gsort(a,n);
 cout << "\nsortirani niz:\n";
 for (i=0; i<n; i++) cout << a[i] << " ";
 delete [] a;
 }
}

```

## Zadatak 2

Implementirati binarno sortirano stablo i funkcije za njegovo pretraživanje (engl. *binary tree search*). Realizovati funkcije umetanja, brisanja elementa iz stabla, kao i pretraživanja stabla (pronalaženja čvora u stablu sa zadatim elementom, ako postoji).

## Uputstvo

Binarno sortirano stablo je specijalan slučaj binarnog stabla. Binarno stablo je takva struktura podataka u kojoj su elementi smešteni u čvorovima stabla koje ima definisan koren (engl. *root*), pri čemu, u zavisnosti od svoje vrednosti, svaki roditeljski čvor može imati 0, 1 ili 2 čvora potomka (engl. *child nodes*). Čvor koji nema roditeljski čvor naziva se koren (engl. *root*). Čvor koji nema potomke naziva se list (engl. *leaf*). Za ovu strukturu tipično se definišu operacije umetanja, brisanja elementa iz stabla, kao i pretraživanja stabla.

Binarno sortirano stablo ima dodatnu osobinu da svaki čvor u levom podstablu čvora  $X$  ima manju (ili ne veću) vrednost od  $X$ , dok svaki čvor u desnom podstablu čvora  $X$  ima vrednost ne manju (ili veću) od  $X$ .

Da bi se pronalazila određena vrednost u binarnom sortiranom stablu, pretraživanje počinje od korena. Poređenjem vrednosti svakog čvora sa odgovarajućom vrednošću koja se traži, a zatim, na osnovu rezultata poređenja, prelaskom na potomke na odgovarajućoj strani čvora, moguće je brzo odrediti da li tražena vrednost postoji u stablu ili ne, ukoliko je stablo približno balansirano. Balansirano stablo je ono kod koga svaki čvor ima ili 2 ili 0 potomka. Degenerisano stablo je ono kod koga svi čvorovi (osim lista) imaju po jednog potomka. Degenerisano stablo prestaje u spregnutu listu, pa se, samim tim, prilikom pretraživanja gube i poboljšanja u efikasnosti pretraživanja stabla.

Kompleksnost algoritma pretraživanja stabla koje je približno balansirano je  $O(\log n)$ , gde je  $n$  broj čvorova stabla. U najgorem slučaju pretraživanja degenerisanog stabla, kompleksnost je  $O(n)$ .

## Rešenje

```
// Modul btrees.h

#ifndef _btrees_h
#define _btrees_h

typedef char* T;

struct Node {
 Node* left;
 Node* right;
 Node* parent;
 T data;
};

extern Node* root;

extern void insertNode (T data);
extern void deleteNode (Node*);
extern void deleteTree (Node*);
extern Node* findNode (T data);
extern void printTree (Node*);

#endif

// Modul btrees.cpp

#include <stdlib.h>
#include <string.h>
#include <iostream.h>

#include "btrees.h"

Node* root = 0; // koren stabla

// Pronalazi cvor u stablu
// koji ce biti roditeljski cvor novog cvora x
Node* findParent (Node* x) {
 Node* current=root;
 while (current) {
 int n=strcmp(x->data, current->data);
 if (n) break;
 parent = current;
 current = (n<0) ? current->left : current->right;
 }
 x->parent = parent;
 return x->parent;
}
```

```
// Dodaje element data na odgovarajuće mesto u stablu
// Vraca pokazivac na dodati cvor
void insertNode (T data) {
 Node* x=new Node;
 if (x == 0) exit(1);
 x->data = data;
 x->left = 0;
 x->right = 0;
 Node* parent=findParent(x);
 if (parent) {
 int n=strcmp(x->data, parent->data);
 if (n<0) {
 if (!parent->left) parent->left = x;
 }
 else {
 if (!parent->right) parent->right = x;
 }
 }
 else root = x;
}

// Brise cvor z iz stabla
void deleteNode (Node* z) {
 if (z==0) return;
 Node *x=0, *y=0;
 if (z->left==0 || z->right==0) y = z; // z nema oba potomka
 else {
 // ako z ima potomke, trazi se krajnji levi potomak
 y = z->right;
 while (y->left!=0) y = y->left; // koji je veci od elementa u cvoru z
 }
 if (! (y->left==0 && y->right==0)) // Ako y ima bar jednog potomka,
 if (y->left!=0) // on se dodeljuje cvoru x
 x = y->left;
 else
 x = y->right;
 if (x) x->parent = y->parent;
 if (y->parent)
 if (y == y->parent->left)
 y->parent->left = x;
 else
 y->parent->right = x;
 else
 root = x;
 if (y!=z) {
 // Cvor y zauzima svoje konacno mesto umesto cvora z
 y->left = z->left;
 if (y->left) y->left->parent = y;
 y->right = z->right;
 if (y->right) y->right->parent = y;
 y->parent = z->parent;
 if (z->parent)
 if (z == z->parent->left)
 z->parent->left = y;
 else
 z->parent->right = y;
 else
 root = y;
 delete z;
 }
 else
 delete y;
}
```

```
// Pronalazi cvor u stablu koji sadrzi element data
Node* findNode (T data) {
 Node* current = root;
 while (current!=0) {
 int n=strcmp(data, current->data);
 if (n==0) return current;
 else
 current = (n<0) ? current->left : current->right;
 }
 return 0;
}

// Brise stablo
void deleteTree (Node* p) {
 if (p==0) return;
 deleteTree(p->left);
 deleteTree(p->right);
 delete p;
}

// Ispisuje podatke o cvorovima u stablu
void printTree (Node* p) {
 if (p==0) return;
 printTree(p->left);
 cout << "Cvor: " << p->data << "\n\troditelj=" <<
 if (!p->parent)
 cout << "0";
 else
 cout << p->parent->data;
 cout << ", levi potomak=" <<
 if (!p->left)
 cout << "0";
 else
 cout << p->left->data;
 cout << ", desni potomak=" <<
 if (!p->right)
 cout << "0";
 else
 cout << p->right->data << "\n";
 printTree(p->right);
}
```

```
// Modul btrees_main.cpp
#include "btrees.h"

void main () {
 insertNode("izgleda");
 insertNode("da");
 insertNode("ce");
 insertNode("ovih");
 insertNode("dana");
 insertNode("u");
 insertNode("Beogradu");
 insertNode("biti");
 insertNode("lepo");
 insertNode("toplo");
 insertNode("i");
 insertNode("suncano");
 insertNode("vreme");
 insertNode("dana");

 printTree(root);

 Node* n1=findNode("Beogradu");
 deleteNode(n1);
 printTree(root);
 deleteTree(root);
}
```

### Zadatak 3

Implementiraj funkcije za rad sa heš (engl. *hash*) tabelom. Realizovati funkcije umetanja, brisanja elementa iz tabele, kao i pretraživanja tabele.

#### Uputstvo

Heš tabela je struktura podataka koja se koristi za maksimiranje brzine pristupa određenom podatku. Operacije tipične za heš tabele jesu umetanje, brisanje elementa i pretraživanje tabele.

Element se ubacuje u tabelu tako što se prvo pozove heš funkcija elementa koja izračunava lokaciju na koju element treba ubaciti. Zatim se proverava da li je izračunata lokacija slobodna. Ako jeste, element se smesti u nju, a ukoliko nije, potrebno je razrešiti nastalu koliziju.

Bitan faktor je biranje odgovarajuće heš funkcije da bi se smanjile kolizije. Funkcija koja uniformno distribuira vrednosti po celoj tabeli doprinese znatnom povećanju brzine pristupa elementima tabele. Postoji više metoda za povećanje uniformnosti heš funkcija, i samim tim za povećanje efikasnosti tabele.

Jedna od metoda je i metoda deljenja (eng. *division method*). Funkcija izračunava ostatak pri deljenju vrednosti podatka (engl. *key value*) ukupnom veličinom tabele. Pokazuje se da je ovaj metod najefikasniji (u smislu što rede pojave kolizija) ukoliko je veličina tabele prost broj.

```
int hashFunction(DataItem item) {
 return item.key % hashtableSize;
}
```

Jedan od načina rešavanja kolizija koje se neminovno javljaju, ma koliko da se pažljivo biraju heš funkcije i veličine tabele, jeste tzv. *open hashing* metoda. Ova metoda se zasniva na tome da se svaka lokacija tabele zapravo pretvara u početak (glavu) jednostruko ulančane liste podataka. Ako se pri pokušaju umetanja određenog elementa u tabelu dogodi kolizija, element se dodaje na početak (ili kraj) odgovarajuće liste koja je određena izračunatom heš vrednošću.

### Rešenje

```
// Modul hash.h

#ifndef hash_h
#define hash_h

typedef int T;

struct Node {
 Node* next; // Sledeći cvor
 T data; // Element smesten u cvoru
};

extern Node** hashTable;
extern int hashTableSize;

// Umeće novi element u tabelu
extern Node* insertNode (T data);

// Pronalazi element u tabeli
extern Node* findNode (T data);

// Brise element iz tabele
extern void deleteNode (T data);

// Ispisuje elemente tabele
extern void tablePrint ();

#endif

// Modul hash.cpp

#include <stdlib.h>
#include <iostream.h>
#include "hash.h"

typedef int HashTableIndex;

Node** hashTable;
int hashTableSize; // Broj ulaza u hash tabeli

// Određivanje hash vrednosti
HashTableIndex hash (T data) {
 return (data % hashTableSize);
}
```

```
// Umetanje cvora koji sadrži element Data na pocetak liste
Node* insertNode(T data) {
 HashTableIndex bucket = hash(data);
 Node* p = new Node;
 if (p==0) {
 cout<< "Out of memory (InsertNode).\n";
 exit(1);
 }
 Node* p0 = hashTable[bucket];
 hashTable[bucket] = p;
 p->next = p0;
 p->data = data;
 return p;
}

// Brisanje cvora koji sadrži element data
void deleteNode (T data) {
 HashTableIndex bucket = hash(data);
 Node* p = hashTable[bucket];
 Node* p0 = 0;
 while (p && (p->data!=data)) {
 p0 = p;
 p = p->next;
 }
 if (!p) return;
 if (p0)
 p0->next = p->next;
 else
 hashTable[bucket] = p->next;
 delete p;
}

// Pronalazenje cvora sa elementom data
Node* findNode (T data) {
 Node* p = hashTable[hash(data)];
 while (p && (p->data!=data)) p = p->next;
 return p;
}

// Ispis tabele
void tablePrint () {
 for (int i=0; i<hashTableSize; i++) {
 for (Node* p = hashTable[i]; p!=0; p=p->next)
 cout << p->data << "\t";
 cout << "\n";
 }
}
```

```
// Modul hash_main.cpp
#include <stdlib.h>
#include <iostream.h>
#include "hash.h"

void main () {
 hashtableSize = 5;
 hashtable = new Node* [hashtableSize];
 for (int i=0; i<hashtableSize; i++) hashtable[i]=0;

 Node *n1, *n2, *n3, *n4, *n5, *n6, *n7, *n8, *n9, *n10;
 n1=insertNode(73);
 n2=insertNode(12);
 n3=insertNode(33);
 n4=insertNode(11);
 n5=insertNode(5);
 n6=insertNode(10);
 n7=insertNode(44);
 n8=insertNode(104);
 n9=insertNode(21);

 cout << "Tabela T:\n";
 tablePrint();

 n10 = findNode(17);
 n10 = findNode(12);
 deleteNode(5);

 cout << "\nTabela T' bez cvora sa elementom Data=5:\nT'=\n";
 tablePrint();
}
```

## Zadatak 4

Implementirati Dijkstra algoritam za pronalaženje minimalnog puta između dva čvora u neusmerenom grafu čijim su granama pridružene dužine kao nenegativni brojevi.

### Uputstvo

Dijkstra algoritam je jedan od najefikasnijih algoritama za pronalaženje minimalnog puta između dva unapred zadata čvora u neusmerenom grafu. Algoritam se, zapravo, sastoji iz dva dela. Prvi deo algoritma predstavlja određivanje minimalnog rastojanja između dva čvora u grafu, a drugi pronalaženje odgovarajućeg minimalnog puta kroz graf (dužina tog puta je upravo ona koja je izračunata u prvom delu algoritma).

U prvom koraku algoritma, rastojanja svih čvorova grafa od početnog čvora postavljaju se na vrlo veliku vrednost (teoretski beskonačno), dok sam početni čvor ima rastojanje 0. Algoritam se zatim svodi na pretraživanje grafa dok god postoji par čvorova grafa  $(i, j)$  za koje važi da je  $d_j - d_i > l_{ij}$ , gde je:

$d_i$  - tekuće izračunato rastojanje čvora  $i$  grafa od početnog čvora;

$d_j$  - tekuće izračunato rastojanje čvora  $j$  od početnog čvora;

$l_{ij}$  - dužina grane između čvorova  $i$  i  $j$  grafa.

Ukoliko se pronađe takav par čvorova, rastojanje čvora  $j$  od početnog čvora postavlja se na:  $d_j := d_i + l_{ij}$ .

Po završetku pretraživanja, u vrednostima  $d_i$  svih čvorova grafa dobija se minimalno rastojanje od početnog čvora, čime je završen prvi deo algoritma.

U drugom delu algoritma kreće se od krajnjeg izabranog čvora (za koji je određeno minimalno rastojanje) ka početnom izabranom čvoru, ali tako što se traži sused  $i$  datog čvora  $j$  takav da važi da je  $d_j - d_i = l_{ij}$ . Kao rezultat ovog dela algoritma dobija se niz čvorova kroz koje prolazi minimalan put između dva izabrana čvora grafa.

### Rešenje

```
// Modul dijkstra.h
#ifndef _dijkstra_h
#define _dijkstra_h

extern int numOfNodes, startNodeid, endNodeid;

// Inicijalizuje graf
extern void initGraph (int numOfNodes);

// Brise sve strukture
extern void destroy ();

// Pronalazi minimalno rastojanje između dva čvora
extern void minPath (int startNodeid, int endNodeid);

// Stampa najkraci put do ciljnog čvora
extern void printPath (int endNodeid);

#endif

// Modul dijkstra.cpp
#include <iostream.h>
#include <stdlib.h>
#include "dijkstra.h"

const int NONE = 9999;

// Struktura koja predstavlja cvor grafa
struct Node {
 // Tekuće minimalno
 // izracunato rastojanje
 int dist;

 // Veza ka sledecem cvoru u
 // listi cvorova za pretrazivanje
 int nextId;

 // Veza prema cvoru preko kog je
 // ostvareno tekuće minimalno rastojanje
 int prevMinDistId;
}
```

```

// Niz cvorova grafa
Node *graph;

// Glava i rep liste cvorova za pretrazivanje i broj cvorova u toj listi
int headNodeId=-1;
int tailNodeId=-1;
int cnt;

// Matrica grana u grafu
int **matrix;

// Inicijalizacija matrice grana grafa;
// Ako između dva cvorova (i,j) postoji grana,
// onda matrix[i][j] predstavlja njenu duzinu;
// Ako je matrix[i][j]<0 znaci da između cvorova (i,j) ne postoji grana

void initMatrix (int numOfNodes) {
 matrix=new int* [numOfNodes];
 for (int i=0; i<numOfNodes; i++)
 matrix[i]=new int[numOfNodes];

 for (i=0; i<numOfNodes-1; i++){
 for (int j=i+1; j<numOfNodes; j++){
 cout << "Nunesite duzinu grane između cvorova (" << i
 << ", " << j << ") l=";
 int l;
 cin >> l;
 matrix[i][j]=matrix[j][i]=l;
 }
 matrix[i][i]=0;
 }
}

// Inicijalizuje cvorove grafa;
// dist u svim cvorovima postavlja na NONE,
// osim u pocetnom cvoru;

void initGraph (int numOfNodes) {
 graph=new Node[numOfNodes];
 for (int i=0; i<numOfNodes; i++){
 graph[i].nextId=-1;
 graph[i].prevMinDist=-1;
 if (i!=startNodeId)
 graph[i].dist=NONE;
 else
 graph[i].dist=0;
 }
 initMatrix(numOfNodes);
}

// Brise sve strukture
void destroy () {
 delete [] graph;
 delete [] matrix;
}

```

```

// Vraca broj cvorova u listi za pretrazivanje
inline int count() { return cnt; }

// Ulanjava cvor u listu cvorova za pretrazivanje
void put (int n) {
 if (tailNodeId>-1) {
 graph[tailNodeId].nextId=n;
 tailNodeId=n;
 }

 if (headNodeId==-1)
 headNodeId=tailNodeId=n;
 cnt++;

 // Razlanjava i vraca prvi cvor iz liste za pretrazivanje
 int remove () {
 int tmp=headNodeId;
 if (headNodeId>-1) {
 headNodeId=graph[headNodeId].nextId;
 if (headNodeId==-1) tailNodeId=-1;
 cnt--;
 return tmp;
 }
 return -1;
 }

 // Pronalazi minimalno rastojanje između pocetnog i krajnjeg cvorova
 int findDist (int startNodeId) {
 put(startNodeId); // Ulančavanje startnog cvorova u listu
 while (count()>0) {
 int n=remove();
 if (n>-1)
 for (int i=0; i<numOfNodes; i++)
 if (matrix[n][i]>0) // ukoliko postoji grana
 if (graph[i].dist >
 graph[i].prevMinDist+n;
 graph[i].prevMinDist=n;
 // trenutno min. rastojanje
 // se ostvaruje preko cvorova n
 put(i);
 }
 return graph[endNodeId].dist;
 }
 }

 // Stampa putanju preko koje se ostvaruje minimalno
 // rastojanje između zadatih cvorova
 void printPath (int nodeId) {
 if (graph[nodeId].prevMinDist>-1)
 printPath(graph[nodeId].prevMinDistId);
 cout << nodeId << " ";
 }
}

```

```
// Ispisuje trazenu putanju
void minPath (int startNodeId, int endNodeId) {
 int d=findDist(startNodeId);
 cout << d;
}
```

```
// Modul dijkskra_main.cpp
#include <iostream.h>
#include "dijkstra.h"
int numofNodes, startNodeId, endNodeId;
```

```
void main() {
 cout << "Unesite broj cvorova grafa: ";
 cin >> numofNodes;

 cout << "Unesite pocetni cvor: ";
 cin >> startNodeId;

 cout << "Unesite krajnji cvor: ";
 cin >> endNodeId;

 initGraph(numofNodes);

 cout << "Minimalno rastojanje izmedju cvorova ("
 << startNodeId<<","<<endNodeId<<") = ";
 minPath(startNodeId, endNodeId);

 cout << "Najkraca putanja prolazi kroz cvorove: "
 << minPath(endNodeId);
 destroy();
}
```

## Glava 8

# Klase i preklapanje operatora

## Zadatak 1

Projekovati klasu koja realizuje polinom sa realnim koeficijentima i realnim argumentom. U glavnom programu demonstrirati mogućnosti klase.

### Rešenje

```
// Modul pol.h
#ifndef _pol_h
#define _pol_h
class istream;
class ostream;
class Polinom {
public:
 Polinom () : n(-1), a(0) {}
 Polinom (const double* coef, int numofCoeef);
 Polinom (const Polinom& p);
 ~Polinom ();

 Polinom& operator= (const Polinom&);
 int getPower () const {return n;};

 friend Polinom operator+ (const Polinom& p1, const Polinom& p2);
 friend Polinom operator- (const Polinom& p1, const Polinom& p2);
 friend istream& operator>> (istream& is, Polinom& p);
 friend ostream& operator<< (ostream& os, const Polinom& p);

 // Vraca vrednost polinoma u tacki x, P(x)
 double operator() (double x) const;

protected:
 // Rise cec polinom
 void release ();

private:
 // Red polinoma
 int n;
 // Niz koeficijenata polinoma
 double* a;
};
#endif
```

```

// Modul pol.cpp
#include <iostream.h>
#include "pol.h"

Polinom::Polinom (const double* k, int r) : n(r) {
 a=new double[r+1];
 if (a)
 for (int i=0; i<n; i++) a[i]=k[i];
}

Polinom::Polinom (const Polinom& p) : n(p.n) {
 if (n>0)
 a=new double[n+1];
 else
 a=0;
 if (a)
 for (int i=0; i<n; i++) a[i]=p.a[i];
 else
 n=-1;
}

Polinom::~Polinom() {
 release();
}

void Polinom::release() {
 n=-1;
 delete [] a;
}

//Izracunava vrednost polinoma u tacki x, P(x)
double Polinom::operator() (double x) const {
 double s=0;
 for (int i=n; i>=0; s=s*x+a[i--]);
 return s;
}

Polinom& Polinom::operator= (const Polinom& p) {
 if (&p != this) {
 a=(p.n>0) ? new double[p.n+1] : 0;
 if (a) {
 n=p.n;
 for (int i=0; i<n; i++) a[i]=p.a[i];
 }
 return *this;
 }
}

Polinom operator+ (const Polinom& p1, const Polinom& p2) {
 Polinom q;
 q.a=(p1.n>p2.n) ? new double [p1.n+1] : new double [p2.n+1];
 for (int i=0; i<p1.n&&(i<p2.n); i++) {
 q.a[i]=p1.a[i]+p2.a[i];
 if (q.a[i]!=0)
 q.n=i;
 }
}

```

```

 for (;i<p1.n;i++) {
 q.a[i]=p1.a[i];
 q.n=i;
 }
 for (;i<p2.n;i++) {
 q.a[i]=p2.a[i];
 q.n=i;
 }
 return q;
 }

 Polinom operator- (const Polinom& p1, const Polinom& p2) {
 Polinom q;
 q.a=(p1.n>p2.n) ? new double [p1.n+1] : new double [p2.n+1];
 for (int i=0; i<p1.n&&(i<p2.n); i++) {
 q.a[i]=p1.a[i]-p2.a[i];
 if (q.a[i]!=0)
 q.n=i;
 }
 for (;i<p1.n;i++) {
 q.a[i]=p1.a[i];
 q.n=i;
 }
 for (;i<p2.n;i++) {
 q.a[i]=(-1)*p2.a[i];
 q.n=i;
 }
 return q;
 }

 istream& operator>> (istream& is, Polinom& p) {
 cout<<"Unesite red polinoma: ";
 is>> p.n;
 p.a=new double[p.n+1];
 cout<<"Unesite koeficijente polinoma: ";
 for (int i=p.n; i>=0; i--)
 is>> p.a[i];
 return is;
 }

 ostream& operator<< (ostream& os, const Polinom& p) {
 os<<"\nRed polinoma: "<<p.n<<"\n";
 os<<"Koeficijenti polinoma: [";
 for (int i=p.n; i>=0; i--)
 os<<p.a[i]<<" ";
 os<<"\n";
 return os;
 }

 // Modul pol_main.cpp
#include <iostream.h>
#include "pol.h"

void main () {
 double a1[3]={2,1,3,7,1,12};
 double a2[4]={-2,1,1,3,-1,12,1,5};
 Polinom p1(a1,2),p2(a2,3);
 cout<<p1;

```



```

 cout<<p2;
 int i=p1.getPow();
 cout<<i<<"\n";
 double s=p2(3);
 cout<<"\nVrednost polinoma p2(3) ="<<s<<"\n";
 Polinom q= p1+p2;
 i=q.getPow();
 cout<<q;

 q=p1-p2;
 cout<<q;
}

```

## Zadatak 2

Projektovati klasu koja realizuje matricu proizvoljnih dimenzija. Dimenzije matrice se zadaju pri njihovom nastanku, u vreme izvršavanja programa, što znači da je matrica dinamička. Smatrati da su elementi matrice celi brojevi.

### Rešenje

```

// Modul matrix.h

#ifndef Matrix
#define _Matrix

class istream;
class ostream;

typedef int Type;

class Matrix{
public:
 Matrix (int m, int n);
 Matrix (const Matrix& mat);
 Matrix ();

 Matrix& operator = (const Matrix&);

 friend int operator == (const Matrix&, const Matrix&);
 friend int operator != (const Matrix&, const Matrix&);
 Type& operator () (int i, int j);

 friend Matrix operator + (const Matrix&);
 friend Matrix operator - (const Matrix&);
 friend Matrix operator + (const Matrix&, const Matrix&);
 friend Matrix operator - (const Matrix&, const Matrix&);
 friend Matrix operator * (const Matrix&, const Matrix&);
 friend Matrix operator * (const Matrix&, const Type&);
 friend Matrix operator * (const Type&, const Matrix&);

 friend Matrix trans (const Matrix&);
 friend Matrix compl (const Matrix&, int i, int j);
 friend Type det (const Matrix&);
 friend Type cofact (const Matrix&, int i, int j);
 friend Matrix adj (const Matrix&);
 friend Matrix inv (const Matrix&);

 Matrix& operator += (const Matrix&);
 Matrix& operator -= (const Matrix&);
 Matrix& operator *= (const Matrix&);
 Matrix& operator *= (const Type&);
}

```

```

 friend istream& operator >> (istream&, Matrix&);
 friend ostream& operator << (ostream&, const Matrix&);

protected:
 void allocate ();
 void release ();
 void copy (const Matrix&);

private:
 int cols, rows;
 Type **mat;
}

#endif

// Modul matrix.cpp

// Definicija funkcija iz modula matrix.h
#include <stdlib.h>
#include <iostream.h>
#include "matrix.h"

// Alokacija dinamičke strukture za predstavljanje matrice
Matrix::allocate () {
 mat=new Type*[rows];
 for (int i=0;i<rows;i++) {
 mat[i]=new Type[cols];
 for (int j=0;j<cols;j++) mat[i][j]=Type(0);
 }
}

// Dealokacija dinamičke strukture za predstavljanje matrice
void Matrix::release () {
 for (int i=0;i<rows;i++) delete [] mat[i];
 delete [] mat;
}

// Kopiranje matrice-argumenta u strukturu objekta čiji je član
void Matrix::copy (const Matrix& right) {
 for (int i=0;i<rows;i++)
 for (int j=0;j<cols;j++) this->mat[i][j]=right.mat[i][j];
}

// Konstruktor
Matrix::Matrix (int m, int n): rows(m), cols(n) {
 allocate();
}

// Konstruktor kopije
Matrix::Matrix (const Matrix& right): rows(right.rows), cols(right.cols) {
 allocate();
 copy(right);
}

```

```

// Destruktor
Matrix::~Matrix () {
 release ();
}

Matrix& Matrix::operator = (const Matrix& right) {
 if (right==this) return *this;
 if (right.rows!=rows || right.cols!=cols) return *this;
 release();
 allocate();
 copy(right);
 return *this;
}

Type& Matrix::operator () (int i, int j) {
 static Type error(0);
 if (i<0 || i>rows || j<0 || j>cols) return error;
 return mat[i-1][j-1];
}

int operator == (const Matrix& left, const Matrix& right) {
 if (right.rows!=left.rows || right.cols!=left.cols) return 0;
 for (int i=0; i<left.rows; i++)
 for (int j=0; j<left.cols; j++)
 if (left.mat[i][j]!=right.mat[i][j]) return 0;
 return 1;
}

int operator != (const Matrix& left, const Matrix& right) {
 return !(left==right);
}

Matrix operator + (const Matrix& m) {return m;}

Matrix operator - (const Matrix& m) {return -Type(1)*m;}

Matrix operator * (const Matrix& left, const Matrix& right) {
 Matrix m=left;
 return m*=right;
}

Matrix operator - (const Matrix& left, const Matrix& right) {
 Matrix m=left;
 return m-=right;
}

Matrix& Matrix::operator += (const Matrix& right) {
 if (rows!=right.rows || cols!=right.cols) return *this;
 for (int i=0; i<rows; i++)
 for (int j=0; j<cols; j++)
 mat[i][j]+=right.mat[i][j];
 return *this;
}

```

```

Matrix& Matrix::operator -= (const Matrix& right) {
 if (rows!=right.rows || cols!=right.cols) return *this;
 for (int i=0; i<rows; i++)
 for (int j=0; j<cols; j++)
 mat[i][j]-=right.mat[i][j];
 return *this;
}

Matrix operator * (const Matrix& left, const Matrix& right) {
 Matrix m (left.rows, right.cols);
 if (left.cols!=right.rows) return m;
 for (int i=0; i<left.rows; i++)
 for (int j=0; j<right.cols; j++)
 for (int k=0; k<left.cols; k++)
 m.mat[i][j]+=left.mat[i][k]*right.mat[k][j];
 return m;
}

Matrix& Matrix::operator *= (const Type& t) {
 for (int i=0; i<rows; i++)
 for (int j=0; j<cols; j++)
 mat[i][j]*=t;
 return *this;
}

Matrix operator * (const Matrix& left, const Type& t) {
 Matrix m=left;
 return m*=t;
}

Matrix operator * (const Type& t, const Matrix& right) {
 return right*t;
}

Matrix& Matrix::operator *= (const Matrix& right) {
 return *this*=right;
}

istream& operator >> (istream& is, Matrix& m) {
 for (int i=0; i<m.rows; i++)
 for (int j=0; j<m.cols; j++)
 is>>m.mat[i][j];
 return is;
}

ostream& operator << (ostream& os, const Matrix& m) {
 os<<"\n";
 for (int i=0; i<m.rows; i++) {
 os<<"{";
 for (int j=0; j<m.cols; j++)
 os<< m.mat[i][j] << ((j<m.cols-1)?", ":"\n");
 }
 return os;
}

```

```

// Funkcija za transponovanje matrice
Matrix trans (const Matrix& m) {
 Matrix temp(m.cols, m.rows);
 for (int i=0; i<m.rows; i++)
 for (int j=0; j<m.cols; j++)
 temp.mat[j][i] = m.mat[i][j];
 return temp;
}

// Vraca matricu sa izostavljenom odgovarajucom vrstom i kolonom
Matrix compl (const Matrix& m, int r, int c) {
 Matrix temp(m.rows-1, m.cols-1);
 if (m.cols<1 || m.rows<1) return temp;
 for (int i=0; i<m.rows; i++) {
 if (i==r-1) continue;
 for (int j=0; j<m.cols; j++) {
 if (j==c-1) continue;
 temp.mat[i][j] = m.mat[i][j];
 }
 x++;
 }
 return temp;
}

// Izracunava determinantu matrice
Type det (const Matrix& m) {
 if (m.rows==1) return 0;
 if (m.rows==1) return m.mat[0][0];
 Type temp=Type(0);
 for (int j=0; j<m.cols; j++) {
 if (j==c-1) continue;
 temp+= (Type(c)*m.mat[0][j]*det(compl(m, 1, j+1)));
 }
 return temp;
}

// Izracunava kofaktor
Type cofact (const Matrix& m, int r, int c) {
 if (m.rows==1) return 0;
 if ((r+c)%2==0)
 return (-det (compl(m, r, c)));
 else
 return (det (compl(m, r, c)));
}

// Vraca adjungovanu matricu
Matrix adj (const Matrix& m) {
 Matrix temp(m.rows, m.cols);
 if (m.rows==1) return temp;
 for (int i=0; i<m.rows; i++)
 for (int j=0; j<m.cols; j++)
 temp.mat[i][j] = cofact(m, i+1, j+1);
 return trans(temp);
}

// Vraca inverznu matricu
Matrix inv (const Matrix& m) {
 Type temp=det(m);
 if (temp!=0)
 return (1/temp)*adj(m);
 else
 return Matrix(0,0);
}

```

## Zadatak 3

Realizovati klasu *RecycleBin* koja se koristi za potrebe brze dinamičke alokacije i dealokacije objekata neke klase. Za neku klasu *T* za koju se želi brza alokacija i dealokacija objekata, treba obezbediti jedan objekat klase *RecycleBin*. Ovaj objekat funkcioniše tako što u svojoj listi čuva sve dealocirane objekte date klase. Kada se traži alokacija novog elementa, *RecycleBin* prvo vadi iz svoje liste već spreman reciklirani objekat, tako da je alokacija u tom slučaju veoma brza. Ako je lista prazna, tj. ako nema prethodno recikliranih objekata date klase, *RecycleBin* pri alokaciji koristi ugrađeni alokator *new*. Pri dealokaciji, objekat se ne oslobađa već se reciklira, tj. upisuje se u listu klase *RecycleBin*.

### Uputstvo

Pored klase *RecycleBin*, potrebno je napraviti i klase *List* i *ListElement*, koje realizuju apstrakcije dinamičke dvostruko ulančane liste i elementa te liste, respektivno. Da bi klasa *RecycleBin* mogla da se koristi u nekoj klasi, ta klasa mora biti izvedena iz klase *ListElement* jer će se objekti te klase smeštati u internu listu klase *RecycleBin*. Izvedena klasa mora posedovati jedan statički objekat klase *RecycleBin* i mora definisati operatore *new* i *delete*. Ovo je potrebno da bi se što efikasnije implementirale operacije umetanja i izbacivanja elementa iz liste.

### Rešenje

```

// Modul Recycle.h
#ifndef Recycle_h
#define Recycle_h
#include <stddef.h>

// Element liste.
class ListElement {
protected:
 ListElement() : next(0), prev(0) {}
 ~ListElement() {}
 const ListElement* getNext() const;
 const ListElement* getPrev() const;
private:
 ListElement* next;
 ListElement* prev;
 friend class List;
 friend class ListIterator;
};

// Lista elemenata tipa ListElement.
class List {
public:
 List() : head(0), tail(0), cnt(0) {}

```

```

// Smesta novi element na pocetak liste.
void putFirst(ListElement* e);

// Smesta novi element na kraj liste.
void putLast(ListElement* e);

// Vadi iz liste i vraca prvi element.
ListElement* removeFirst();

// Vadi iz liste i vraca poslednji element.
ListElement* removeLast();

// UKlanja dati element iz liste.
void remove(ListElement* e);

// Smesta novi element e liste iza datog elementa after.
void insertAfter(ListElement* e, ListElement* after);

// Smesta novi element e liste ispred datog elementa before.
void insertBefore(ListElement* e, ListElement* before);

// Prazni listu (ne unistava sadrzaj!).
void clear();

// Vraca prvi element, bez vadjjenja.
const ListElement* getFirst() const;

// Vraca poslednji element, bez vadjjenja.
const ListElement* getLast() const;

// Vraca True ako je lista prazna, inace vraca False.
int isEmpty() const;

// Kreira i vraca (po vrednosti) iterator za sebe.
ListIterator createIterator() const;

private:
// Glava liste.
ListElement* head;

// Rep liste.
ListElement* tail;

// Broj elemenata u listi
int cnt;

friend class ListIterator;
};

// Iterator liste.
// Moze da iterira i unapred i unazad.

class ListIterator {
public:
// Konstruktor.
// Vezuje iterator za datu listu.
ListIterator(const List* lst);

// Postavlja iterator na pocetak liste.
void goFirst();

// Postavlja iterator na kraj liste.
void goLast();

```

```

// Postavlja iterator na pocetak liste.
// Isto kao goFirst().
void reset();

// Postavlja iterator na sledeci element liste.
void next();

// Postavlja iterator na prethodni element liste.
void prev();

// Vraca True ako je iterator stigao do kraja iteracije,
// za oba smeru (unapred ili unazad).
int isDone() const;

// Vraca tekuci element liste.
const ListElement* currentItem() const;

private:
// Tekuci element u listi.
const ListElement* cur;

// Lista koju iterator iterira.
const List* myList;
};

class RecycleBin;

// Klasa koja koristi klasu RecycleBin
class Element : public ListElement {
public:
// Konstruktor
Element(int k) : el(k) {}

// Redefinisani operator new
static void* operator new (size_t);

// Redefinisani operator delete
static void operator delete (void *);

private:
static RecycleBin recycleBin;
int el;
};

// Klasa koja sluzi za alokaciju i dealokaciju
// objekata argument klase X.
// X mora biti izvedena iz ListElement.

class RecycleBin {
public:
// Vraca jedan nov, recikliran objekat.
// Ako takvog nema u RecycleBin, alocira ga sa new.
void* getNew();

// Vraca u reciklazu jedan objekat klase X.
void recycle(Element *);

// Prazni RecycleBin.
void empty();

```

```

protected:
 RecycleBin();

 friend class Element;

private:
 // Repozitorijum recikliranih (dealociranih) objekata koji
 // su spremni za ponovno korišćenje.
 List repository;
};

// Class ListElement
inline const ListElement* ListElement::getNext() const {
 return next;
}

inline const ListElement* ListElement::getPrev() const {
 return prev;
}

// Class List
inline void List::clear() {
 head=tail=0;
}

inline const ListElement* List::getFirst() const {
 return head;
}

inline const ListElement* List::getLast() const {
 return tail;
}

inline int List::isEmpty() const {
 return !head;
}

inline ListIterator List::createIterator() const {
 return ListIterator((List*)this);
}

// Class ListIterator
inline ListIterator::ListIterator(const List* lst) : cur(lst->head), mylist(lst)
{}

inline void ListIterator::goFirst() {
 cur=mylist->head;
}

inline void ListIterator::goLast() {
 cur=mylist->tail;
}

inline void ListIterator::reset() {
 cur=mylist->head;
}

```

```

inline void ListIterator::next() {
 if (cur) cur=cur->next;
}

inline void ListIterator::prev() {
 if (cur) cur=cur->prev;
}

inline int ListIterator::isDone() const {
 return !cur;
}

inline const ListElement* ListIterator::currentItem() const {
 return cur;
}

// Class Element
inline void* Element::operator new (size_t) {
 return recycleBin.getNew();
}

inline void Element::operator delete (void *e) {
 recycleBin.recycle((Element*)e);
}

// Class RecycleBin
inline RecycleBin::RecycleBin() {}

#endif

// Modul Recycle.cpp
#include <stdlib.h>
#include <iostream.h>
#include "Recycle.h"

void List::putFirst(ListElement* e) {
 if (!e)
 return;
 if (head)
 head->prev=e;
 e->next=head;
 e->prev=0;
 head=e;
 if (!tail)
 tail=e;
 cnt++;
}

void List::putLast(ListElement* e) {
 if (!e)
 return;
 if (tail)
 tail->next=e;
 e->prev=tail;
 e->next=0;
 tail=e;
 if (!head)
 head=e;
 cnt++;
}

```

```

ListElement* List::removeFirst() {
 ListElement* e=head;
 if (e && e->next)
 e->next->prev=0;
 if (e)
 head=e->next;
 if (e && !head)
 tail=0;
 cnt--;
 return e;
}

```

```

ListElement* List::removeLast() {
 ListElement* e=tail;
 if (e && e->prev)
 e->prev->next=0;
 if (e)
 tail=e->prev;
 if (e && !tail)
 head=0;
 cnt--;
 return e;
}

```

```

void List::remove(ListElement* e) {
 if (!e)
 return;
 if (!e->prev) {
 removeFirst();
 return;
 }
 if (!e->next) {
 removeLast();
 return;
 }
 e->prev->next=e->next;
 e->next->prev=e->prev;
 e->prev->next=0;
 cnt--;
}

```

```

void List::insertAfter(ListElement* e, ListElement* after) {
 if (!e)
 return;
 if (!after) {
 putFirst(e);
 return;
 }
 if (!after->next) {
 putLast(e);
 return;
 }
 e->prev=after;
 e->next=after->next;
 after->next->prev=e;
 after->next=e;
 cnt++;
}

```

```

void List::insertBefore(ListElement* e, ListElement* before) {
 if (!e)
 return;
 if (!before) {
 putLast(e);
 return;
 }
}

```

```

if (!before->prev) {
 putFirst(e);
 return;
}
e->next=before;
e->prev=before->prev;
before->prev->next=e;
before->prev=e;
cnt++;
}

RecycleBin Element::recycleBin;

void* RecycleBin::getNew() {
 if (repository.isEmpty())
 return new char[sizeof(Element)];
 else
 return repository.removeFirst();
}

void RecycleBin::recycle(Element* e) {
 repository.putLast(e);
}

void RecycleBin::empty() {
 while (!repository.isEmpty()) {
 const ListElement* e=repository.removeFirst();
 delete [] (char*)e;
 }
}

// Modul Recycle_main.cpp
#include <iostream.h>
#include "Recycle.h"

void main () {
 Element* e1=new Element(5);
 Element* e2=new Element(3);
 Element* e3=new Element(4);

 delete e1;
 delete e2;
 delete e3;

 e1=new Element(1);
 e2=new Element(0);
}

```

## Nasledivanje

### Zadatak 1

Realizovati potrebne klase za definisanje geometrijskih figura u ravni. Zajedničke osobine za sve figure su položaj i orijentacija u ravni, površina i obim. Položaj se zadaje koordinatama težišta. Pored virtuelnih funkcija za izračunavanje površine i obima, potrebno je obezbediti i virtuelnu funkciju *draw* za iscrtaavanje grafika koji mogu biti primitivni (sastojati se od grafika jedne geometrijske figure), ili složeni (sastojati se od više grupisanih grafika). Za ove potrebe koristiti projektni obrazac *Composite*. U glavnom programu prikazati mogućnosti klase primenjene na različite geometrijske figure.

#### Uputstvo

Projektni obrazac *Composite* obezbeduje grupisanje objekata u hijerarhijsku strukturu stabla. Moguće je grupisati komponente da bi se oformile veće, kompleksnije strukture, koje se dalje rekurzivno mogu grupisati tako da formiraju još složenije strukture (engl. *compositions*), itd. Takođe, ovaj obrazac obezbeduje korišćenje kompozitne strukture na isti način kao i prostog objekta, tako da korisnik ne mora da vodi računa o tome da li radi sa individualnim (primitivnim) objektom ili sa kompozicijom, već ih tretira na potpuno isti način.

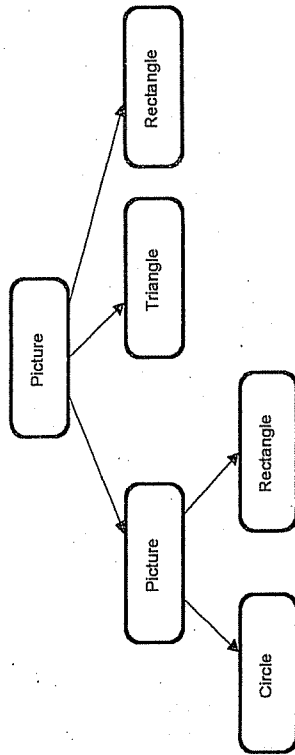
Ključna ideja ovog projektnog obrasca je apstraktna klasa (*Graphic*) koja predstavlja kako primitivne, tako i složene grafike (kompozite). Ova klasa predstavlja interfejs za objekte koji učestvuju u kompoziciji i implementira podrazumevano ponašanje zajedničko za sve klase.

Izvedene klase *Circle*, *Rectangle* i *Triangle* definišu objekte primitivnih grafika i predstavljaju listove u strukturi složenog grafika. Ovi objekti implementiraju operacije specifično za svaku od navedenih figura. Pošto ovi grafici nemaju grafike potomke, nijedna od ovih klasa ne implementira operacije koje se tiču grafika potomaka.

Klasa *Picture* definiše agregat objekata grafika. Ova klasa je izvedena iz klase *Graphic*, ali je sa njom i u asocijaciji, tj. slika može sadržavati više grafika koji mogu biti primitivni ili kompozitni. Ova klasa definiše ponašanje komponenti koje imaju potomke, sadrži komponente potomke i implementira operacije koje se tiču potomaka.

Korisnik upotrebljava interfejs klase *Graphic* da bi se obratio objektima u kompozitnoj strukturi. Ako je referencirani objekat kompozitni (*Picture*), on poslednje zahtev svojim potomcima, koji ga rekurzivno prosledjuju dalje, tako da se, u tom slučaju, stablo obilazi po dubini sve dok se ne stigne do krajnjeg referenciranog čvora, tj. objekta.

Na sledećem dijagramu prikazana je tipična kompozitna struktura rekurzivno grupisanih objekata klase *Graphic*.



### Rešenje

// Definicija klase Point (modul point.h)

```

#ifndef _point_h
#define _point_h

class istream;
class ostream;

typedef double Real;

class Point {
public:
 // Konstruktori
 Point (const Real xx=0, const Real yy=0) : x(xx), y(yy) {}
 Point (const Point& t) : x(t.x), y(t.y) {}

 // Apscisa tacke
 Real x() const {return x;}

 // Ordinata tacke
 Real y() const {return y;}

 friend istream& operator >> (istream& is, Point& t);
 friend ostream& operator << (ostream& os, const Point& t);

private:
 // Koordinate tacke
 Real x,y;
};

const Point ORG;

#endif

// Definicije funkcija iz klase Point definisane u point.h (modul point.cpp)

#include "point.h"
#include <iostream.h>

```

```

istream& operator >> (istream& is, Point& t) {
 return is >> t.x >> t.y;
}

ostream& operator << (ostream& os, const Point& t) {
 return os << "(" << t.x << ", " << t.y << ")";
}

// Definicija osnovne klase geometrijskih figura (modul graphic.h)

#ifndef _graphic_h
#define _graphic_h

#include "point.h"
#include "recycle.h"

class Graphic : public ListElement {
public:
 // Virtualni destruktor
 virtual ~Graphic () {}

 // Definisane položaja figure
 virtual void set (const Real xx, const Real yy) { T=Point(xx,yy); }
 // Virtualno pomeranje figure
 virtual void move (const Real dx, const Real dy)
 { T=Point(T.x+dx, T.y+dy); }

 // Izracunava obim
 virtual Real perimeter () const {return 0;}
 // Izracunava površinu
 virtual Real area () const {return 0;}
 // Izracunava dijagonalu
 virtual Real diagonal () const {return 0;}
 // Izracunava poluprečnik upisanog kruga
 virtual Real r () const {return 0;}
 // Izracunava poluprečnik opisanog kruga
 virtual Real R () const {return 0;}

 // Cisto virtualna funkcija za ispis parametara figure
 // Definisana u izvedenim klasama
 virtual void draw () const {} = 0;

protected:
 // Težište figure
 Point T;
};

#endif

// Definicija složenog grafika geometrijskih figura (modul picture.h)

#ifndef _picture_h
#define _picture_h

#include "graphic.h"

```



```
class ListIterator;
class List;
```

```
class Picture : public Graphic {
public:
 Picture () {}
 virtual ~Picture();

 // Funkcija za manipulaciju "potomcima"
 void add (Graphic*);
 void remove (Graphic*);

 // Pravi iterator koji iterira kroz
 // listu grafika koji cine sliku
 ListIterator createIterator();

 // Funkcija za ispis parametara
 // grafika koji cine sliku
 virtual void draw ();

protected:
 void release();

private:
 // lista grafika koji cine sliku
 List myGraphics;

 friend class ListIterator;
};

#endif

// Modul picture.cpp

#include "picture.h"

Picture::~Picture() {
 release();
}

void Picture::release() {
 Graphic* g=(Graphic*)myGraphics.removeFirst();
 while (g->getNext()) {
 Graphic* tmp=g;
 g=(Graphic*)g->getNext();
 delete tmp;
 }
 delete (Graphic*)myGraphics.getLast();
}

void Picture::add(Graphic* g) {
 this->myGraphics.putLast(g);
}

void Picture::remove(Graphic* g) {
 this->myGraphics.remove(g);
}
```

```
ListIterator Picture::createIterator() {
 return this->myGraphics.createIterator();
}

void Picture::draw () {
 ListIterator i=this->createIterator();
 for (i.reset(); !i.isDone(); i.next()) {
 Graphic* g=(Graphic*)i.currentItem();
 g->Draw();
 }
}

// Definicija klase Circle (modul circle.h)

#ifndef _circle_h
#define _circle_h

#include "graphic.h"

static const Real Pi=3.1415926535;

class Circle : public Graphic {
public:
 // Konstruktor
 Circle(const Real r=1, const Point& t=ORG) : r0(r) {T=t;}

 // Izracunava obim
 virtual Real perimeter () const {return 2*Pi*r0;}
 // Izracunava povrstinu
 virtual Real area () const {return r0*r0*Pi;}
 // Izracunava dijagonalu
 virtual Real diagonal () const {return 2*r0;}
 // Izracunava poluprecnik upisanog kruga
 virtual Real r () const {return r0;}
 // Izracunava poluprecnik opisanog kruga
 virtual Real R () const {return r0;}

 // Funkcija za ispis parametara konkretne figure
 virtual void draw ();

protected:
 // Poluprecnik kruga
 Real r0;
};

#endif

// Definicija funkcija iz klase Circle (modul circle.cpp)

#include "circle.h"
#include <iostream.h>

void Circle::draw() {
 cout<<"\nKrug [T="<<T<<", r0="<<r0<<", d="<<diagonal();
 cout<<"\n", O="<<perimeter()<<"\n", P="<<area()<<"\n";
}
```

```
// Definicija klase Rectangle (modul rectangle.h)
#ifndef _rectangle_h
#define _rectangle_h
#include "graphic.h"
#include <math.h>

class Rectangle : public Graphic {
public:
 // Konstruktor
 Rectangle(Real aa=1, Point t=ORIGIN) : a(aa) { T=t; }

 // Izracunava obim
 virtual Real perimeter () const {return 4*a;}
 // Izracunava površinu
 virtual Real area () const {return a*a;}
 // Izracunava dijagonalu
 virtual Real diagonal () const {return sqrt(2)*a;}
 // Izracunava poluprecnik upisanog kruga
 virtual Real r () const {return a/2;}
 // Izracunava poluprecnik opisanog kruga
 virtual Real R () const {return diagonal()/2;}

 // Funkcija za ispis parametara konkretne figure
 virtual void draw ();

protected:
 // Stranica kvadrata
 Real a;
};
```

#endif

```
// Definicije funkcija iz klase Rectangle (modul rectangle.cpp)
#include "rectangle.h"
#include <iostream.h>
```

```
void Rectangle::draw() {
 cout<<"\nkvadrat [T="<<T<<" ("<<a<<"", d="<<diagonal()<<"",
 cout<<"", O="<<perimeter()<<"", P="<<area()<<"")\n";
}
```

```
// Modul figure_main.cpp
```

```
#include "graphic.h"
#include "picture.h"
#include "circle.h"
#include "rectangle.h"
```

```
void main () {
```

```
 Picture *root = new Picture;
```

```
 Graphic* k = new Circle(2);
 root->add(k);
```

```
 Picture* p1 = new Picture;
```

```
Graphic* r1 = new Rectangle(3);
p1->add(r1);

Graphic* r2 = new Rectangle(5);
p1->add(r2);

root->add(p1);

Picture* p2 = new Picture;

Graphic* r3 = new Rectangle(4);
p2->add(r3);

Graphic* c2 = new Circle(6);
p2->add(c2);

p1->add(p2);

root->draw();

delete root;
```

}

## Zadatak 2

Realizovati potrebne klase za implementaciju apstrakcije konačnog automata koristeći projektni obrazac *State*.

### Uputstvo

Konačni automat je apstrakcija kojom se opisuju dinamički aspekti (ponašanje) sistema pokretanih događajima. Automat definiše sekvencu stanja kroz koja prolazi jedan objekat tokom svog životnog veka i odgovarajuće aktivnosti koje preduzima prilikom promene stanja, kao odgovor na spoljašnje pobude (događaje). Kada se odgovarajući događaj desi, odigrava se određena akcija koja, pored pobude koja je izaziva, zavisi i od stanja u kome se objekat nalazio u trenutku pobuđivanja.

Upravo u slučajevima kada je potrebno implementirati objekat čije ponašanje zavisi od stanja u kome se on nalazi, i koje se mora menjati tokom izvršavanja u skladu sa promenama stanja, može se koristiti projektni obrazac *State*.

Ključna ideja ovog obrasca jeste uvođenje apstrakne klase *BaseState* koja reprezentuje stanja konačnog automata. Ova klasa je zajednički interfejs za sve klase koje predstavljaju različita stanja objekta. Ona implementira podrazumevano ponašanje za sve pobude. Klase koje se izvedu iz *BaseState* predstavljaju konkretna stanja u kojima se objekat može naći i svaka od njih implementira specifično ponašanje tog objekta pridruženo datom stanju.

Pored ovih klasa, obrazac predviđa i klasu *FSM* koja definiše interfejs automata (sve događaje koji se upućuju automatu, kao i sve tranzicije koje se vrše kao posledice događaja), i sadrži referencu (pokazivač) na instancu neke od konkretnih klasa izvedenih iz klase *BaseState*, koja predstavlja tekuće stanje automata. Kad god automat prelazi iz stanja u stanje, instanca klase *FSM* preusmerava tu referencu na odgovarajući objekat stanja.

Pošto se ponašanje objekta pridruženo nekom stanju enkapsulira u jednoj instanci neke od klasa izvedenih iz *BaseState*, dodavanje novih stanja i tranzicija je relativno lako, a postiže se samo definisanjem novih izvedenih klasa, što je prednost ovog obrasca. Takođe, uvođenjem pojedinačnih objekata za različita stanja, tranzicije (prelazi iz stanja u stanje) postaju mnogo očiglednije nego kada se tekuće stanje objekta definiše samo pomoću vrednosti nekih internih podataka, kada tranzicije nemaju eksplicitnu manifestaciju, nego se ispoljavaju samo kao dodeljivanja vrednosti određenim promenljivama.

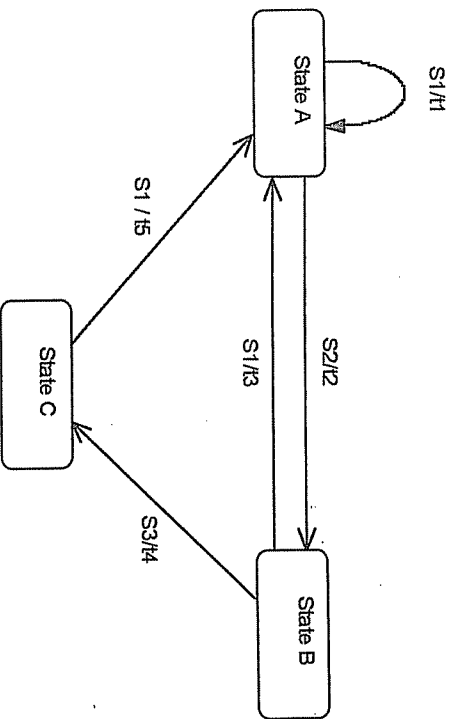
Model konkretnog automata koji treba implementirati prikazan je na donjoj slici.

S1, S2, S3 predstavljaju pobude koje se upućuju automatu i izazivaju tranzicije t1, t2, t3, t4, i t5.

U stanju *State A* automat reaguje na pobude s1 i s2.

U stanju *State B* automat reaguje na pobude s1 i s3.

U stanju *State C* automat reaguje samo na pobudu s1.



### Rešenje

```
// Modul states.h
#ifndef _states_h
#define _states_h
#include <iostream.h>

class FSM;
class StateA;
class StateB;
class StateC;
```

```
class BaseState {
public:
 BaseState(FSM* fsm) : myFSM(fsm) {}
 virtual BaseState* signal1() {return this;}
 virtual BaseState* signal2() {return this;}
 virtual BaseState* signal3() {return this;}
 virtual void entry() {}
 virtual void exit() {}
protected:
 FSM* fsm() const {return myFSM;}
private:
 FSM* myFSM;
};

class StateA : public BaseState {
public:
 StateA(FSM* fsm) : BaseState(fsm) {}
 virtual void exit() {cout<<"Exit StateA\n";}
 virtual void entry() {cout<<"Entry StateA\n";}
 virtual BaseState* signal1();
 virtual BaseState* signal2();
};

class StateB : public BaseState {
public:
 StateB(FSM* fsm) : BaseState(fsm) {}
 virtual void exit() {cout<<"Exit StateB\n";}
 virtual void entry() {cout<<"Entry StateB\n";}
 virtual BaseState* signal1();
 virtual BaseState* signal3();
};

class StateC : public BaseState {
public:
 StateC(FSM* fsm) : BaseState(fsm) {}
 virtual void exit() {cout<<"Exit StateC\n";}
 virtual void entry() {cout<<"Entry StateC\n";}
 virtual BaseState* signal1();
};
```

```

class FSM {
public:
 FSM();
 void signal1();
 void signal2();
 void signal3();

protected:
 friend class StateA;
 friend class StateB;
 friend class StateC;

 void transition1() { cout<<"Transition 1\n"; }
 void transition2() { cout<<"Transition 2\n"; }
 void transition3() { cout<<"Transition 3\n"; }
 void transition4() { cout<<"Transition 4\n"; }
 void transition5() { cout<<"Transition 5\n"; }

private:
 StateA stateA;
 StateB stateB;
 StateC stateC;

 BaseState* currentState;
};

#endif

// Modul states.cpp
#include "states.h"

FSM::FSM () : stateA(this), stateB(this), stateC(this),
 currentState(&stateA) {
}

void FSM::signal1() {
 currentState->exit();
 currentState=currentState->signal1();
 currentState->entry();
}

void FSM::signal2() {
 currentState->exit();
 currentState=currentState->signal2();
 currentState->entry();
}

void FSM::signal3() {
 currentState->exit();
 currentState=currentState->signal3();
 currentState->entry();
}

BaseState* StateA::signal1() {
 fsm()->transition1();
 return this;
}

BaseState* StateA::signal2() {
 fsm()->transition2();
 return &(fsm()->stateB);
}

BaseState* StateB::signal1() {
 fsm()->transition3();
 return &(fsm()->stateA);
}

BaseState* StateB::signal3() {
 fsm()->transition4();
 return &(fsm()->stateC);
}

BaseState* StateC::signal1() {
 fsm()->transition5();
 return &(fsm()->stateA);
}

// Modul states_main.cpp
#include <iostream.h>
#include "states.h"

void main () {
 FSM fsm;
 cout <<"\n";
 fsm.signal1();
 cout <<"\n";
 fsm.signal2();
 cout <<"\n";
 fsm.signal1();
 cout <<"\n";
 fsm.signal3();
 cout <<"\n";
 fsm.signal1();
 cout <<"\n";
 fsm.signal2();
 cout <<"\n";
 fsm.signal3();
 cout <<"\n";
 fsm.signal1();
 cout <<"\n";
 fsm.signal2();
 cout <<"\n";
 fsm.signal1();
 cout <<"\n";
}

```

**Jelena Marušić**

# **Integrirano razvojno okruženje Microsoft Visual C++**

## ***Uputstvo za korišćenje***

**Glava 10**

Uvod

**Glava 11**

Opšte manipulacije projektima

**Glava 12**

Uređivanje programa

**Glava 13**

Prevođenje programa

**Glava 14**

Debagovanje

## Uvod

U ovoj glavi biće dat kratak uvod u terminologiju okruženja Microsoft Visual C++, odnosno paketa Developer Studio, koji predstavlja zajedničko integrisano razvojno okruženje grupe Microsoftovih prevodilaca različitih programskih jezika.

## Osnovni pojmovi

U okruženju Microsoft Visual C++ (dalje kratko MSVC) razvijate jedan programski paket pomoću radnog prostora (engl. *workspace*) koji sadrži jedan ili više projekata.

*Projekat* (engl. *project*) se sastoji iz fajlova: *izvornih* (engl. *source*), *zaglavja* (engl. *header*), *resursnih* (engl. *resource*) i fajlova koji sadrže informacije o podešavanjima i konfiguraciji projekta (fajlovi svojstveni samom paketu MSVC). Programski kôd definisan unutar jednog projekta prevodi se u jedan nezavisni izvršni program ili biblioteku. Unutar jednog radnog prostora može postojati više projekata, recimo za različite aplikacije unutar jednog programskog paketa, ili za različite uslužne programe koji čine paket, za različite verzije programa, biblioteke itd.

*Fajl-zaglavje* (engl. *header file*) predstavlja interfejs jedne programske celine (modula) koji sadrži deklaracije elemenata definisanih u tom modulu koje koriste drugi moduli (deklaracije klase, funkcija, objekata, tipova, konstanti itd.). *Izvorni fajl* (engl. *source file*) sadrži implementaciju programskog modula. *Resursni fajl* (engl. *resource file*) sadrži definicije elemenata grafičkog interfejsa koje koristi Windows kada izvršava dati program (bit mape, meniji, dijalozi itd.).

Pravljenje novog programa počinjete stvaranjem novog radnog prostora i projekta. Kada želite da nastavite rad na programu, otvorite odgovarajući radni prostor (fajl sa nastavkom *dsw*), radije nego jedan po jedan fajl.

## Radno okruženje

Zone koje čine radno okruženje Developer Studio su sledeće:

- Na vrhu su glavni meni i kutija sa alatima.
- Sa leve strane nalazi se prozor radnog prostora (engl. *workspace*). Ovaj deo je ključ za kretanje kroz razne delove vašeg projekta. U sledećem odeljku naći ćete detaljnije objašnjenje.
- Sa desne strane je glavna radna površina, na kojoj uređujete fajlove.
- Na dnu su izlazni pano (engl. *output pane*) i statusna linija (engl. *status bar*). Izlazni pano možda neće biti vidljiv kada startujete program prvi put, ali se pojavljuje pri prvom prevodenju programa. Na izlaznom panou vidite poruke koje vam prevodilac šalje (npr. greške) pri prevodenju i povezivanju.

### Prozor radnog prostora

Ovaj prozor vam omogućava kretanje kroz delove projekta na tri različita načina:

**Class View** vam omogućava kretanje i manipulaciju izvorim C++ kodom na nivou klasa. On koristi različite ikone za predstavljanje privatnih, zaštićenih i javnih funkcija članica i atributa. Ako dvaput pritisnete mišem bilo koju stavku, otvara se odgovarajući fajl pri čemu je kursor automatski postavljen na odgovarajuće mesto. Npr. dvostruko pritiskanje mišem imena klase postavlja kursor na njenu deklaraciju, a dvostruko pritiskanje imena funkcije postavlja kursor na početak njene definicije. Dvostruko pritiskanje imena atributa postavlja kursor na mesto gde je on deklarisan.

Pritiskanje imena klase desnim tasterom miša otvara kontekstni meni koji vam omogućava da brzo nađete mesto gde je klasa definisana, da dodate funkciju ili podatak toj klasi, pogledate listu svih mesta gde je klasa referisana, njene odnose sa drugim klasama itd. Slično tome, pritiskanje imena funkcije desnim tasterom miša otvara kontekstni meni koji omogućava da nađete mesto gde je funkcija deklarisana odnosno definisana, da je obrišete, postavite tačku prekida (engl. *breakpoint*), pogledate listu svih mesta u kodu odakle se funkcija poziva itd. Pritiskanje imena atributa desnim tasterom otvara manji meni sličan opisanim.

**Resource View** vam omogućava pretraživanje i uređivanje raznih resursa aplikacije, kao što su dijalozi, ikone i meniji.

**File View** vam omogućava kretanje kroz fajlove od kojih se sastoji vaš aplikacija i njihovo otvaranje. Željani fajl otvarate dvostrukim pritiskom njegovog imena.

## Glava 11

# Opšte manipulacije projektima

## Započinjanje novog projekta

Biranjem opcije **File→New** otvara se dijalog sa sledećim stranama: *Files*, *Projects*, *Workspaces* i *Other Documents*. Na strani *Projects* birate vrstu projekta koji želite da napravite. U ovom trenutku značajni su *Win32 Console Application*, *Win32 Dynamic-Link Library* i *Win32 Static Library*. Unesite ime projekta u polje *Project name*. U polje *Location* upišite putanju do direktorijuma na disku gde želite da bude upisan novi projekat. Na strani *Workspaces* u početku ćete moći da pravite samo prazan radni prostor. Ako ne napravite radni prostor pri započinjanju projekta, MSVC će ponašati da napravi podrazumevani radni prostor za vas.

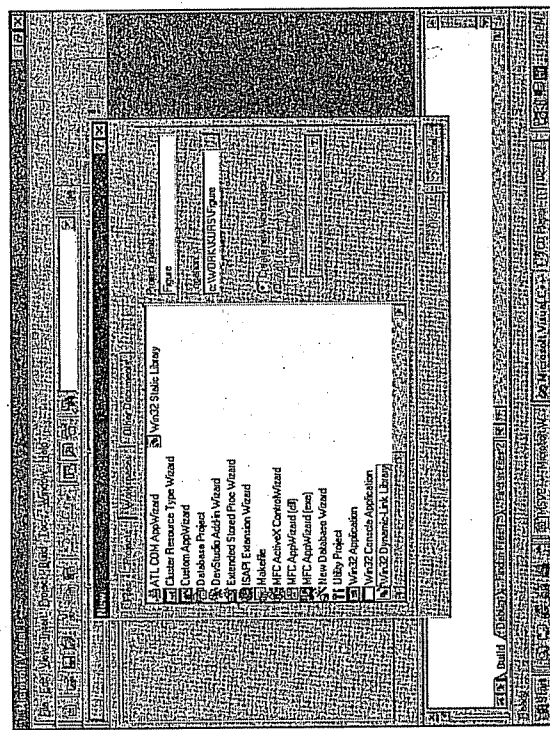
Kada pritisnete **OK**, pojavljuje se dijalog u kome možete da odaberete vrstu aplikacije koju želite da napravite:

- Prazan projekat znači upravo to – biće napravljen projekat koji neće sadržati nijedan fajl niti klasu.
- Ako odaberete jednostavnu aplikaciju, biće napravljen projekat koji sadrži neke početne fajlove, odnosno kosur aplikacije u koji dodajete kod.
- Ako odaberete tip aplikacije „Hello, world!“, MSVC će napraviti program koji, kada se startuje, štampa „Hello, world!“.
- U okviru ovog kursa nećemo opisivati tip aplikacije koji podržava korišćenje MFC (biblioteka klasa *Microsoft Foundation Classes*).

Kada odaberete tip aplikacije i pritisnete **Finish**, prikazaće se prozor koji sadrži informacije o projektu koji će biti napravljen. Pritisnite **OK** i pogledajte šta je MSVC napravio za vas.

### Korak po korak:

1. Odaberite **File→New**.
2. Odaberite stranu *Projects*.
3. Odaberite *Win32 Dynamic-Link Library*.
4. U polje *Project* upišite: *Figure*.
5. Pritisnite dugme pored polja *Location* da biste odabrali gde ćete da smestite svoj novi projekat ili upišite lokaciju direktno u polje predviđeno za to.
6. Pritisnite **OK**.
7. U dijalogu koji se pojavi odaberite *An empty DLL project* i pritisnite **Finish**.
8. Pritisnite **OK** u dijalogu koji prikazuje informacije o projektu koji će biti napravljen.



## Rad sa više projekata

MSVC nije ograničen na rad samo sa jednim projektom. Biranjem komande *Insert Project into Workspace* menija *Project* možete da dodate postojeći projekat u trenutno otvoren radni prostor.

Projekte možete dodati kao projekte najvišeg nivoa (engl. *top-level*) ili kao potprojekte (engl. *subprojects*). Glavna razlika je u tome što potprojekat učestvuje u procesu izgradnje (engl. *build*) svog natprojekta (njemu nadređenog) – kada se gradi *top-level* projekat, prvo se izgrade njegovi potprojekti.

### Korak po korak:

1. Odaberite *File* → *Close Workspace* ako je prethodno formirani radni prostor još otvoren.
2. Odaberite *File* → *New*.
3. U dijalogu *New* odaberite stranu *Project* i odaberite opciju *Win32 Console Application*.
4. U polje *Project* upišite *MojProgram*.
5. Odaberite lokaciju i pritisnite *OK*.
6. U dijalogu koji se pojavi odaberite opciju *A simple application* i pritisnite *Finish*.
7. U sledećem dijalogu pritisnite *OK*.
8. Odaberite *Project* → *Insert Project into Workspace* i pronađite prethodni projekat (fajl *Figure.dsp*).

## Osnovna podešavanja

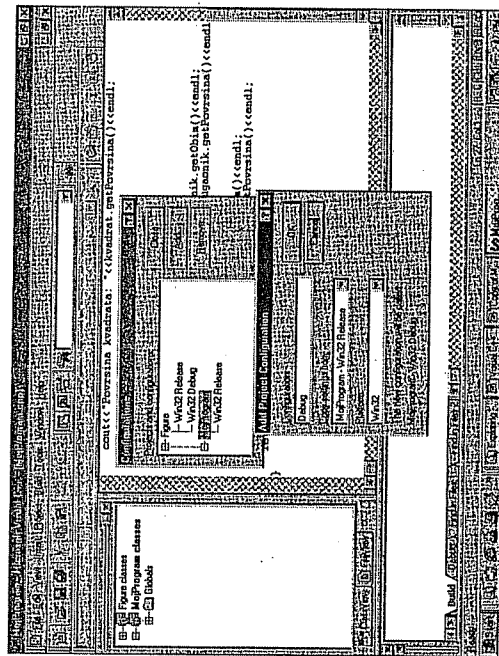
Konfiguracija projekta definiše kako će izvršni program ili biblioteka koje projekat predstavlja biti sagrađeni (engl. *build*). Kada pravite projekat pomoću nekog od čarobnjaka (engl. *wizard*), njemu će biti dodeljen skup podrazumevanih konfiguracija. Podrazumevane konfiguracije su:

- *Debug*: služi za pravljenje programa u radnoj fazi.
- *Release*: služi za konačno pravljenje programa (kada se smatra da je spreman za isporuku).

Konfiguracije možete dodati ili obrisati biranjem komande *Configurations* iz menija *Build*. Određena konfiguracija se podešava biranjem komande *Settings* iz menija *Project*. Na to ćemo se vratiti kasnije.

### Korak po korak:

1. Odaberite *Build* → *Configurations*.
2. Izaberite konfiguraciju *Win32 Debug* projekta *MojProgram* i pritisnite *Remove*.
3. Potvrdite da želite da uklonite tu konfiguraciju.
4. Pritisnite ime projekta *MojProgram* da biste ga izabrali.
5. Pritisnite *Add*.
6. U polje *Configuration* unesite *Debug*. Naravno, da bi konfiguracija služila za dobavljanje ona ne mora da se zove *Debug* – možete je nazvati kako god želite.
7. U listi *Copy settings from* izaberite *MojProgram* – *Win32 Release* (koji je, doduše, i jedini u listi, ali u opštem slučaju ćete imati priliku da izaberete konfiguraciju čija će se podešavanja kopirati). To znači da ne krećete iz početka kada pravite konfiguraciju – kasnije ćete moći da promenite podešavanja koja želite.
8. U opštem slučaju, u listi *Platform* nalaziće se samo *Win32*.
9. Pritisnite *OK* u dijalogu *Add Project Configuration*, i zatim *Close* u dijalogu *Configurations*.





## Uređivanje programa

### Dodavanje fajlova u projekat

Često ćete želeći da dodate postojeći fajl u aktuelan projekat. To možete da uradite biranjem Project → Add to Project → Files. Novi fajl ćete dodati biranjem Project → Add to Project → New i biranjem vrste fajla.

Fajlovi se mogu hijerarhijski organizovati unutar projekta, prema logičkoj organizaciji programa.

#### Korak po korak:

1. Zatvorite trenutno otvoren radni prostor (ako je otvoren) biranjem File → Close Workspace.
2. Odaberite File → New.
3. Odaberite stranu *Files* i odaberite *C/C++ Header File*.
4. U polje *File name* unesite Pravougaonik.
5. U polje *Location* upišite putanju projekta Figure ili pritisnite odgovarajuće dugme i pronađite direktorijum Figure.
6. Pritisnite *OK*.
7. Unesite sledeći kod u fajl Pravougaonik.h:

```
typedef unsigned int UINT;

class CPravougaonik {
public:
 CPravougaonik();
 virtual ~CPravougaonik();
private:
 UINT visina;
 UINT sirina;
};
```

8. Odaberite File → Save i zatvorite fajl biranjem File → Close.
9. Otvorite radni prostor MojProgram biranjem File → Recent Workspaces.
10. Odaberite Project → Add to Project → Files.
11. U dijalogu *Insert Files into Project* pronađite fajl Pravougaonik.h i pritisnite *OK*. Ako radni prostor sadrži više projekata, izabrani fajl će biti dodan u trenutno aktivan projekat (njegovo ime je prikazano polunimnim slovima u prozoru radnog prostora). Ako želite da drugi projekat bude aktivan, pritisnite desnim tastrom miša njegovo ime i u kontekstnom meniju odaberite *Set as Active Project*.

Ako želite da u projekat dodate nov fajl, izaberite File → New.

### Korak po korak:

1. Odaberite File→New.
2. Odaberite stranu *Files* i odaberite C++ *Source File*.
3. U polje *File name* unesite Pravougaonik i odaberite istu lokaciju kao za Pravougaonik.h.
4. Potvrdite *Add to project*, a u listi ispod odaberite projekat Figure.
5. Unesite sledeći kôd u Pravougaonik.cpp:

```
#include "Pravougaonik.h"
CPravougaonik::CPravougaonik()
{
 sirina=8;
 visina=5;
}

CPravougaonik::~CPravougaonik()
{
 sirina=0;
 visina=0;
}
```

### Osnovni alati

*ClassWizard* predstavlja jednu od najmoćnijih alata MSVC okruženja. Njegove najvažnije funkcije tiču se rada sa MFC projektima, ali i u radu sa konzolnim aplikacijama biće od koristi (mada neuporedivo manje). Nešto više o tome saznate kasnije u ovom poglavlju.

*WizardBar* omogućava brz pristup mnogim funkcijama *ClassWizarda*. Iz prve liste možete odabrati bilo koju klasu iz aktuelnog projekta, iz druge birate filter, a iz treće funkciju te klase koju želite da vidite ili menjate. Dugme *WizardBar Actions* omogućava vam brz pristup brojnim komandama (spisak tih komandi zavisi od tipa projekta, kao i od toga da li je trenutno odabrana klasa ili globalna funkcija itd.).

*SourceBrowser* omogućava brzo nalaženje definicije i reference simbola i traženje veza između simbola. Ovaj alat zahteva prisustvo fajla koji sadrži potrebne informacije. Da biste imali ovaj fajl morate da promenite podešavanja svog projekta. Najlakši način je da, kada ste upozoreni da taj fajl ne postoji (dešava se kada odaberete neku od komandi koje pokreću *SourceBrowser*, npr. *References* iz kontekstnog menija podatka člana), dozvolite MSVC-u da ga "automatski napravi". Drugi način je ručno podešavanje: odaberite komandu *Settings* iz menija *Project*, odaberite stranu *Browse Info* i potvrdite *Build browse info file*.

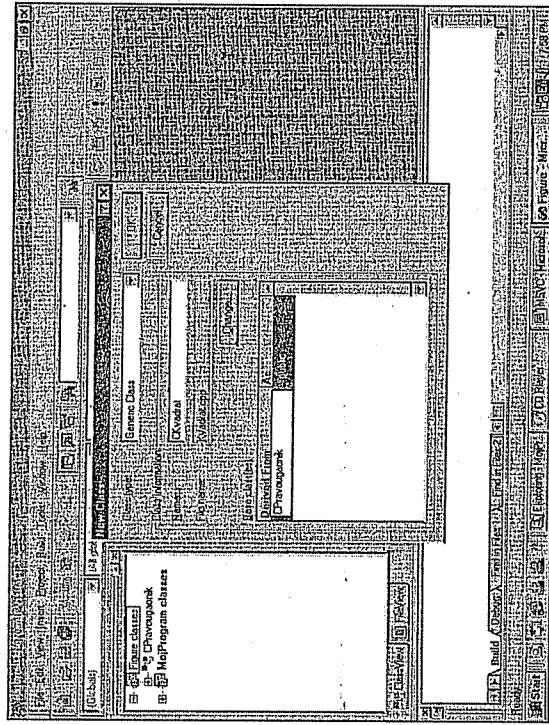
### ClassWizard

U osnovi, *ClassWizard* je alat za automatsko generisanje klase. Pokreće se biranjem komande *New Class* bilo iz *WizardBar Actions* (o tome kasnije) ili iz menija *Insert*. Takođe se može pokrenuti biranjem komande *ClassWizard* iz menija *View* (ako je komanda omogućena).

Kada se otvori dijalog *New Class*, unesite ime klase i odaberite njenu osnovnu klasu iz spiska. Ukoliko se klasa iz koje želite da izvedete novu klasu ne nalazi na spisku, to ćete rešiti ručnim menjanjem koda. Kada pritisnete *OK*, *ClassWizard* će generisati željenu klasu i automatski dodati konstruktor i destruktor.

### Korak po korak:

1. U prikazu *Class View* pritisnite desnim tasterom miša projekat Figure.
2. U kontekstnom meniju odaberite *New Class*.
3. U dijalogu *New Class*, u polje *Name* unesite CKvadrat.
4. Pritisnite mišem ispod *Derived From* i unesite CPravougaonik.
5. Ispod *As* će se pojaviti reč *public*, što je podrazumevan način izvođenja. Neka ostane tako.
6. Pritisnite *OK*.



### Korak po korak:

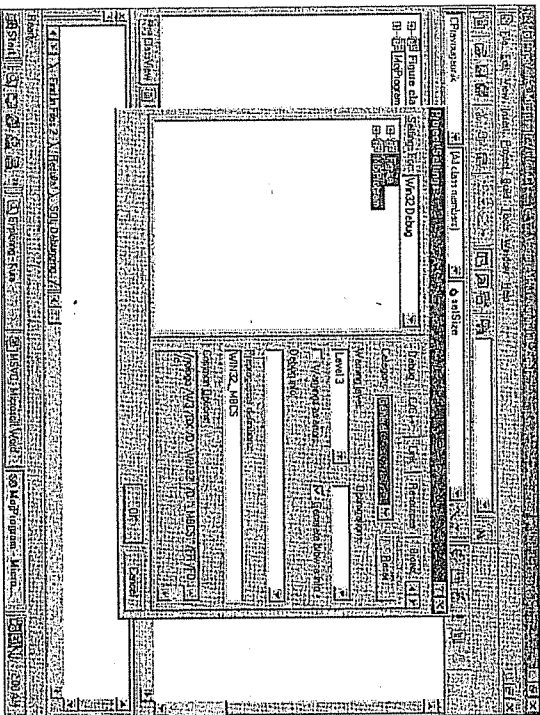
1. U prikazu *Class View* pritisnite desnim tasterom miša ime klase CPravougaonik i odaberite *Add Member Function*.
2. U dijalogu *Add Member Function* unesite void u polje *Function Type*, a u polje *Function Declaration* unesite: setVelicina(UINT a, UINT b).
3. U opcijama *Access* odaberite *public*.
4. Pritisnite *OK*.

### Korak po korak:

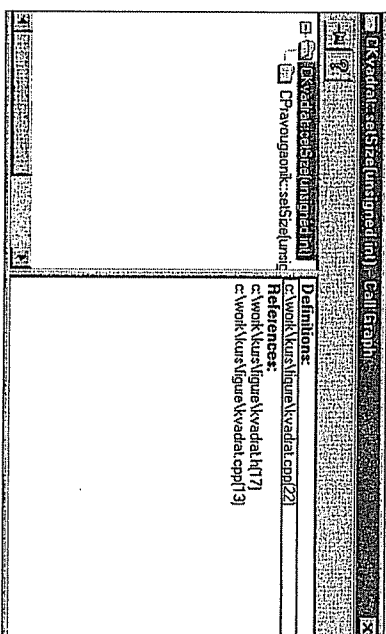
1. Ako projekat Figure nije trenutno aktivan, aktivirajte ga.
2. Iz prve liste *WizardBar* odaberite klasu CPravougaonik.
3. Iz druge liste odaberite *All class members*.
4. Iz treće liste odaberite setVelicina. Otvoriće se odgovarajući fajl (Pravougaonik.cpp), sa kursorom postavljenim na početak definicije funkcije i obeleženim potpisom (engl. *signature*) funkcije.

**Korak po korak:**

1. Odaberite komandu *Project*→*Settings*.
2. Otvorite se dijalog *Project Settings* sa oba projekta izabrana sa leve strane. Neka ostane tako.
3. Iz liste sa leve strane odaberite *Win32 Debug*. Kako su oba projekta izabrana, i oba imaju konfiguraciju koja se zove *Win32 Debug*, sva podešavanja koja budete promenili sa desne strane odražiće se na oba projekta.
4. Odaberite stranu *C/C++*, odaberite *General* iz liste *Category* i potvrdite *Generate browse info*.
5. Odaberite stranu *Browse Info* i potvrdite *Build browse info file*.



6. Da biste mogli da koristite *SourceBrowser*, morate da odaberete komandu *Build*→*Rebuild All*, koja će iz početka prevesti sve fajlove i povezati projekte sa novim podešavanjima.
7. U prikazu *ClassView* pritisnite desnim tasterom miša ime funkcije članice *setSize* klase *CKVadrat* i odaberite opciju *References*, *Calls* ili *Called by* iz kontekstnog menija. Pri tom imajte u vidu da *SourceBrowser* možete koristiti samo za trenutno aktivan projekat.
8. Prozor koji se pojavio sadrži sa leve strane informacije o tome odakle se poziva funkcija *CKVadrat::setSize*, ili o funkcijama koje ona poziva, u zavisnosti od toga koju ste komandu menija izabrali. Sa desne strane nalazi se ime fajla u kome je funkcija definisana, a ispod toga spisak fajlova u kojima se ona spominje. Pritisnite dvaput bilo koji fajl sa spiska (sa desne strane), ili bilo koju funkciju (sa leve strane).

**Uređivanje koda**

Ako ste ikada koristili program za obradu teksta, onda ste spremni da koristite MSVC editor. Unošenje, izmena, brisanje, kopiranje teksta itd. isti su kao u drugim editorima teksta, npr. Microsoft Wordu.

Međutim, kako je MSVC okruženje za programiranje, ima i neke specifične mogućnosti. Ako pritisnete desnim tasterom miša bilo gde u okviru fajla koji menjate, otvoriće se kontekstni meni koji sadrži sledeće komande:

- *Cut* iseca odabrani tekst i kopira ga na Clipboard.
- *Copy* kopira odabrani tekst na Clipboard.
- *Paste* zamenjuje odabrani tekst tekстом koji je trenutno na Clipboardu. Ako nije odabran nikakav tekst, ubacuje sadržaj Clipboarda na mesto gde se trenutno nalazi kursor.
- *Insert File Into Project* dodaje fajl koji menjate u trenutno otvoren projekat.
- *Check Out* – ovu komandu nećemo opisivati.
- *Open* otvara fajl iznad čijeg imena se nalazi kursor miša. Ovo je posebno korisno ako želite da otvorite zaglavlje.
- *List Members* prikazuje listu podataka i funkcija članica objekta iznad čijeg imena se nalazi kursor miša.
- *Type Info* prikazuje mali prozor koji vas podseća na tip podatka iznad čijeg imena se nalazi kursora miša.
- *Parameter Info* podseća vas na parametre koje prima funkcija iznad čijeg imena se nalazi kursor miša.
- *Complete Word* pomaže u vezi s imenom promenljive ili funkcije koje je delimično otkucano.
- *Go To Definition* otvara fajl u kome je definisano ime iznad kojeg se nalazi kursor.
- *Go To Reference* postavlja kursor na mesto gde se sledeći put referiše ime iznad kojeg se nalazi kursor.

- *Insert/Remove Breakpoints* postavlja tačku prekida (engl. *breakpoint*) na mesto gde se nalazi kursor, ili je uklanja ako ona već postoji. Više o tome naći ćete u poglavljima koja se bave debugovanjem.
  - *Enable Breakpoint* omogućava onemogućenu tačku prekida.
  - *Class Wizard* otvara dijalog *Class Wizard*.
  - *Properties* otvara listu svojstava.
- Sve ove komande imaju ekvivalente u meniju i kutiji sa alatima.

## Sintaksno bojenje

MSVC označava elemente koda sintaksnim bojenjem. Podrazumevane boje su: crna za kôd koji se prevodi, zelena za komentare i plava za rezervisane reči. Stringovima, brojevima itd. možete da dodelite druge boje tako što ćete odabrati komandu *Options* iz menija *Tools* i odabrati stranu *Format*.

## Vežba

Koristeći što više *Class Wizard*, uradite sve što treba da bi klase *CPravougaonik* i *CKvadrat* i funkcija *main* projekta *MojProgram* izgledale ovako:

```
//Pravougaonik.h
typedef unsigned int UINT;

class CPravougaonik {
public:
 CPravougaonik();
 virtual ~CPravougaonik();

 UINT getObim();
 UINT getPovrsina();
 void setVelicina(UINT a, UINT b);

private:
 UINT visina;
 UINT sirina;
};

//Pravougaonik.cpp
#include "Pravougaonik.h"

CPravougaonik::CPravougaonik()
{
 sirina=8;
 visina=5;
}

CPravougaonik::~CPravougaonik()
{
 sirina=0;
 visina=0;
}

void CPravougaonik::setVelicina(UINT a, UINT b)
{
 sirina=a;
 visina=b;
}
```

```
UINT CPravougaonik::getPovrsina()
{
 return sirina*visina;
}

UINT CPravougaonik::getObim()
{
 return 2*(sirina+visina);
}

// Kvadrat.h
...
class CKvadrat : public CPravougaonik {
public:
 CKvadrat();
 virtual ~CKvadrat();
 void setVelicina(UINT a);
};
...

// Kvadrat.cpp
#include "Kvadrat.h"
CKvadrat::CKvadrat()
{
 setVelicina(5);
}

CKvadrat::~CKvadrat()
{
}

void CKvadrat::setVelicina(UINT a)
{
 CPravougaonik::setVelicina(a,a);
}

// MojProgram.cpp
...
int main(int argc, char* argv[])
{
 CPravougaonik pravougaonik;
 CKvadrat kvadrat;
 UINT a;
 UINT b;
 cout<<endl;
 cout<<"Obim pravougaonika: "<<pravougaonik.getObim()<<endl;
 cout<<"Povrsina pravougaonika: "<<pravougaonik.getPovrsina()<<endl;
 cout<<endl;
 cout<<"Obim kvadrata: "<<kvadrat.getObim()<<endl;
 cout<<"Povrsina kvadrata: "<<kvadrat.getPovrsina()<<endl;
 a=12;
 b=9;
 pravougaonik.setVelicina(a,b);
 kvadrat.setVelicina(a);
 cout<<endl;
 cout<<"Obim pravougaonika: "<<pravougaonik.getObim()<<endl;
 cout<<"Povrsina pravougaonika: "<<pravougaonik.getPovrsina()<<endl;
 cout<<endl;
 cout<<"Obim kvadrata: "<<kvadrat.getObim()<<endl;
 cout<<"Povrsina kvadrata: "<<kvadrat.getPovrsina()<<endl;
 return 0;
}
```

## Prevođenje programa

### Prevođenje

Svaki izvorni fajl u projektu mora biti najpre preveden (engl. *compiled*), da bi se potom moglo preći na povezivanje (engl. *linking*). Kada radite na većem projektu biće zamorno da, svaki put kada promenite kôd, sve prevodite iz početka. Najbrži način je da ponovo prevodate samo fajlove u koje ste uneli promene.

Fajl ćete prevesti biranjem komande *Compile nekiFajl.cpp* iz menija *Build*. Ovde *nekiFajl.cpp* predstavlja fajl koji je trenutno otvoren u editoru.

U toku prevođenja, MSVC šalje poruke o greškama i upozorenjima na već pomenuti izlazni pano (engl. *output pane*). Pod greškom (engl. *error*) se podrazumeva nešto što nije moglo da se prevede jer nije ispravno sa stanovišta sintakse ili semantike jezika C++. Upozorenje (engl. *warning*) se odnosi na nešto što nije greška sa stanovišta samog jezika, tj. može da se prevede, ali MSVC smatra da tu postoji mogućnost semantičke greške u programu. Na primer, deklarisi ste promenljivu, ali je posle niste koristili – to nije greška, ali MSVC vas na to upozorava, jer sumnja da ste možda nešto zaboravili. Nakon izlistavanja postojećih grešaka i upozorenja (ukoliko broj grešaka ne prelazi određenu granicu posle koje prevodič prekida prevođenje), na izlaznom panou se pojavljuje broj grešaka i upozorenja. Ako je program sintaksno ispravan, a MSVC ga smatra i semantički korektnim, javiće: 0 grešaka, 0 upozorenja.

### Korak po korak:

1. Pritisnite desnim tasterom miša ime projekta Figure i odaberite *Set as Active Project*.
2. Otvorite fajl *Pravougaonik.cpp* tako što ćete dvaput pritisnuti njegovo ime u prikazu *File View*.
3. Odaberite *Build* → *Compile Pravougaonik.cpp*. Korak 1 je bio neophodan jer ne možete posebno prevesti fajl koji nije deo trenutno aktivnog projekta. Imajte na umu da se fajlovi zaglavljaju ne prevode.

### Za vežbu:

Prevedite na isti način i *Kvadrat.cpp* i *MojProgram.cpp*. Zašto MSVC prijavljuje greške? Šta nedostaje?

## Pravljenje projekta

Pravljenje projekta znači prevođenje i povezivanje svih njegovih komponenti da bi se proizvela ciljna verzija izvršnog programa ili biblioteke. Svaki projekat ima svoj *make* fajl, koji sadrži podatke o zavisnostima između pojedinih fajlova. Kada se neki fajl promeni, onda svi fajlovi koji zavise od njega moraju ponovo da se prevedu i svi fajlovi povežu.

Kada želite da napravite program, odaberite komandu *Build nekiProjekat* iz menija *Build*, gde *nekiProjekat* predstavlja trenutno aktivan projekat. Biranjem ove komande, MSVC će prevesti samo one fajlove koji su promenjeni posle prethodnog prevođenja, zatim fajlove koji su od njih zavisni, i uradiće povezivanje.

Ako želite da se svi fajlovi bezuslovno prevedu i povežu, odaberite komandu *Rebuild All* iz menija *Build*. Ovo je potrebno uraditi kada se promene neke opcije prevođenja od kojih zavise svi delovi programa, pa ne bi bilo dobro da neki fajlovi budu prevedeni sa starijim, a neki sa novim podešavanjima.

### Korak po korak:

1. Aktivirajte projekat Figure.
2. Odaberite *Build* → *Build Figure.dll*.

### Vežba

Probajte da napravite *MojProgram.exe*. Greškama koje MSVC prijavljuje pozabavićemo se kasnije.

## Dobijanje pomoći u radu

Dok je aktivan prozor editora, postavite kursor na mesto gde se nalazi reč koja vas zanima i pritisnite taster F1. Automatski će se pokrenuti MSDN (*Microsoft Developer Network*, kolekcija članaka koji se tiču MSVC) sa spiskom članaka u kojima se sponinje odabrana reč.

Drugi način da dobijete pomoć jeste tzv. *Info View*, koji je deo prozora *Workspace*. Ako odaberete prikaz *Info View*, u ovom prozoru će se prikazati kolekcija članaka vezanih za MSVC i *Developer Studio*.

Ne treba zaboraviti na gorespomenute funkcije, npr. *Parameter Info*, *Complete Word* itd.

### Vežba

U prethodnoj vežbi MSVC je prijavio mnoštvo grešaka u povezivanju. Pritisnite dvaput neku od tih poruka u izlaznom prozoru dok se ista ta poruka ne pojavi na statusnoj liniji, a zatim pritisnite F1. Pažljivim čitanjem fajla koji se pojavio pokušajte da dokučite šta je izazvalo grešku.

## Podešavanja

MSVC obezbeđuje podešavanje mnoštva opcija prevođenja i debugovanja. Odaberite komandu *Settings* menija *Project*.

Na levoj strani dijaloga *Project Settings* videćete listu svih konfiguracija projekta. Na desnoj strani možete izabrati određenu stranicu i tako pristupiti mnogim opcijama projekta.

Ako želite da podesite neku konfiguraciju, prvo morate da je izaberete na levoj strani. Ako želite da definišete podešavanja za određeni fajl projekta, odaberite ga na levoj strani tako što ćete raširiti stablo pritiskom znaka plus pored imena konfiguracije.

Prva strana je strana *General*. Ovde možete definisati da li vaš program koristi MFC i kako (mi ga za sada nećemo koristiti). Takođe, na ovoj strani možete izabrati direktorijume u kojima će biti smešteni tzv. među-fajlovi i krajnji fajlovi vašeg projekta.

Druga strana, *Debug*, definiše opcije debugovanja. Biranjem kategorije *General* ili *Additional DLLs* videćete da ova strana ima dve podstrane. Za nas je od značaja polje *Executable for debug session*. Ako je vaš projekat standardni izvršni program, onda nema razloga da se sadržaj ovog polja razlikuje od imena programa. Ali, na primer, ako radite na DLL projektu, definisanjem izvršnog programa moći ćete i njega da debugujete.

Treća strana, *C/C++*, omogućava vam da podesite opcije prevodioca. Ova strana ima više podstrana koje se menjaju biranjem *Category*.

Četvrta strana, *Link*, omogućava pristup raznim opcijama povezivanja.

Da sponenemo još šestu stranu, *Browse info*. Ako želite da koristite mogućnosti pretraživanja koda (engl. *browse*), potvrdite *Build browse info file*.

## Debagovanje

MSVC debager se aktivira kada izvršavate program u režimu debagovanja (engl. *debug mode*). To se postiže biranjem neke od komandi podmenija *Start debug* koji se nalazi u meniju *Build*. Komande su: *Go*, *Step Into*, *Run to Cursor*.

Ali, pre nego što se pokrene debager, aplikacija mora biti prevedena tako da sadrži informacije potrebne za debagovanje (engl. *debug info*).

### Pripremanje aplikacije za debagovanje

Ako ste projekat napravili pomoću nekog od čarobnjaka, verovatno nećete morati ništa da menjate – konfiguracija *Debug* je jedna od podrazumevanih. Ali, ako morate sami da napravite ovu konfiguraciju, to ćete uraditi podešavanjem u dijalogu *Project settings*. Ovaj dijalog se otvara biranjem komande *Settings* menija *Project*.

Kada ste pokrenuli dijalog, odaberite stranu *C/C++*. Odaberite kategoriju *General* i sa leve strane odaberite konfiguraciju za koju želite da bude *debug* konfiguracija. Sada uradite sledeće: u polju *Debug info* odaberite *Program Database*, a u polju *Optimizations* odaberite *Disable (Debug)*.

Još je bitno da projekat bude povezan sa *debug* verzijom biblioteke *C Run-time Library*. Odaberite kategoriju *Code Generation* a zatim i željenu *debug* biblioteku.

Treba još podesiti povezivanje. Odaberite stranu *Link* i u polju *Category* odaberite *General*. Sada potvrdite *Generate debug info*.

### Korak po korak:

1. Odaberite komandu *Project* → *Settings*.
2. Pošto smo obrisali samo konfiguraciju *Debug* za projekat *MojProgram* (u poslednjem ponovo napravili, ali sa drugačijim podešavanjima), izaberite na desnoj strani samo taj projekat.
3. Odaberite stranu *C/C++* i odaberite kategoriju *General*. Iz liste *Optimizations* odaberite *Disable (Debug)*, a iz liste *Debug info* stavku *Program Database for Edit and Continue*. Zatim odaberite kategoriju *Code Generation* i iz liste *Use run-time library* stavku *Debug Single-Threaded*.
4. Odaberite stranu *Link* i potvrdite *Generate debug info* i *Link incrementally*.
5. Pritisnite *OK*.

6. Da biste se osigurali da će sledeći put biti sagrađena *Debug* konfiguracija projekta, odaberite komandu *Build* → *Set Active Configuration*. U dijalogu *Set Active Project Configuration* odaberite konfiguraciju *MojProgram - Win32 Debug* i pritisnite *OK*.
7. Da bi projekat bio kompletno spreman za debugovanje, treba ga ponovo izgraditi sa novim podešavanjima. Ako projekat *MojProgram* nije trenutno aktivan, aktivirajte ga i odaberite komandu *Build* → *Rebuild All*.

## Pokretanje aplikacije u režimu debugovanja

Pošto ponovo prevedete celu aplikaciju nakon svih podešavanja, možete je pokrenuti biranjem bilo koje komande iz podmenija *Start Debug* menija *Build*.

Kada započnete debugovanje, pojaviće se neki od prozora debagera. Takođe će nestati neki prozori koji su inače vidljivi. Glavni meni će se promeniti i meni *Build* će biti zamenjen menijem *Debug*.

Aplikacija koju pokrećete u režimu debugovanja izvršava se dok ne stigne do tačke prekida (engl. *breakpoint*) ili je ne prekinete u izvršavanju biranjem *Stop debugging* ili *Break*.

Tokom debugovanja, *MSVC* prikazuje informacije u nizu prozora. Ako neki od njih nisu vidljivi, mogu se prikazati biranjem odgovarajuće komande podmenija *Debug Windows* menija *View*.

## Prozori debagera i pregled objekata

Prozor *Variables* prikazuje vrednosti objekata u tekućoj funkciji. Ovaj prozor ima tri strane:

- Strana *Auto* prikazuje objekte koji se koriste u tekućem i prethodnom redu programskog koda.
- Strana *Locals* prikazuje objekte koji su lokalni u tekućoj funkciji, uključujući i argumente funkcije.
- Strana *This* prikazuje podatke članove objekta na koji pokazuje *this* pokazivač tekuće funkcije članice.

Prozor *Variables* takođe može da prikaže objekte u oblasti važenja funkcija koje su pozvale tekuću funkciju.

Ovaj prozor može da se koristi za modifikovanje vrednosti prethodnih objekata ugrađenog tipa. Da biste promenili takvu vrednost, pritisnite je dvaput u prozoru *Variables*. Ako je modifikacija moguća, pojaviće se kursor tastature.

Prozor *Watch* se koristi za proveru vrednosti izraza. Izraz unesite u polje *Name*. Ovaj prozor ima četiri iste strane, tako da u isto vreme možete pratiti vrednosti četiri različita izraza. Isto kao i prozor *Variables*, i on se može koristiti za modifikovanje vrednosti.

Postoje još dva načina pregleda promenljivih: *QuickWatch* i *DataTips*.

*DataTips* liči na *ToolTips*. Za vreme debugovanja, postavite i zadržite par sekundi kursor miša iznad imena objekta, i njegova vrednost će se prikazati (ako ta vrednost može da se odredi). Za izračunavanje vrednosti izraza koristite dijalog *QuickWatch* koji ćete otvoriti komandom *QuickWatch* menija *Debug*. Ako se kursor trenutno nalazi iznad imena objekta, njegova vrednost se automatski pojavljuje u dijalogu *QuickWatch*. Ako je obeležen izraz, pojavljuje se njegova vrednost.

Prozor *Registers* prikazuje trenutne vrednosti u registrima procesora.

Prozor *Memory* vam omogućava da pratite sadržaj memorije u adresnom prostoru procesa koji trenutno debugujete.

Prozor *Call Stack* prikazuje istoriju poziva funkcija koja je dovela do poziva tekuće funkcije. Dvostruko pritisicanje funkcije u ovom prozoru promenice sadržaj ostalih *debug* prozora, tako da odgovaraju toj funkciji.

Prozor *Disassembly* obezbeđuje pogled na generisani asemblerski kôd. Dok ovaj prozor ima fokus, izvršavanje korak po korak drugačije funkcioniše: umesto da prolazi kroz redove C++ koda, prolazi kroz zasebne asemblerske instrukcije.

## Tačke prekida

Najjednostavniji način da postavite tačku prekida (engl. *breakpoint*) jeste da postavite kursor na željeno mesto u kodu i pritisnete *F9*. Pojaviće se kružić sa leve strane reda.

Mnogo bolja kontrola postiže se biranjem *Edit* → *Breakpoints*. Biranjem ove komande pojavljuje se dijalog *Breakpoints*, gde možete da postavite tri različite vrste tačaka prekida:

- *Location breakpoints* Prekidaju izvršavanje programa na određenoj lokaciji. Ovo su tačke prekida koje postavljate sa *F9*. Možete dodati i proveru uslova na tački prekida, tako da se izvršavanje programa ne prekida svaki put, već samo kada je ispunjen taj uslov.
- *Data breakpoints* Prekidaju izvršavanje programa kada se promeni vrednost određenog izraza.
- *Message breakpoints* Prekidaju izvršavanje programa kada neka od vaših procedura dobije određenu poruku.

## Komande za izvršavanje korak po korak

Komanda *Step Into* izvršava sledeći red vašeg koda, ili sledeću instrukciju u prozoru *Disassembly*. Ako je u tom redu poziv funkcije, ovom komandom se ulazi u telo te funkcije.

Komanda *Step Into Specific Function* koristi se radi kontrole u koju se funkciju ulazi u slučaju ugneženih poziva funkcija u jednom redu koda.

Komanda *Step Over* je slična komandi *Step Into*, osim što se ne ulazi u telo funkcije ako je u tom redu koda poziv funkcije, već se samo izvrši poziv kao jedinstvena akcija.

Komanda *Step Out* nastavlja izvršavanje programa do kraja tekuće funkcije, izlazi iz nje i prekida izvršavanje iza reda u kome je ta funkcija pozvana.

Komanda *Run to Cursor* nastavlja izvršavanje programa do reda gde se trenutno nalazi kursor tastature ili dok se ne nađe na tačku prekida.

Dok izvršavanje programa stoji, možete pomoću komande *Set Next Statement* zadati naredbu gde izvršavanje treba da se nastavi.



**Korak po korak:**

1. Otvorite fajl `MojProgram.cpp`.
2. Postavite kursor tastature na prvi red koda u funkciji `main`.
3. Pritisnite `F9`.
4. Odaberite komandu `Build` → `Start Debug` → `Go`.
5. Kada se program pokrene u režimu debugovanja i stane kod tačke prekida koju ste postavili, pritisnite `F11` (ili odaberite `Debug` → `Step Into`). Pritisnite `F11` i pratite kuda prolazi izvršavanje programa. Kada želite da se izvršavanje ne nastavi u telu funkcije koja se poziva u tekućem redu koda, pritisnite `F10`. Dok teče izvršavanje programa, pratite šta se dešava u vidljivim prozorima. Postavite kursor miša iznad imena objekta čija vas trenutna vrednost zanima.
6. Kada izvršavanje programa stigne do drugog reda koda funkcije `main`, pa se nakon lančanih poziva funkcija nađe u funkciji članici `CPravougaonik::setVelicina()`, odaberite komandu `View` → `Debug Windows` → `Call Stack`. Proučite sadržaj tog prozora i uklonite ga. Zatim pritisnite `SHIFT+F11` da bi se izvršavanje programa nastavilo izvan tekuće funkcije.
7. Kada izvršavanje programa stane nakon postavljanja promenljivih `a` i `b` u funkciji `main`, proverite vrednost izraza `a*2*a*b+b*b` koristeći prozor *Watch*.
8. Kada izvršavanje programa stane na kvadrat `setVelicina(a)`, promenite vrednost promenljive `a`, tako što ćete pritisnuti polje *Value* u prozoru *Variables* i uneti željenu vrednost. Imajte u vidu da je `a` označen ceo broj, pa i vrednost koju zadajete mora biti odgovarajućeg tipa.

